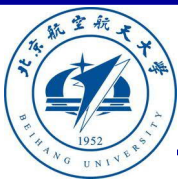


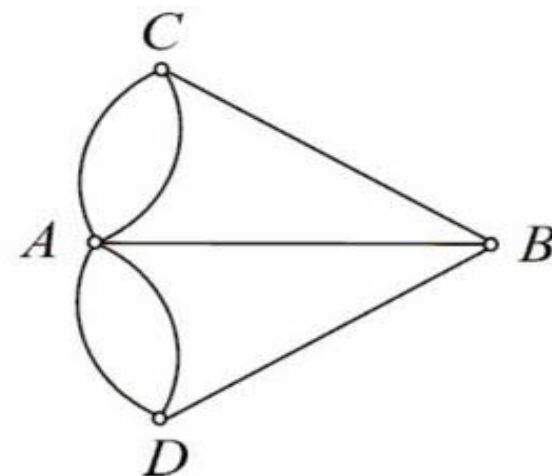
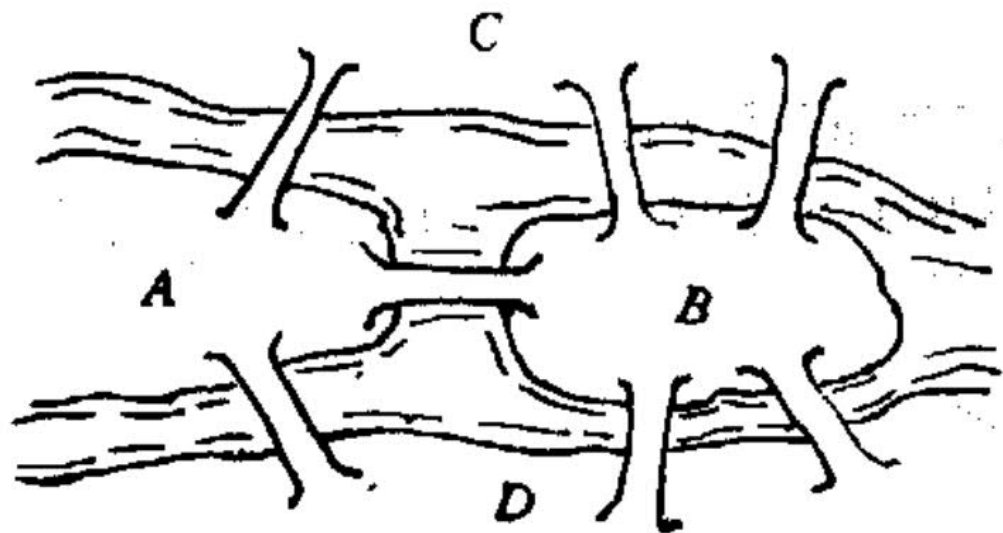
第11章 图的基本问题

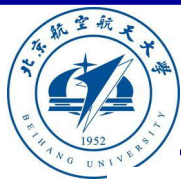
北京航空航天大学
《离散数学》课程组
2025-10



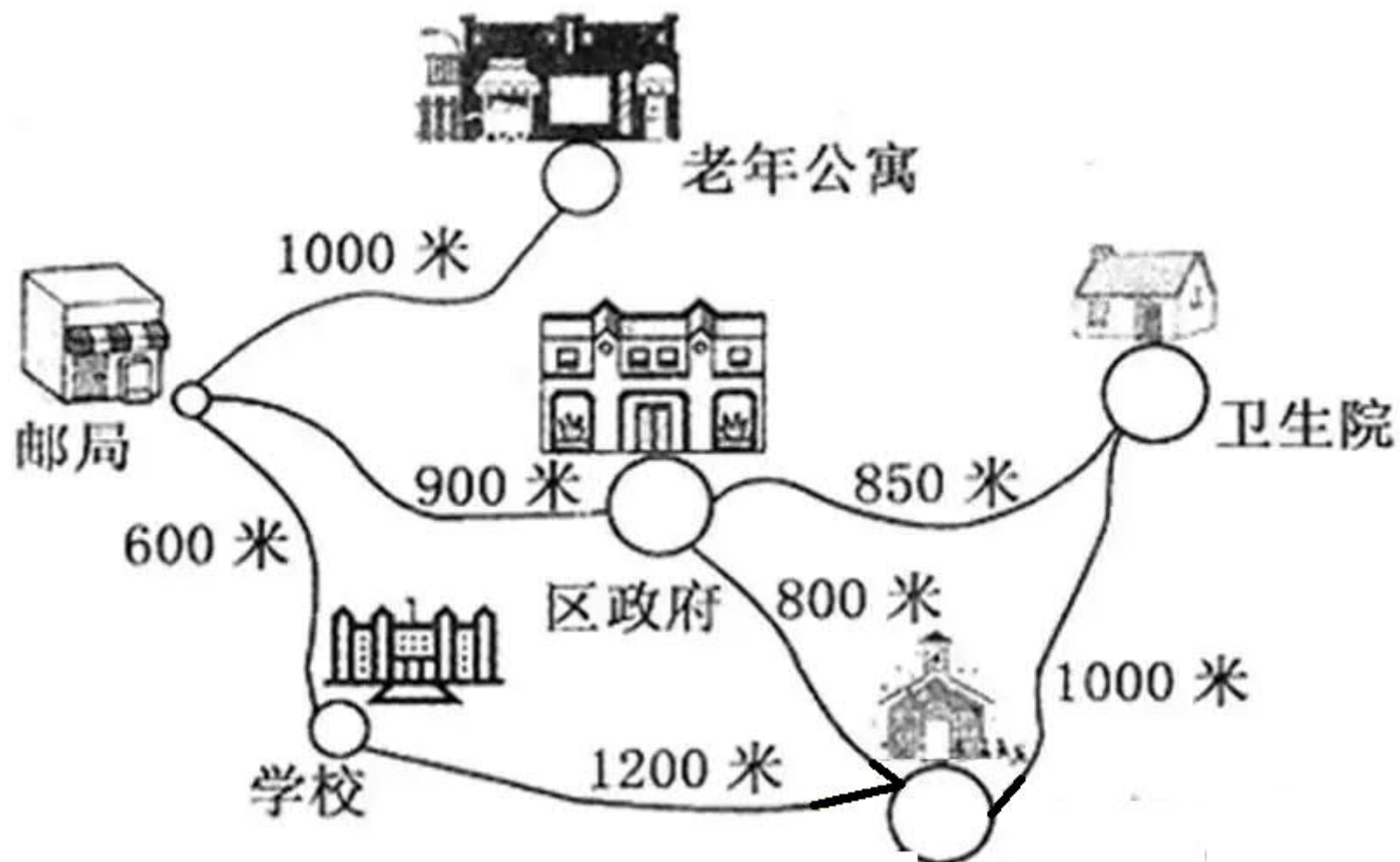
图的基本问题

- **哥尼斯堡七桥问题**。18世纪，哥尼斯堡为东普鲁士的首府，有一条横贯全市的普雷格尔河，河中的两个岛与两岸用七座桥联结起来。





图的基本问题





本章内容

- 穿程问题
 - 欧拉图
 - 哈密顿图
 - 骑士周游
- 通路问题
 - 最短通路
 - 关键通路
- 匹配问题
 - 二分图
 - 二分图匹配



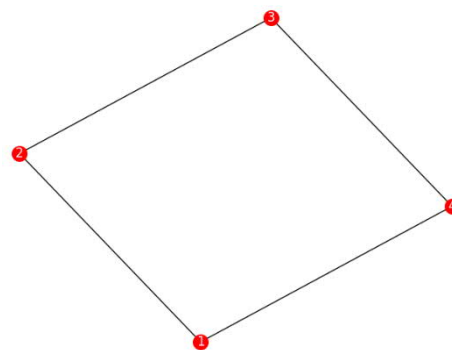
欧拉图问题提出

- 设 $G = \langle V, E \rangle$ 是连通无向图

$$V = \{1, 2, 3, 4\}, E = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$$

- V 中每个节点出现一次且仅一次
- E 中每个边出现一次且仅一次

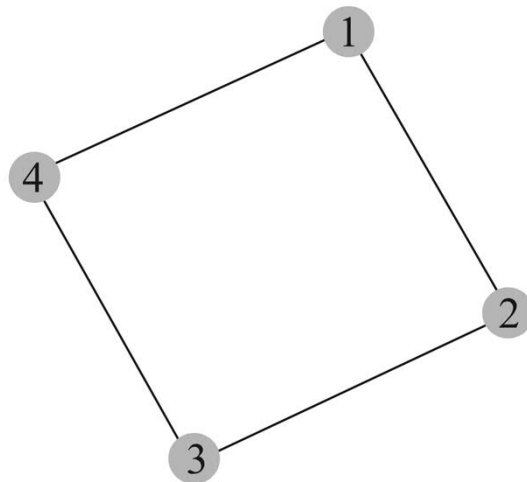
$V = \{1, 2, 3, 4\}$
 $E = \{(1, 2), (2, 3), (3, 4), (4, 1)\}$
`drawgraph(E)`





穿程问题

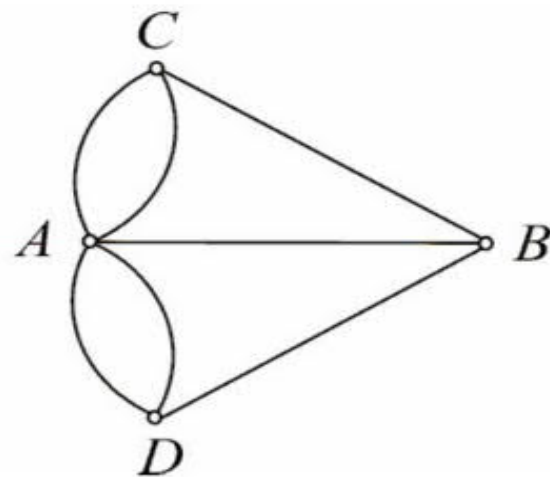
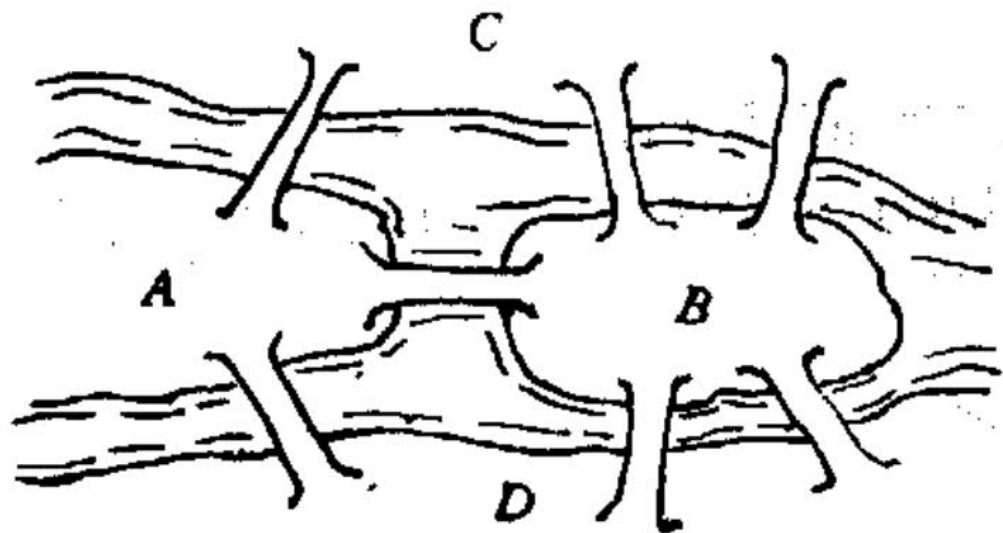
- 对于图 G ，如何从一个指定节点出发，恰好经过**每条边**一次且仅一次，并且回到该点？
- 同理，如何从一个指定节点出发，恰好经过**每个节点**一次且仅一次，即遍历每个节点一次且仅一次，并且回到该点？
- 针对图 G ，遍历每条边，或遍历每个节点的问题，称为**穿程问题**。



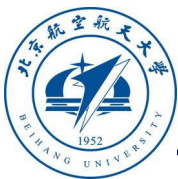


哥尼斯堡七桥问题

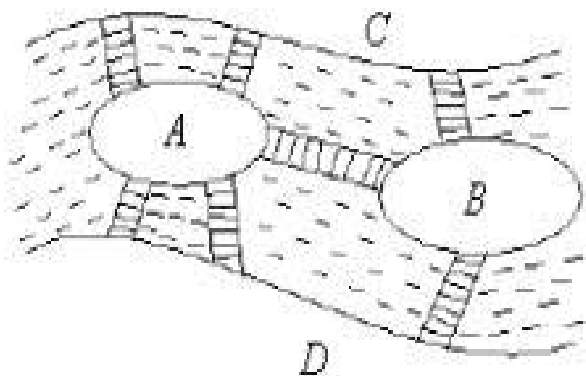
- **哥尼斯堡七桥问题**。18世纪，哥尼斯堡为东普鲁士的首府，有一条横贯全市的普雷格尔河，河中的两个岛与两岸用七座桥联结起来。



- $V = \{A, B, C, D\}$
- $E = \{(A, C, 1), (A, C, 2), (A, D, 1), (A, D, 2), (A, B, 1), (C, B, 1), (D, B, 1)\}$

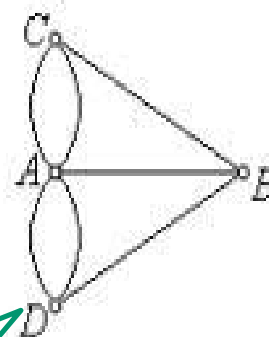


哥尼斯堡桥问题 (Königsberg bridges problem)



(1)

既无欧拉回路，
又无欧拉路径



(2)

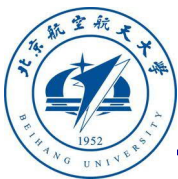


从四块陆地的任何一块出发，怎样通过每座桥恰巧一次，最终回到出发地点？（即找包含所有边的简单回路）

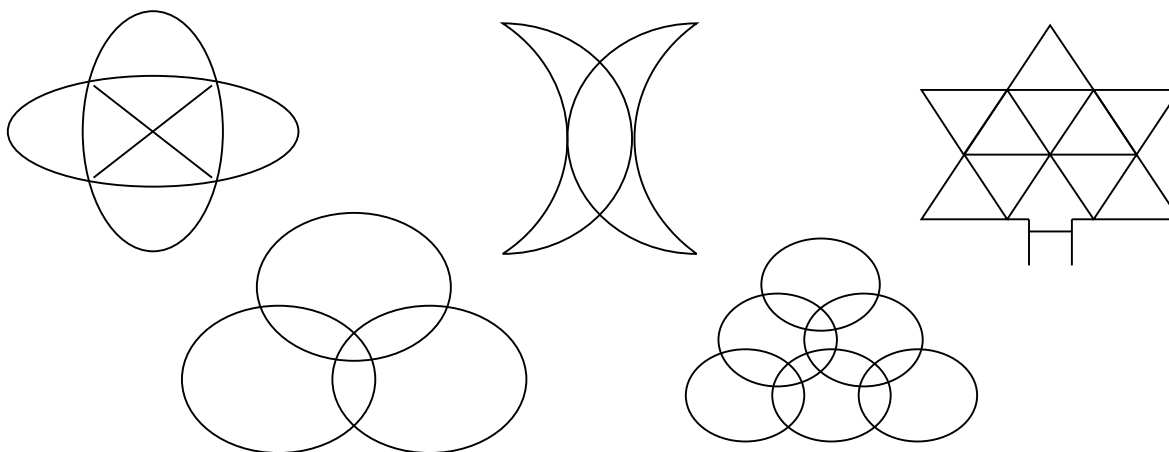
Euler 1736

瑞士数学家

证明不可能



一笔画



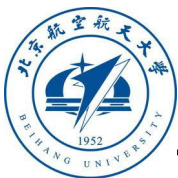
一张连通图能由一笔画出来的充要条件是：

- ✓ 每个交点处的线条数都是偶数；或 （欧拉回路）
- ✓ 恰有两个交点处的线条数是奇数。 （欧拉链）



欧拉图

- **定义11.2.1** : 设 $G = \langle V, E \rangle$ 是连通无向图, 经过 G 中 **每条边一次且仅一次** 的简单回路称为 **欧拉圈**, 也称为欧拉回路。具有欧拉圈的图称为 **欧拉图**。
- **定义11.2.2** : 设 $G = \langle V, E \rangle$ 是连通无向图, 经过 G 中 **每条边一次且仅一次** 的简单通路称为 **欧拉链**。
- $\forall e_k (e_k \in E \rightarrow e_k \in \langle e_1, \dots, e_n, e_1 \rangle)$
- $\forall e_i \forall e_j (e_i \in \langle e_1, \dots, e_n, e_1 \rangle \wedge e_j \in \langle e_1, \dots, e_n, e_1 \rangle \rightarrow e_i \neq e_j)$



判断有圈的图

```
import graph.graph as gt
```

```
E1={('a','b'),('b','c'),('c','d'),('d','a')}
```

```
E2={('a','b'), ('c','d'),('a','e'),('b','e'),('c','e'),('d','e')}
```

```
E3={('a','b'),('b','c'),('c','d'),('d','a'),('a','c'),('b','e'),('c','e')}
```

```
E4={('a','b'),('b','c'),('c','d'),('d','a'),('a','e'),('b','e'),('c','e'),('d','e')}
```

```
gt.drawgraph(E1)
```

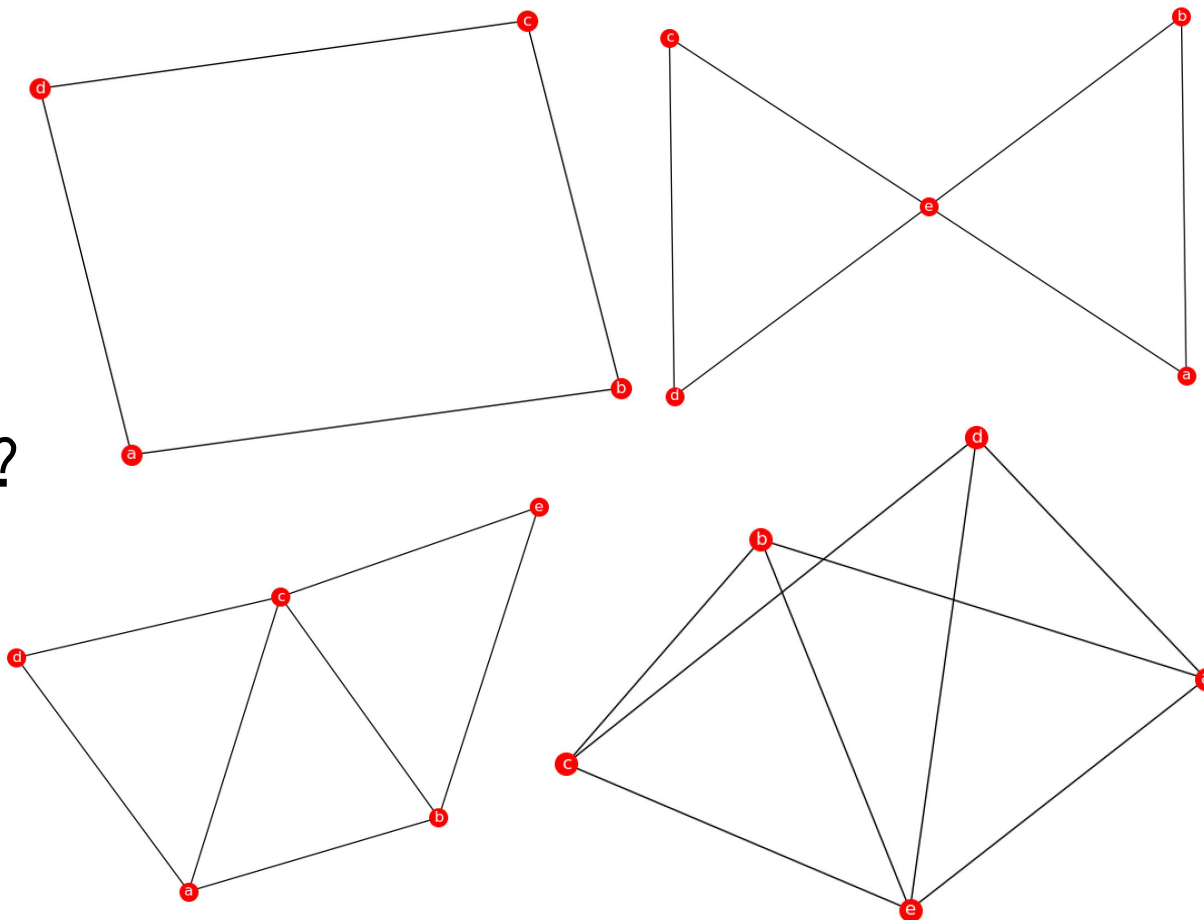
```
gt.drawgraph(E2)
```

```
gt.drawgraph(E3)
```

```
gt.drawgraph(E4)
```

观察四个图的特点：

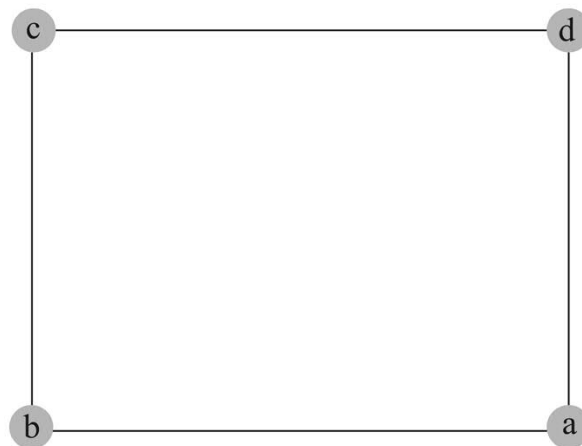
- 哪个图有欧拉圈？
- 哪个图有欧拉通路？
- 哪个图既无欧拉圈也无欧拉通路？



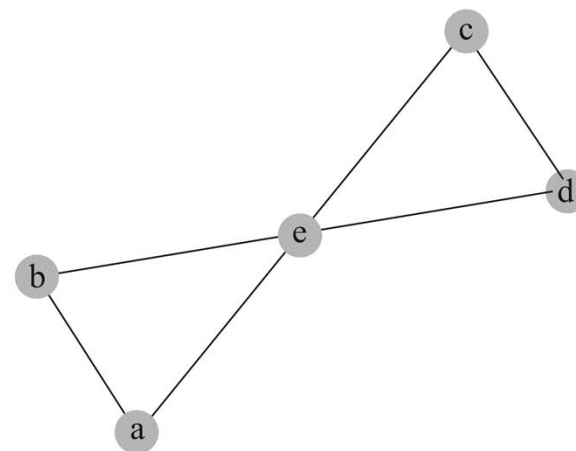


判断有圈的图

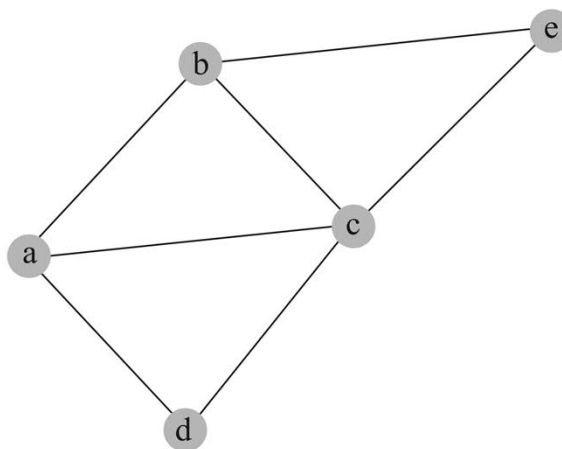
- G_1 有欧拉 $abcd a$
- G_2 有欧拉 $abecdea$
- G_3 有欧拉链 $abcdaceb$, 而没有欧拉圈
- G_4 既没有欧拉圈, 也没有欧拉链



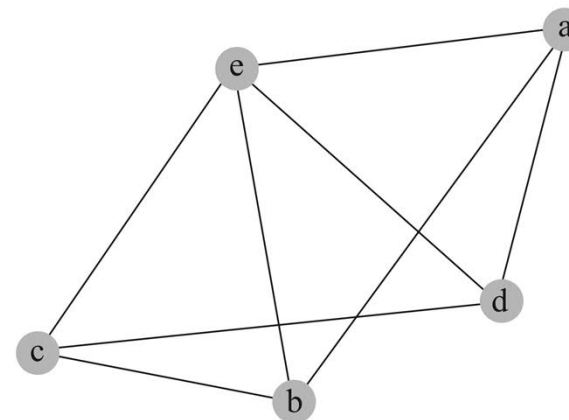
G_1



G_2



G_3



G_4



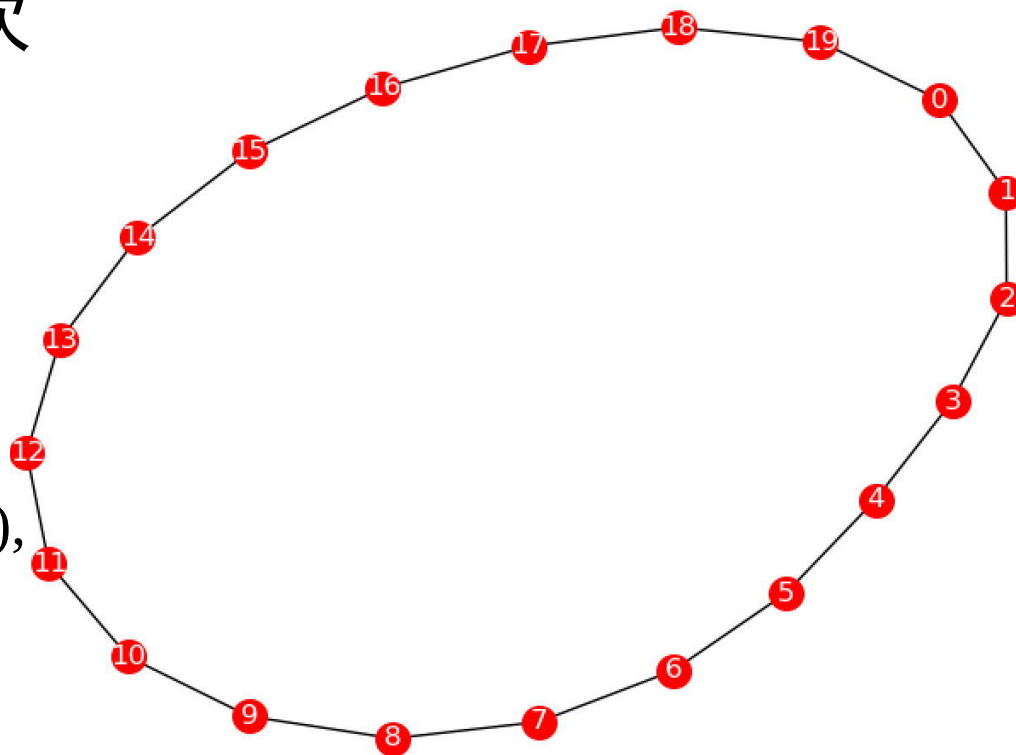
欧拉图

- 在 G 中，经过每条边一次且仅一次
- 相同节点可能通过多次
- 节点度有何特点？

$$G = (V, E)$$

$V = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19\}$

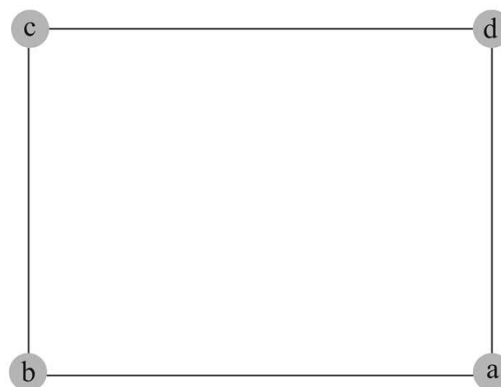
$E = [(0, 1), (1, 2), (2, 3), (3, 4), (4, 5), (5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12), (12, 13), (13, 14), (14, 15), (15, 16), (16, 17), (17, 18), (18, 19), (19, 0)]$



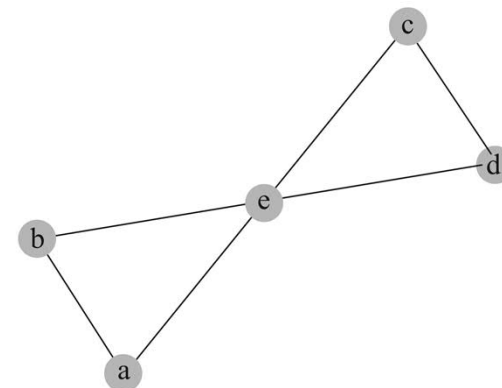


判断有圈的图

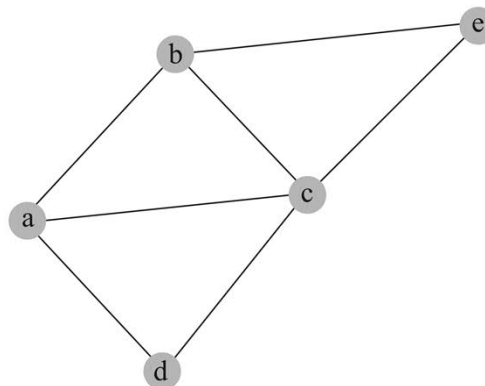
- G_1 有欧拉圈 $abcda$
所有节点的度数都是偶数
- G_2 有欧拉圈 $abecdea$
所有节点的度数都是偶数
- G_3 有欧拉链 $abcdaceb$,
而没有欧拉圈
两个节点度数为奇数
其他节点度数为偶数
- G_4 既没有欧拉圈, 也
没有欧拉链
一个节点度数为偶数
其他节点度数为奇数



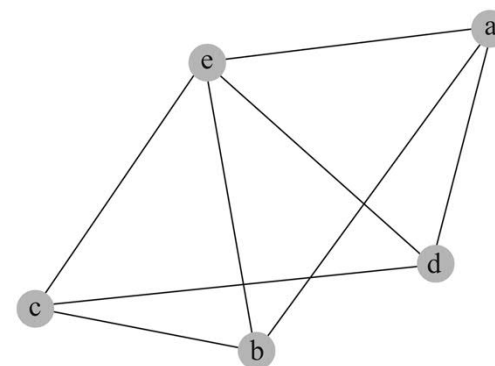
G_1



G_2



G_3

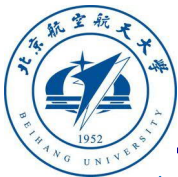


G_4



定理

- **定义11.2.3** 设 P 是 $u_0, u_1, \dots, u_{n-1}$, 并且没有更长路径包含路径 P , 则路径 P 称为**极大路径**。
- 设 G 是无向连通图, 则如下三个命题等价
 - G 是欧拉图。
 - G 中所有节点的度数是偶数。
 - G 是若干不重边圈的并。
- **定理11.2.1** 如果 $G = \langle V, E \rangle$ 是连通图, 每个节点的度数是偶数, 则图 G 是欧拉图。



定理

- **定理11.2.1** 如果 $G = \langle V, E \rangle$ 是连通图，每个节点的度数是偶数，则图 G 是欧拉图。

- **证明：**若 G 是平凡图，则结论成立。
 - 对边数 m 做数学归纳
 - $m = 1$ 时， G 为一个环，则 G 为欧拉图。



定理

■ **定理11.2.1** 如果 $G = \langle V, E \rangle$ 是连通图，每个节点的度数是偶数，则图 G 是欧拉图。

■ **证明：**

- 设 $m \leq k$ ， $k > 1$ 时，结论为真
 - $m = k + 1$ 时，证明如下：
 - 因为 G 连通且每个节点度数为偶数，所以 G 中必含圈。
- 不妨设 C 为 G 中一个圈，删除 C 上所有边，得生成子图 G' 。

定理 图 G 不是非循环图当且仅当 G 有子图 G' ，使得对于 G' 的任意结点 v ，皆有 $d_{G'}(v) > 1$ 。



定理

■ **定理11.2.1** 如果 $G = \langle V, E \rangle$ 是连通图，每个节点的度数是偶数，则图 G 是欧拉图。

■ **证明：**

- 设 $m \leq k$ ， $k > 1$ 时，结论为真
- $m = k + 1$ 时，证明如下：

➤ 因为 G 连通且每个节点度数为偶数，所以 G 中必含圈。

不妨设 C 为 G 中一个圈，删除 C 上所有边，得生成子图 G' 。

设 G' 有 n 个连通分支 $G'_1, G'_2, \dots, G'_n$ ，每个分支至多有 k 条边，且每个节点度数为偶数。

设 C 与 G'_i 有公共节点为 v_{ij}

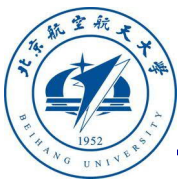
由归纳假设可知 G'_i 是欧拉图，因而 G'_i 存在一条欧拉回路。

在 C 上，从节点 v 开始，最后回到 v_{ij} ，得到一条扩展的欧拉回路。

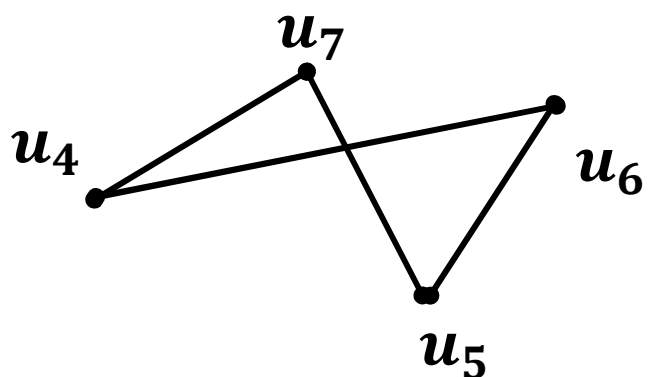
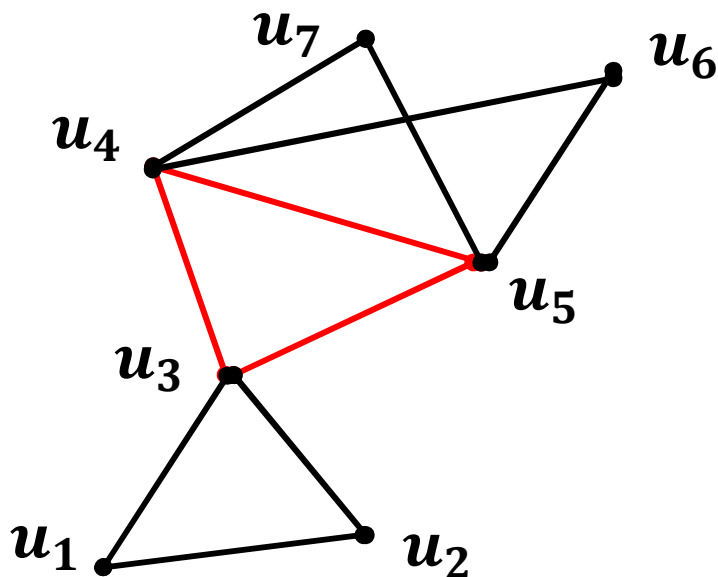
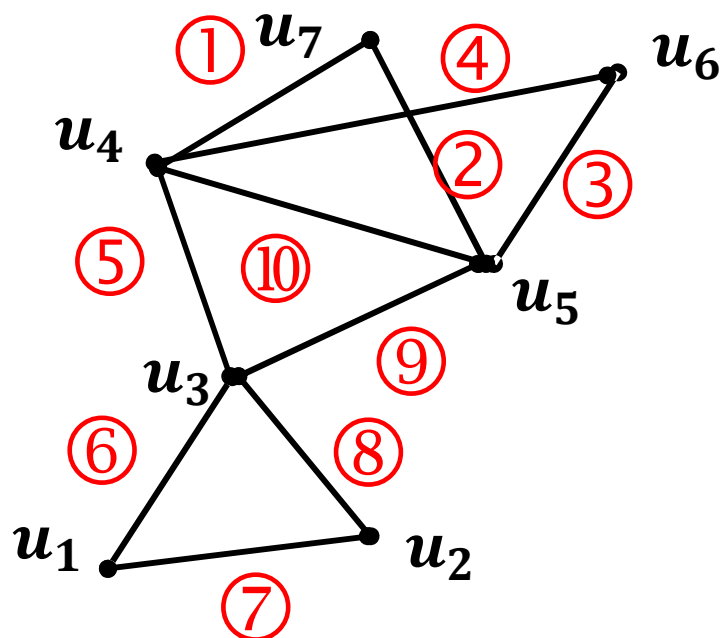
依此将 $G'_1, G'_2, \dots, G'_n$ 的所有子欧拉图并入 C ，形成一条欧拉回路。

所以， G 是欧拉图。证毕。

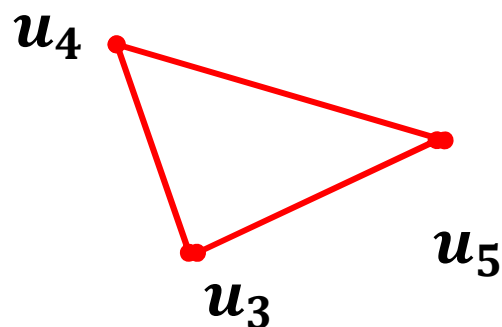
从以上证明不难看出：欧拉图是若干个边不交的圈的并集。



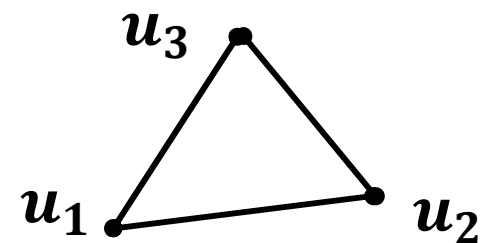
欧拉图



$u_4 u_7 u_5 u_6 u_4$



$u_4 u_3 u_5 u_4$



$u_3 u_1 u_2 u_3$

$u_4 u_7 u_5 u_6 u_4 u_3 u_1 u_2 u_3 u_5 u_4$



定理

- **定理 11.2.2** 设图 G 是欧拉图，则 G 是连通图，并且 G 的每一个节点都是偶节点。

证明：若图 G 不是连通图，则图 G 不是欧拉图

因此，若图 G 是欧拉图，则图 G 连通。

若图 G 是欧拉图，即图 G 可以组成一个圈。

在圈上每个边出现一次且仅一次。但节点可出现多次。

在圈上，从一个节点开始，节点每出现一次，该节点的度数增加2

因此，回到开始节点，每个节点度数分别合计为偶数。



序列图时欧拉图

- 节点 u 的度等于其入度+出度
- $d_i(v) = |\{u | (u, v) \in E\}|$, $d_o(v) = |\{v | (u, v) \in E\}|$

```
def degreeset(V,E):
```

```
    V=sorted(V)
```

```
    m=len(V)
```

```
    di=[0]*m
```

```
    do=[0]*m
```

```
    d=[0]*m
```

```
    for (u,v) in E:
```

```
        i=V.index(u)
```

```
        j=V.index(v)
```

```
        di[j]+=1
```

```
        do[i]+=1
```

```
        d[i]+=1
```

```
        d[j]+=1
```

```
    return [d,di,do]
```

```
import graph.graph as gg
```

```
m=3
```

```
[V,E]=gg.sequence01graphset(m)
```

```
[d,di,do]=gg.degreeset(V,E)
```

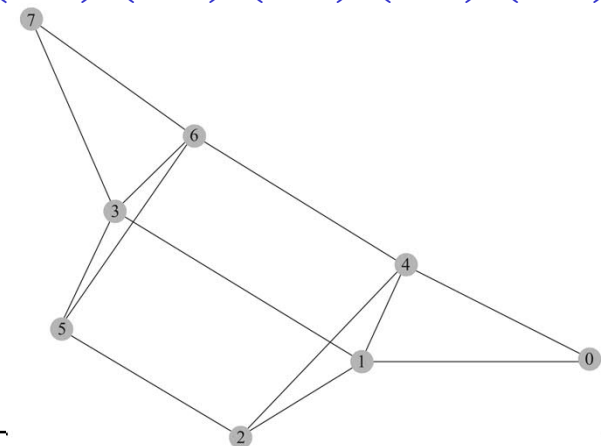
```
gg.drawgraph(E)
```

```
print("V,E",V,E)
```

```
print(d)
```

```
V,E {0, 1, 2, 3, 4, 5, 6, 7} {(0, 1), (2, 4), (1, 2), (4, 0), (7, 7),  
(6, 5), (0, 0), (4, 1), (3, 7), (6, 4), (7, 6), (3, 6), (5, 3), (2, 5),  
(1, 3), (5, 2)}
```

```
[4, 4, 4, 4, 4, 4, 4, 4]
```





定理

- **定理 11.2.3** G 是欧拉图，当且仅当 G 是若干个边不相交的圈的并。
- **定理 11.2.4** 无向图 $G = \langle V, E \rangle$ 中有连接 u_i 和 u_j 的欧拉链，当且仅当， G 是连通图，并且 G 中只 u_i 和 u_j 是奇节点。
- $E = E(G) \cup \{(u_i, u_j)\}$



证明1

- **定理** 设 $G = \langle V, E \rangle$ 为连通无向图, $v_1, v_2 \in V$ 且 $v_1 \neq v_2$ 。则 G 有一条从 v_1 至 v_2 的欧拉链当且仅当 G 恰有两个奇结点 v_1 和 v_2 。

证明: 令 $e = (v_1, v_2)$, 则

G 有一条从 v_1 至 v_2 的欧拉链 iff

$G' = G + \{e\}$ 有一条欧拉回路 iff

G' 是欧拉图 iff

G' 中每个结点都是偶结点 iff

G 中恰有两个奇结点 v_1 和 v_2



证明2

- **定理** 设 $G = \langle V, E \rangle$ 为连通无向图, $v_1, v_2 \in V$ 且 $v_1 \neq v_2$ 。则 G 有一条从 v_1 至 v_2 的欧拉链当且仅当 G 恰有两个奇结点 v_1 和 v_2 。

证明: 令 $e = (v_1, v_2)$, 若 $e \notin E$, 则

G 有一条从 v_1 至 v_2 的欧拉链 iff

$G' = G + \{e\}$ 有一条欧拉回路 iff

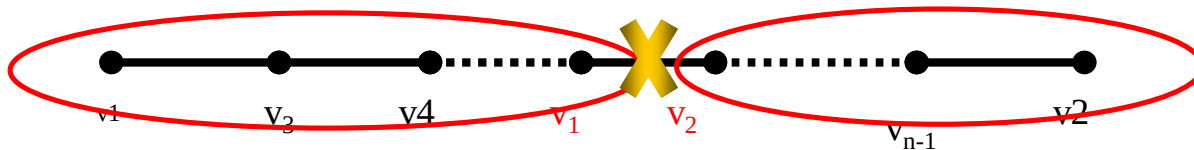
G' 是欧拉图 iff

G' 中每个结点都是偶结点 iff

G 中恰有两个奇结点 v_1 和 v_2



证明2



- **定理** 设 $G = \langle V, E \rangle$ 为连通无向图, $v_1, v_2 \in V$ 且 $v_1 \neq v_2$ 。则 G 有一条从 v_1 至 v_2 的欧拉链当且仅当 G 恰有两个奇结点 v_1 和 v_2 。

证明: 若 $e = (v_1, v_2) \in E$, 则

G 有一条从 v_1 至 v_2 的欧拉链 $v_1, v'_1, \dots, v'_n v_2$

考虑图 $G' = G - \{e\}$

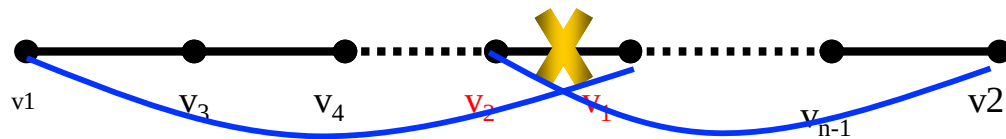
则可证 $G - \{e\}$ 有两个圈 $v_1 v'_1 \dots v'_i v_1$ 和 $v_2 v'_{i+2} \dots v'_n v_2$

G' 中每个结点都是偶结点 iff

G 中恰有两个奇结点 v_1 和 v_2



证明2



- **定理** 设 $G = \langle V, E \rangle$ 为连通无向图, $v_1, v_2 \in V$ 且 $v_1 \neq v_2$ 。则 G 有一条从 v_1 至 v_2 的欧拉链当且仅当 G 恰有两个奇结点 v_1 和 v_2 。

证明: 若 $e = (v_1, v_2) \in E$, 则

G 有一条从 v_1 至 v_2 的欧拉链 $v_1, v'_1, \dots, v'_n v_2$

考虑图 $G' = G - \{e\}$

则可证 $G - \{e\}$ 有两个圈 $v_1 v'_1 \dots v'_i v_1$ 和 $v_2 v'_{i+2} \dots v'_n v_2$

G' 中每个结点都是偶结点 iff

G 中恰有两个奇结点 v_1 和 v_2

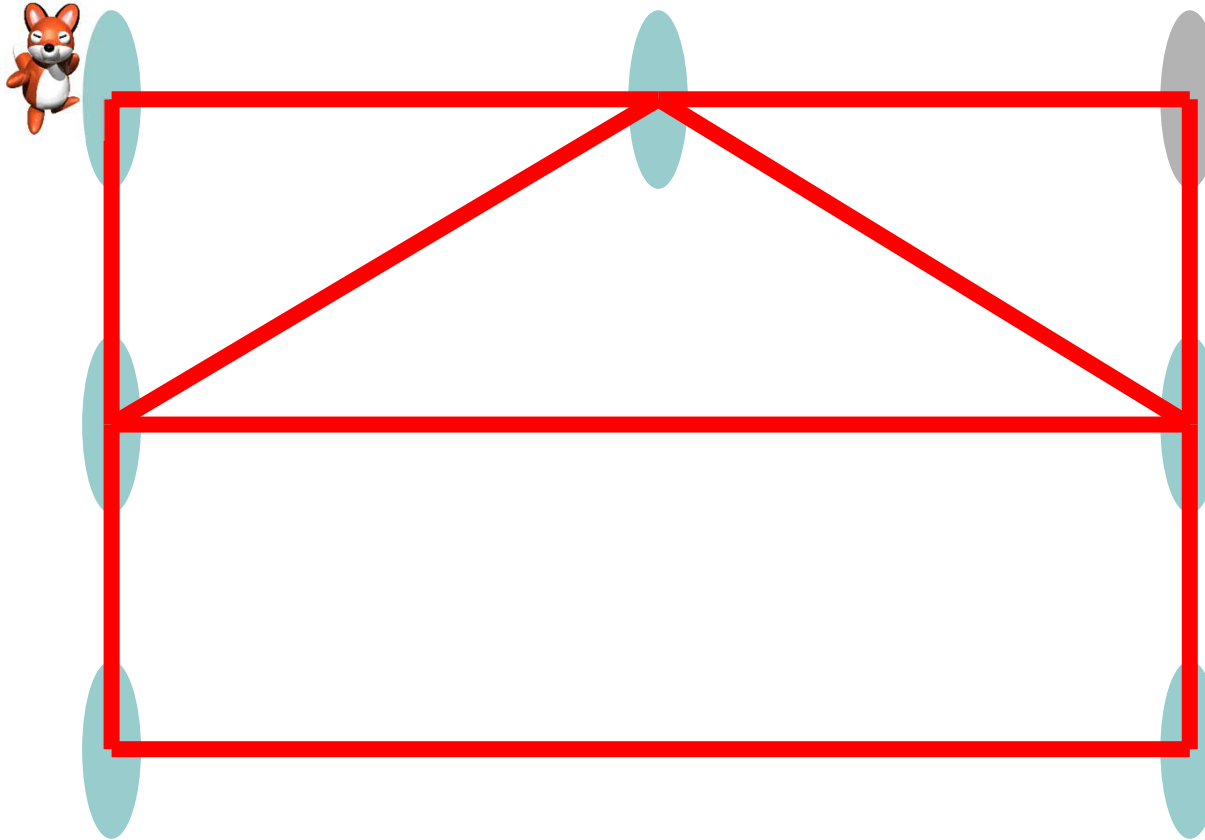


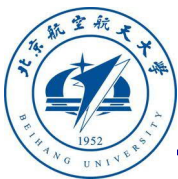
Fleury算法

- ◆ 输入: 无向图 G .
- ◆ 输出: 从 v 到 w 的欧拉链/欧拉圈.
- ◆ 算法:
 1. 从任意一点开始, 沿着没有走过的边向前走
 2. 在每个顶点, 优先选择剩下的非桥边, 除非只有唯一一条边
 3. 直到得到欧拉回路或宣布失败

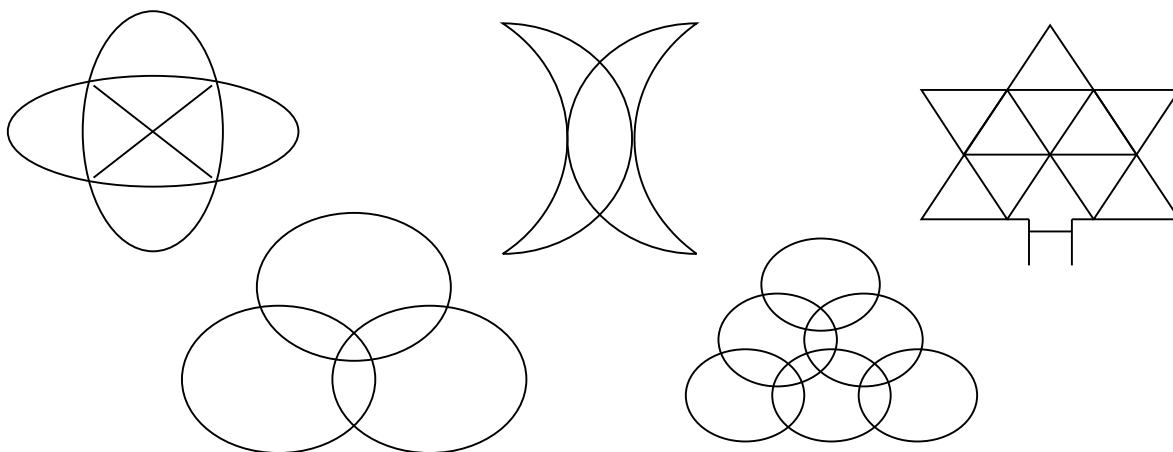


Fleury算法(举例)





一笔画

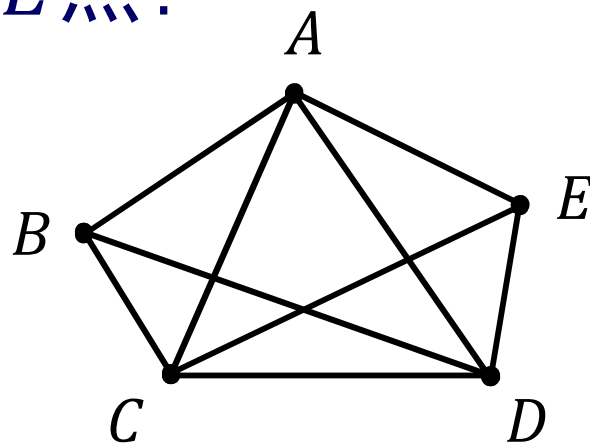


一张连通图能由一笔画出来的充要条件是：

- ✓ 每个交点处的线条数都是偶数；或 （欧拉回路）
- ✓ 恰有两个交点处的线条数是奇数。 （欧拉链）

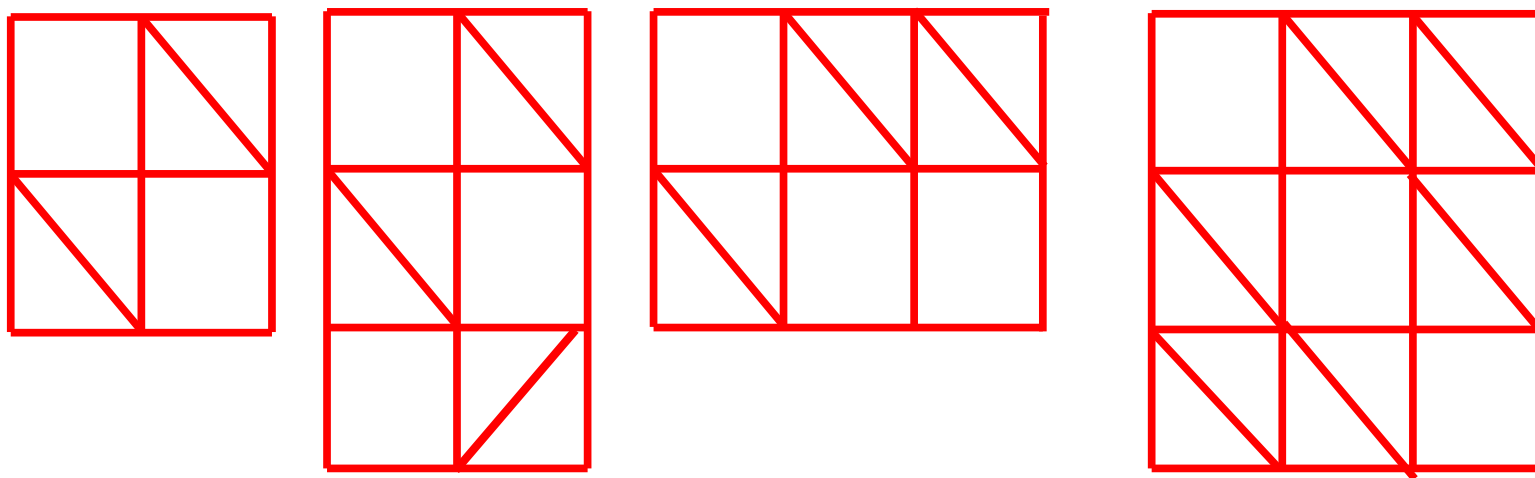


例题：在下图中，甲、乙两只蚂蚁分别处在
 A 、 B 两点.甲蚂蚁向乙蚂蚁提出：“咱俩
比赛，看谁先把这个图中的九条边都爬遍
后到达 E 点.”乙蚂蚁同意.假定两只蚂蚁爬
的速度相同且同时开始.问：究竟哪只蚂蚁
最先到达 E 点？





求所有正整数 (m, n) ，使得在 $m \times n$ 的方格表(有 $m + 1$ 条竖线和 $n + 1$ 条横线)中，可以将某些小格添加对角线(每格至多添加一条，可以不添)，使得整个图(包括所有对角线及表格自身的边)可以一笔画走遍所有边并回到起点。





中国邮递员问题(Chinese Postman Problem)

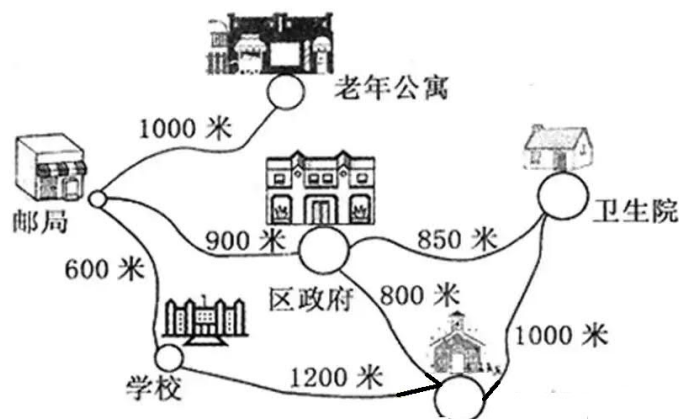
1962年，中国数学家管梅谷提出并解决了“中国邮递员问题”

1、问题

邮递员派信的街道是边赋权连通图。从邮局出发，每条街道至少行走一次，再回邮局。如何行走，使其行走的环游路程最短？

如果邮路图本身是欧拉图，那么由Fleury算法，可得到他的行走路线。

如果邮路图本身是非欧拉图，如何重复行走街道才能使行走总路程最短？





管梅谷的结论

- **定理** 若 W 是包含图 G 的每条边至少一次的回路，则 W 具有最小权值当且仅当下列两个条件被满足：
 - (1) G 的每条边在 W 中最多重复一次；
 - (2) 对于 G 的每个圈上的边来说，在 W 中重复的边的总权值不超过该圈非重复边总权值。

- **Edmonds-Johnson算法—— $O(n^3)$**



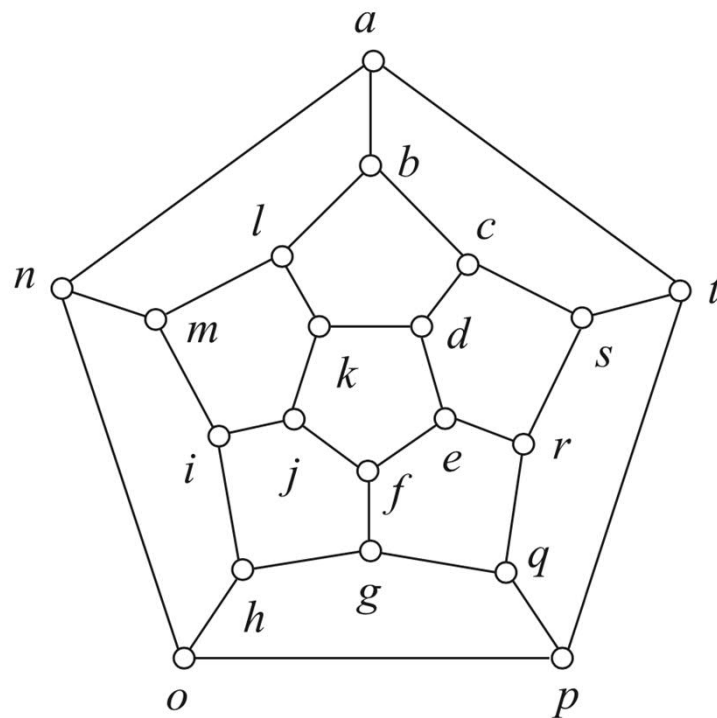
本章内容

- 穿程问题
 - 欧拉图
 - 哈密顿图
 - 骑士周游
- 通路问题
 - 最短通路
 - 关键通路
- 匹配问题
 - 二分图
 - 二分图匹配



哈密顿图

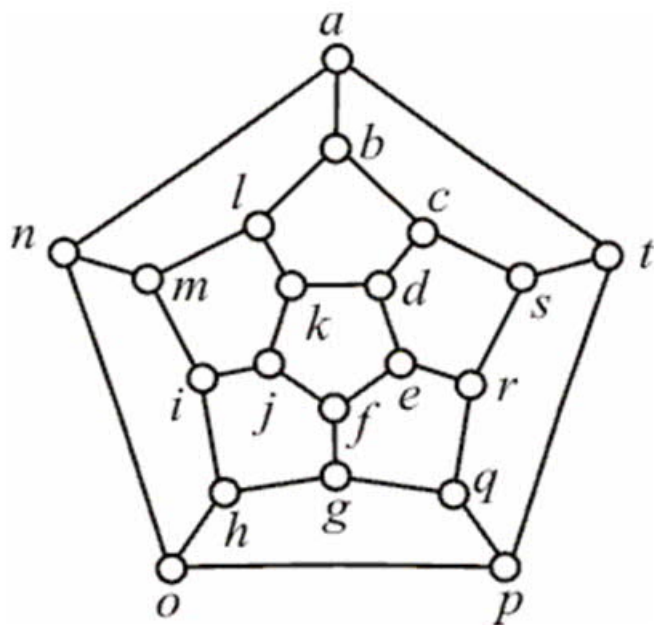
- 哈密顿问题是爱尔兰数学家哈密顿在1857年发明的智力题。从一个城市出发，遍历每个城市一次且仅一次，而后返回出发城市。这就是周游世界问题，也就是判断图中是否存在遍历所有节点的简单回路。
- 通常没有简单的判断方法
- 以右图为例，20个城市
- 节点的组合有如下多种
$$20! = 2432902008176640000$$
- 若每秒验证10000种组合，
- 全部验证完，仍需7714681年





哈密顿图

- $G=\langle V,E\rangle$
- $V=\{'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t'\}$
- $E=\{('a','b'),('b','c'),('c','d'),('d','e'),('e','f'),('f','g'),('g','h'),('h','i'),('i','j'),('j','k'),('k','l'),('l','m'),('m','n'),('n','o'),('o','p'),('p','q'),('q','r'),('r','s'),('s','t'),('a','n'),('b','l'),('c','s'),('d','k'),('e','r'),('f','j'),('g','q'),('h','o'),('i','m'),('p','t'),('t','a')\}$



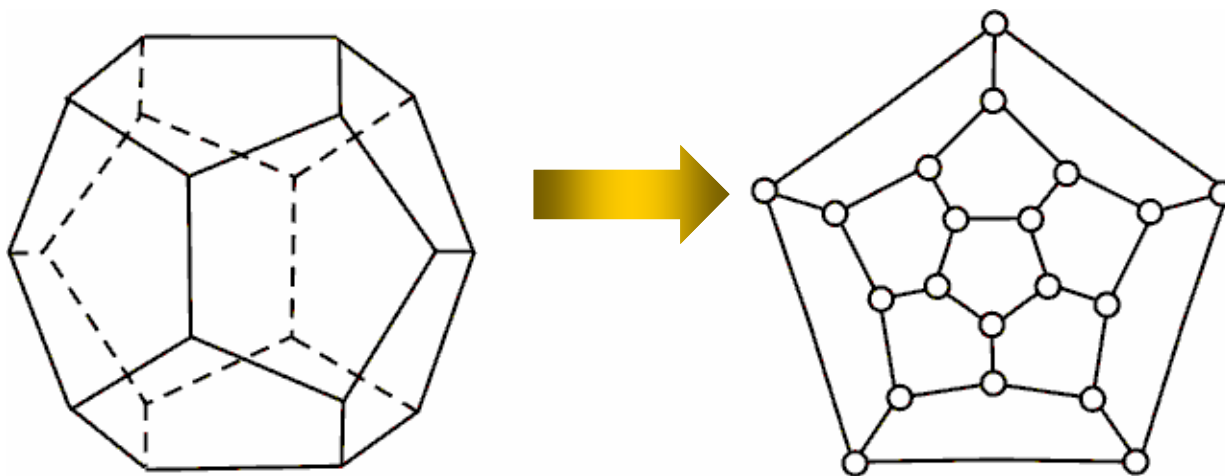
```
import math
m=math.factorial(20)
print(m)
y=math.factorial(20)/(10000*3600*2
4*365)
print(y)
2432902008176640000
7714681.659616439
```

- 有多少通路？有多少回路？
- 人工能求出多少？
- 复杂度： $O(n!)$?
- ✓ 构造排列 $1,2,3,4,5,\dots,n$
- ✓ 判断是否是通路



哈密顿图

- **定义 11.2.4**：无向图 G 中穿过每个节点一次且仅一次的非闭合通路，称为哈密顿通路。
- **定义 11.2.5（哈密顿图）** 无向图 G 中穿过每个节点一次且仅一次的圈，称为哈密顿圈，具有哈密顿圈的图称为哈密顿图。



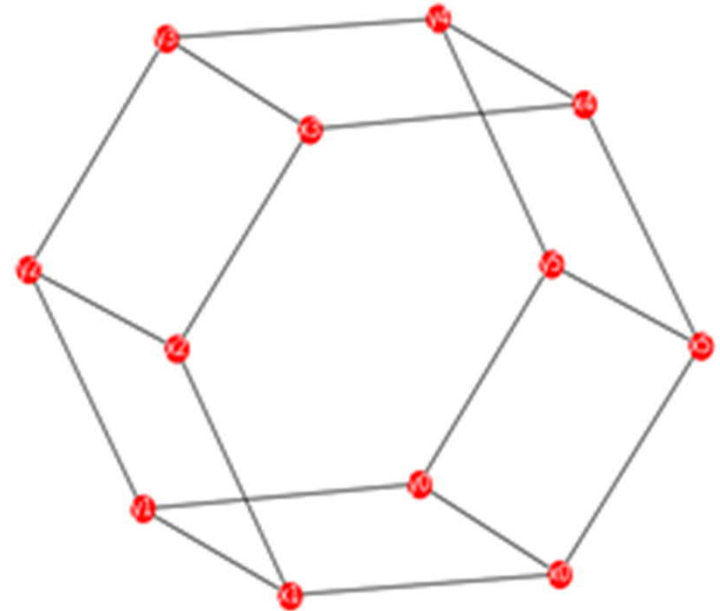
勒代定理（Redei theorem）：每一个竞赛图都有一个有向的哈密顿通路。

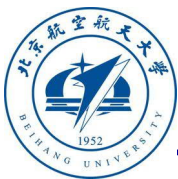


节点周游问题

- V 中每个节点出现一次且仅一次，并形成圈
- 一般图如何解决周游问题

```
import graph.graph as gt
V1={'x1','x2','x3','x4','x5'}
V2={'y1','y2','y3','y4','y5'}
V=V1|V2
E={('x0','x1'), ('x1','x2'), ('x2','x3'), ('x3','x4'),
  ('x4','x5'),
  ('x5','x0'),('y0','y1'), ('y1','y2'), ('y2','y3'),
  ('y3','y4'),
  ('y4','y5'), ('y5','y0'), ('x0','y0'), ('x1','y1'),
  ('x2','y2'),
  ('x3','y3'), ('x4','y4'), ('x5','y5')}\n
gt.drawgraph(E)
```





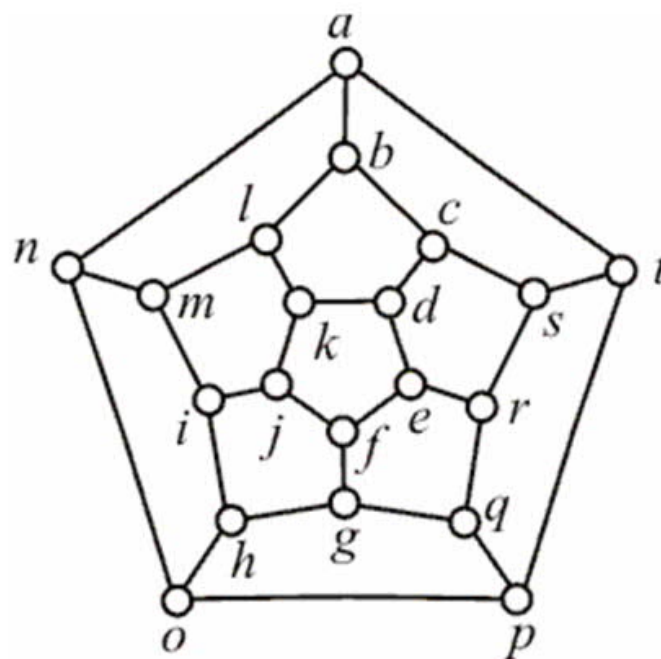
哈密顿图问题

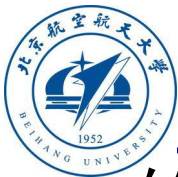
■ 构建邻接表

$V = \{ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't' \}$

$E = \{ ('a', 'b'), ('b', 'c'), ('c', 'd'), ('d', 'e'), ('e', 'f'), ('f', 'g'), ('g', 'h'), ('h', 'i'), ('i', 'j'), ('j', 'k'), ('k', 'l'), ('l', 'm'), ('m', 'n'), ('n', 'o'), ('o', 'p'), ('p', 'q'), ('q', 'r'), ('r', 's'), ('s', 't'), ('a', 'n'), ('b', 'l'), ('c', 's'), ('d', 'k'), ('e', 'r'), ('f', 'j'), ('g', 'q'), ('h', 'o'), ('i', 'm'), ('p', 't'), ('t', 'a') \}$

$Ea = [['a', ['b', 'n', 't']], ['b', ['a', 'c', 'l']], ['c', ['b', 'd', 's']], ['d', ['c', 'e', 'k']], ['e', ['d', 'f', 'r']], ['f', ['e', 'g', 'j']], ['g', ['f', 'h', 'q']], ['h', ['g', 'i', 'o']], ['i', ['h', 'j', 'm']], ['j', ['f', 'i', 'k']], ['k', ['d', 'j', 'l']], ['l', ['b', 'k', 'm']], ['m', ['i', 'l', 'n']], ['n', ['a', 'm', 'o']], ['o', ['h', 'n', 'p']], ['p', ['o', 'q', 't']], ['q', ['g', 'p', 'r']], ['r', ['e', 'q', 's']], ['s', ['c', 'r', 't']], ['t', ['a', 'p', 's']]]$





邻接表

- 给定图 $G = \langle V, E \rangle$, G 是连通图, 对于节点 u 的邻接表为 $[u, v | (u, v) \in E]$

```
def adjacentlist(V,E):
```

```
    Ea=[]
```

```
    for w in V:
```

```
        e0=[w]
```

```
        e1=[]
```

```
        for (u,v) in E:
```

```
            if u == w and (v not in e1):
```

```
                e1=e1+[v]
```

```
            if v==w and (u not in e1):
```

```
                e1=e1+[u]
```

```
        e0=e0+[sorted(e1)]
```

```
        Ea=Ea+[e0]
```

```
    return sorted(Ea)
```

```
import graph.graph as gt
```

```
import graph.basicproblem as gb
```

```
V={'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t'}
```

```
E={('a','b'),('b','c'),('c','d'),('d','e'),('e','f'),('f','g'),('g','h'),('h','i'),('i','j'),('j','k'),('k','l'),('l','m'),('m','n'),('n','o'),('o','p'),('p','q'),('q','r'),('r','s'),('s','t'),('a','n'),('b','l'),('c','s'),('d','k'),('e','r'),('f','j'),('g','q'),('h','o'),('i','m'),('p','t'),('t','a')}
```

```
Ea=gb.adjacentlist(V,E)
```

```
gt.drawgraph(E)
```

```
print("Ea",Ea)
```

```
Ea [['a', ['b', 'n', 't']], ['b', ['a', 'c', 'l']], ['c', ['b', 'd', 's']], ['d', ['c', 'e', 'k']], ['e', ['d', 'f', 'r']], ['f', ['e', 'g', 'j']], ['g', ['f', 'h', 'q']], ['h', ['g', 'i', 'o']], ['i', ['h', 'j', 'm']], ['j', ['f', 'i', 'k']], ['k', ['d', 'j', 'l']], ['l', ['b', 'k', 'm']], ['m', ['i', 'l', 'n']], ['n', ['a', 'm', 'o']], ['o', ['h', 'n', 'p']], ['p', ['o', 'q', 't']], ['q', ['g', 'p', 'r']], ['r', ['e', 'q', 's']], ['s', ['c', 'r', 't']], ['t', ['a', 'p', 's']]]
```



哈密顿图周游算法

- 在构建连通图的邻接表后，哈密顿通路问题就是基于邻接表的、无重复节点的遍历问题。
- 通路path构建的直观方法就是沿着邻接表依次增加通路。若path无法增加新节点，则回溯，沿着邻接表下一个新节点增加path通路节点。



哈密顿图周游算法

程序tourpath0算法：

- (1) 取路径path的末尾节点path[-1]为w。
- (2) 取节点w的邻接表E2。
- (3) 若邻接表E2中节点u不在path上，则u加入path，否则，返回。

程序tourpath1算法：

- (1) 取路径path的末尾节点 $v = \text{path.pop}()$ 为v。
- (2) 若 path非空，取路径path的末尾节点path[-1]为u。
- (3) 从节点u的邻接表从节点v以后依次检查 $k = E2.\text{index}(v)$ 。
- (4) 若 $v = E2[k]$ 不在path，则v为path的新末尾节点，直至邻接表结束。
- (5) 若path有新节点，则结束，否则，path弹出末尾节点。

程序tourpath算法：

- (1) 若 $\text{len}(\text{path}) == m$ ，则求新的周游通路，即 $\text{tourpath1}(V, E, \text{path}, m)$ 。
- (2) 若 $\text{len}(\text{path}) \neq 0$ ，依次迭代执行tourpath0与tourpath1。



哈密顿图周游算法

- (1) 从指定点开始出发。
- (2) 依据邻接关系，依次选择可行的下一步。

```
def tourpath0(V,E,path,m):  
    while(len(path) < m):  
        w=path[-1]  
        i=V.index(w)  
        E1=E[i]  
        E2=E1[1]  
        for u in E2:  
            if(u not in path):  
                path.append(u)  
                break  
        if(path[-1] == w):  
            break  
    return path
```



哈密顿图周游算法

- (3) 退一步，依次选择可行的下一步。

```
def tourpath1(V,E,path,m):  
    v=path.pop( )  
    while(len(path) != 0):  
        u=path[-1]  
        i=V.index(u)  
        E1=E[i]  
        E2=E1[1]  
        k=E2.index(v)  
        while(k < (len(E2)-1)):  
            k=k+1  
            v=E2[k]  
            if(v not in path):  
                path.append(v)  
                break  
            if(u != path[-1]):  
                break  
        v=path.pop( )  
    return path
```




哈密顿图周游算法

- (4) 当前是一条哈密尔顿通路,
 - 则tourpath1
- 否则扩展通路
 - 当前一条哈密尔顿通路, 结束
 - 通路回溯

```
def tourpath(V,E,path,m):  
    if(len(path) == m):  
        path=tourpath1(V,E,path,m)  
    while(len(path) != 0):  
        path=tourpath0(V,E,path,m)  
        if(len(path) == m):  
            break  
        path=tourpath1(V,E,path,m)  
    return path
```



哈密顿图周游算法

- 构建邻接表
- 设置初始点 v_0
- 搜寻一条路径
- 若 $\text{len}(\text{path}) == \text{len}(V)$,
则搜寻下一条路径

```
def Hamiltonpath(V,E,v0):  
    global Ea,paths  
  
    V=sorted(V)  
    Ea=adjacentlist(V,E)  
    paths=set({})  
    path=[v0]  
    m=len(V)  
    path=tourpath(V,Ea,path,m)  
    paths=paths|{tuple(path)}  
    while(len(path) == m):  
        path=tourpath(V,Ea,path,m)  
        if len(path)==m:  
            paths=paths|{tuple(path)}  
            print(path)  
    return paths
```



哈密顿图周游算法

```
import graph.graph as gg
V=['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t']
E={( 'a','b'),( 'b','c'),( 'c','d'),( 'd','e'),( 'e','f'),( 'f','g'),( 'g','h'),( 'h','i'),( 'i','j'),( 'j','k'),( 'k','l'),( 'l','m'),( 'm','n'),( 'n','o'),( 'o','p'),( 'p','q'),( 'q','r'),( 'r','s'),( 's','t'),( 'a','n'),( 'b','l'),( 'c','s'),( 'd','k'),( 'e','r'),( 'f','j'),( 'g','q'),( 'h','o'),( 'i','m'),( 'p','t'),( 't','a')}
gt.drawgraph(E)
v0='a'
paths=gb.Hamiltonpath(V,E,v0)
print(len(paths))

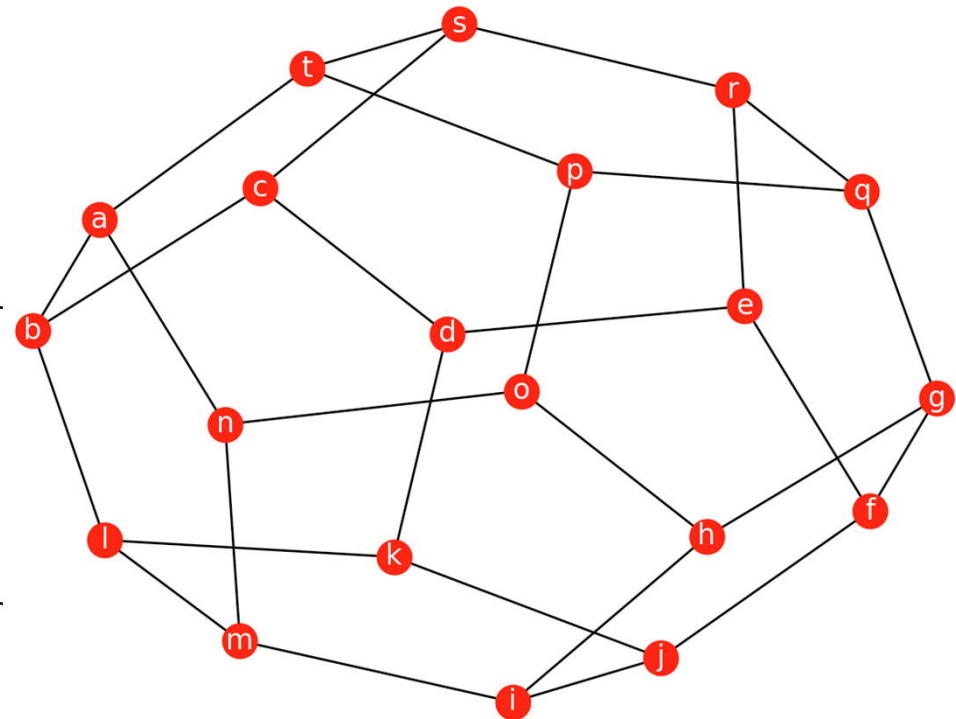
k=0
for path in paths:
    path=list(path)
    if path[-1] in ['b', 'n', 't']:
        k=k+1
        print(k,path)
```

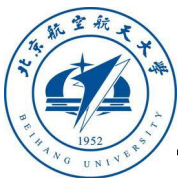


哈密顿图周游算法

```
import graph.graph as gg
V=['a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t']
E={( 'a','b'),( 'b','c'),( 'c','d'),( 'd','e'),( 'e','f'),( 'f','g'),( 'g','h'),( 'h','i'),( 'i','j'),( 'j','k'),( 'k','l'),( 'l','m'),( 'm','n'),( 'n','o'),( 'o','p'),( 'p','q'),( 'q','r'),( 'r','s'),( 's','t'),( 'a','n'),( 'b','l'),( 'c','s'),( 'd','k'),( 'e','r'),( 'f','j'),( 'g','q'),( 'h','o'),( 'i','m'),( 'p','t'),( 't','a')}
gg.drawgraph(E)
v0='a'
paths=gg.Hamiltonpath(V,E,v0)
print(len(paths))
```

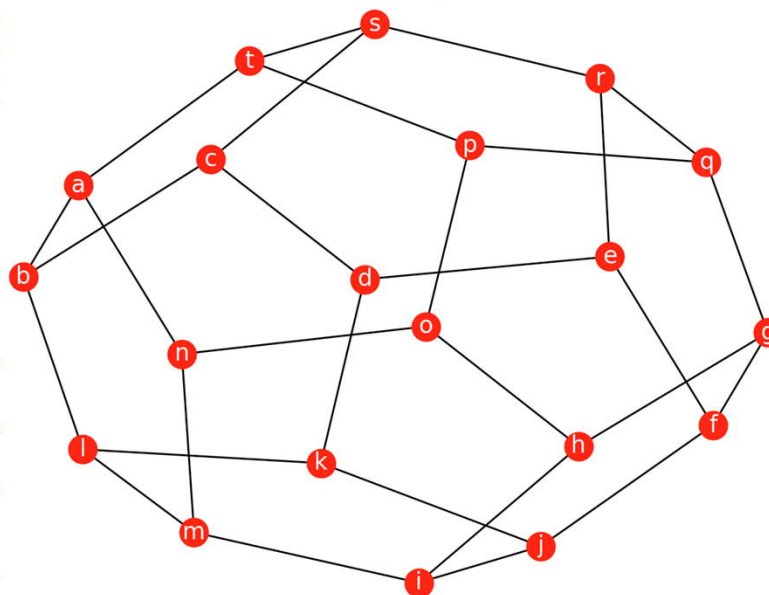
162





哈密顿图周游算法

[a, 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 't', 's', 'r', 'q']
[a, 'b', 'c', 'd', 'e', 'f', 'g', 'q', 'r', 's', 't', 'p', 'o', 'h', 'i', 'j', 'k', 'l', 'm', 'n']
[a, 'b', 'c', 'd', 'e', 'f', 'g', 'q', 'r', 's', 't', 'p', 'o', 'n', 'm', 'l', 'k', 'j', 'i', 'h']
[a, 'b', 'c', 'd', 'e', 'f', 'j', 'i', 'h', 'g', 'q', 'r', 's', 't', 'p', 'o', 'n', 'm', 'l', 'k']
[a, 'b', 'c', 'd', 'e', 'f', 'j', 'k', 'l', 'm', 'i', 'h', 'g', 'q', 'r', 's', 't', 'p', 'o', 'n']
[a, 'b', 'c', 'd', 'e', 'f', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 't', 's', 'r', 'q', 'g', 'h', 'i']
[a, 'b', 'c', 'd', 'e', 'r', 's', 't', 'p', 'q', 'g', 'f', 'j', 'i', 'h', 'o', 'n', 'm', 'l', 'k']
[a, 'b', 'c', 'd', 'e', 'r', 's', 't', 'p', 'q', 'g', 'f', 'j', 'k', 'l', 'm', 'i', 'h', 'o', 'n']
[a, 'b', 'c', 'd', 'e', 'r', 's', 't', 'p', 'q', 'g', 'f', 'j', 'k', 'l', 'm', 'n', 'o', 'h', 'i']
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'i', 'j', 'f', 'e', 'r', 's', 't', '']
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'h', 'g', 'q', 'p', 't']
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'h', 'i', 'j', 'f', 'e',
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'h', 'i', 'j', 'f', 'g',
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'g', 'h', 'i']
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'p', 't', 's', 'r', 'e',
[a, 'b', 'c', 'd', 'k', 'l', 'm', 'n', 'o', 'p', 't', 's', 'r', 'q',
[a, 'b', 'c', 's', 'r', 'e', 'd', 'k', 'l', 'm', 'n', 'o', 'h', 'i',
... ..





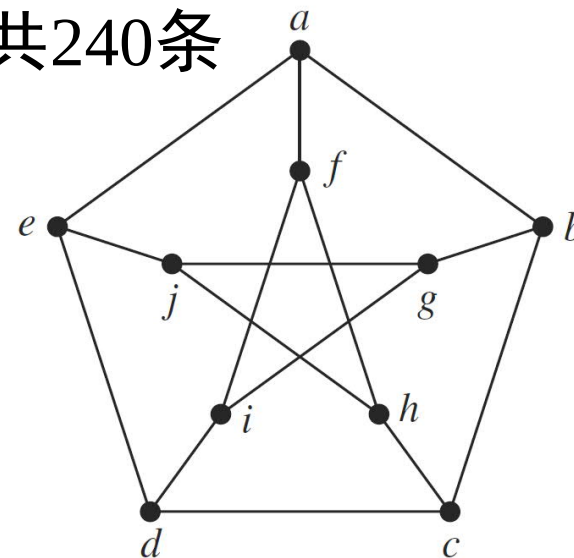
彼得森图 (Petersen graph)

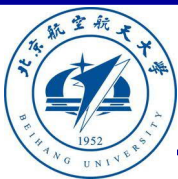
■ 彼得森图

$V = \{ 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j' \}$

$E = \{ ('a', 'b'), ('b', 'c'), ('c', 'd'), ('d', 'e'), ('e', 'a'), ('a', 'f'), ('b', 'g'), ('c', 'h'), ('d', 'i'), ('e', 'j'), ('f', 'h'), ('g', 'i'), ('h', 'j'), ('i', 'f'), ('j', 'g') \}$

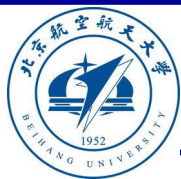
- 每个节点出发有24条哈密尔顿通路，共240条
- 无哈密尔顿回路
- 非欧拉图





彼德森图—哈密尔顿通路

- {'a', 'b', 'g', 'i', 'f', 'h', 'c', 'd', 'e', 'j'}, ■ ('a', 'e', 'd', 'i', 'f', 'h', 'j', 'g', 'b', 'c'),
- ('a', 'e', 'd', 'c', 'b', 'g', 'j', 'h', 'f', 'i'), ■ ('a', 'f', 'h', 'j', 'e', 'd', 'i', 'g', 'b', 'c'),
- ('a', 'b', 'c', 'd', 'e', 'j', 'h', 'f', 'i', 'g'), ■ ('a', 'f', 'i', 'd', 'e', 'j', 'g', 'b', 'c', 'h'),
- ('a', 'e', 'd', 'i', 'f', 'h', 'c', 'b', 'g', 'j'), ■ ('a', 'e', 'd', 'c', 'b', 'g', 'i', 'f', 'h', 'j'),
- ('a', 'e', 'j', 'g', 'b', 'c', 'd', 'i', 'f', 'h'), ■ ('a', 'f', 'h', 'j', 'e', 'd', 'c', 'b', 'g', 'i'),
- ('a', 'e', 'j', 'h', 'f', 'i', 'd', 'c', 'b', 'g'), ■ ('a', 'f', 'h', 'c', 'b', 'g', 'j', 'e', 'd', 'i'),
- ('a', 'b', 'c', 'h', 'f', 'i', 'd', 'e', 'j', 'g'), ■ ('a', 'b', 'g', 'j', 'e', 'd', 'i', 'f', 'h', 'c'),
- ('a', 'b', 'g', 'j', 'e', 'd', 'c', 'h', 'f', 'i'), ■ ('a', 'b', 'g', 'i', 'f', 'h', 'j', 'e', 'd', 'c'),
- ('a', 'b', 'c', 'h', 'f', 'i', 'g', 'j', 'e', 'd'), ■ ('a', 'f', 'i', 'd', 'e', 'j', 'h', 'c', 'b', 'g'),
- ('a', 'e', 'j', 'h', 'f', 'i', 'g', 'b', 'c', 'd'), ■ ('a', 'f', 'i', 'g', 'b', 'c', 'd', 'e', 'j', 'h'),
- ('a', 'b', 'c', 'd', 'e', 'j', 'g', 'i', 'f', 'h'), ■ ('a', 'e', 'j', 'g', 'b', 'c', 'h', 'f', 'i', 'd'),
- ('a', 'f', 'h', 'c', 'b', 'g', 'i', 'd', 'e', 'j'), ■ ('a', 'f', 'i', 'g', 'b', 'c', 'h', 'j', 'e', 'd')}



充分条件

例：设 G 为 n 阶简单无向图，

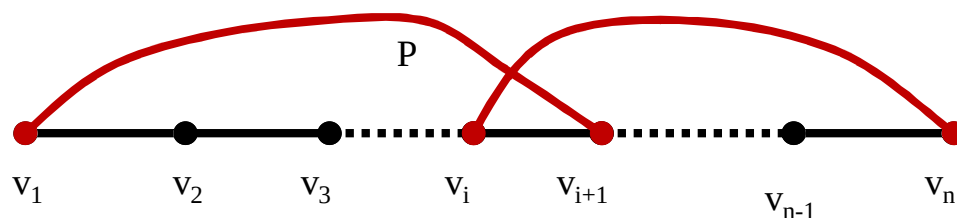
- 1) 若 G 的任意两结点的度数之和大于等于 $n-1$ ，则 G 是连通的
- 2) 若对 G 的任意结点 v ，皆有 $d(v) \geq (n-1)/2$ ，则 G 是连通的

■ **定理** (Dirac, 充分条件) 对于 $n \geq 3$ 的简单图 G ，如果 G 中有：

$$\delta(G) \geq \frac{n}{2}$$

那么 G 是哈密顿图。

G 的顶点度数的最小值





充分条件

- **定理** (Dirac, 充分条件) 对于 $n \geq 3$ 的简单图 G , 如果 G 中有:

$$\delta(G) \geq \frac{n}{2}$$

那么 G 是哈密顿图。

证明: 若不然, 设 G 是一个满足定理条件的**极大**非哈密顿简单图。显然 G 不能是完全图, 否则, G 是哈密顿图。

于是, 可以在 G 中任意取两个不相邻顶点 u 与 v 。

考虑图 $G + uv$

由 G 的极大性, $G + uv$ 是哈密顿图。且 $G + uv$ 的每一个哈密顿圈必然包含边 uv 。

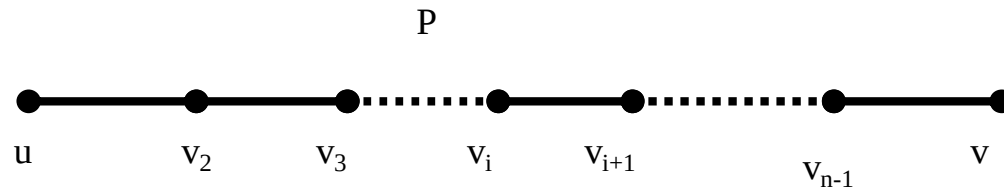


充分条件

所以，在 G 中存在起点为 u 而终点为 v 的哈密顿路 P 。

不失一般性，设起点为 u 而终点为 v 的哈密顿路 P 为：

$$P = v_1 v_2 \dots v_n, v_1 = u, v_n = v$$



$$\text{令: } S = \{v_i | uv_{i+1} \in E(G)\}$$

$$T = \{v_i | v_i v \in E(G)\}$$



充分条件

v 是 v_n , 而 S 中的编号小于 n

对于 S 与 T , 显然, $v_n \notin S \cup T$

所以 $|S \cup T| < n$:

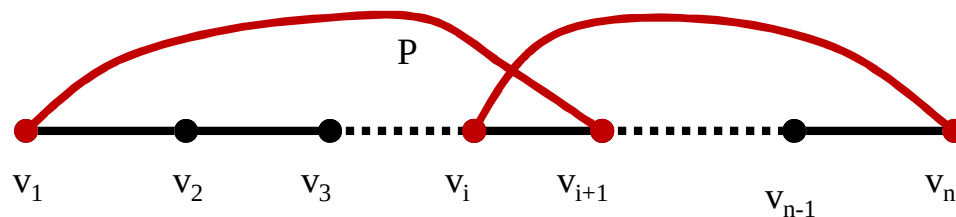
u 与 v 不相邻

另一方面: 可以证明 $|S \cap T| = \emptyset$

否则, 设 $v_i \in S \cap T$

那么, 由 $v_i \in S$ 有 $uv_{i+1} \in E(G), v_iv_{i+1} \in E(G)$

由 $v_i \in T$ 有 $v_iv_n \in E(G)$



这样在 G 中有哈密顿圈, 与假设矛盾!



充分条件

于是：

$$d(u) + d(v) = |S| + |T| = |S \cup T| + |S \cap T| < n$$

这与已知 $\delta(G) \geq \frac{n}{2}$ 矛盾！



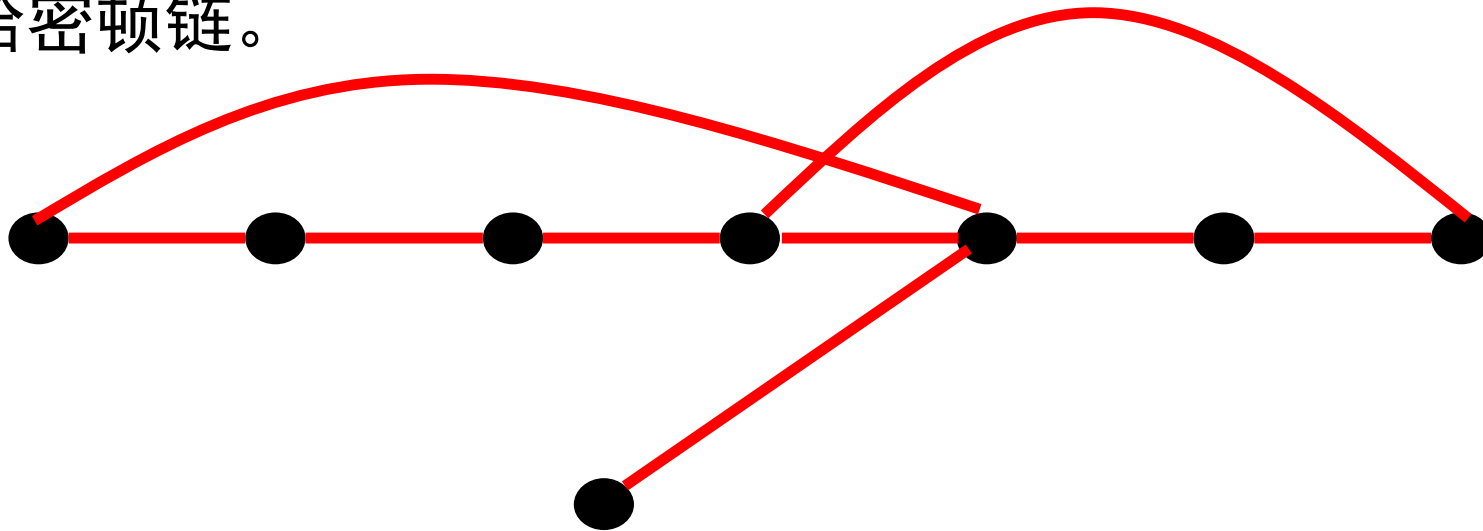
哈密顿圈

- **定理 11.2.5:** 设 G 是具有 n ($n \geq 3$) 个节点的无向简单图。若 G 中每一对节点的度数之和大于或等于 $n - 1$ ，则 G 中存在一条哈密顿链。
- 若 G 中每一对节点的次数之和大于或等于 n ，则 G 中存在一条哈密顿圈。
- 定理在一个有向完全图中必存在一条哈密顿通路
- 定理是充分条件



哈密顿圈

- **定理 11.2.5:** 设 G 是具有 n ($n \geq 3$) 个节点的无向简单图。若 G 中每一对节点的度数之和大于或等于 $n - 1$ ，则 G 中存在一条哈密顿链。



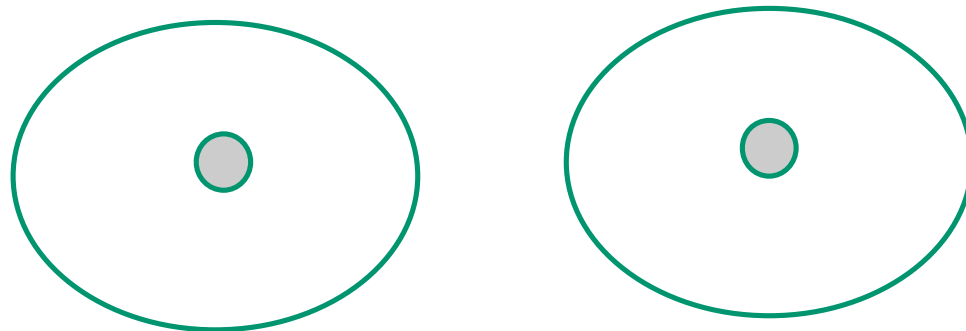


充分条件

■ **定理 11.2.5:** 设 G 是具有 n ($n \geq 3$) 个节点的无向简单图。若 G 中每一对节点的度数之和大于或等于 $n - 1$, 则 G 中存在一条哈密顿链。

■ 证明是连通性（反证法）

- 假设不连通, 至少两个分支 G_1 和 G_2 , 各取 G_1 和 G_2 上两点 u, v
- 则 $|G_1| + |G_2| \leq n, d(u) \leq |G_1| - 1$ 和 $d(v) \leq |G_2| - 1$
- 则 $d(u) + d(v) \leq |G_1| + |G_2| - 2 \leq n - 2$
- 与题设矛盾





充分条件

定理 11.2.5: 设 G 是具有 n ($n \geq 3$) 个节点的无向简单图。若 G 中每一对节点的度数之和大于或等于 $n-1$, 则 G 中存在一条哈密顿链。

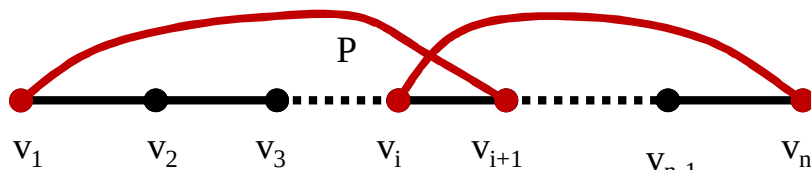
2. 证明存在哈密顿通路

- 假设存在**最长**的基本路径 $\Gamma = v_1 v_2 \dots v_l$
- 如果 $l < n$, **证明存在经过 $v_1 v_2 \dots v_l$ 的回路**
 - i) 如果 v_1, v_l 相邻, 则存在 Γ 与 (v_1, v_l) 构成回路。
 - ii) 如果 v_1, v_l 不相邻, **如何证明仍有回路?**
 - » 基本路径条件 $\rightarrow$ 说明 v_1, v_l 只能与 Γ 内的结点相邻

□ 设 v_1 与 $v_{i1}, v_{i2}, \dots, v_{ik}$ 相邻

假设有 k 个结点

□ 则 v_l 相邻结点数最多为: $l - k - 1$; 不能与 $v_{i1-1}, v_{i2-1}, \dots, v_{ik-1}$ 相邻 (**否则有回路**), 并且 v_l 不与自己相连



否则存在回路

□ 则 $d(v_1) + d(v_l) \leq k + l - k - 1 = l - 1 < n - 1$, 矛盾



- iii) 扩展路径法
 - 已证明当 $l < n$, 内部必存在回路, G 是连通的
 - 存在不在 Γ 的结点 v 与回路结点相邻 (一个圈+一个点)
 - 可以扩展 Γ , 说明 Γ 不是最长的基本路径 (可以从 v 出发)
 - 因此 $l = n$
- iv) $\Gamma = v_1 v_2 \dots v_n$ 是哈密顿通路



哈密顿圈

- **定理** 若 G 中每一对节点的次数之和大于或等于 n ，则 G 中存在一条哈密顿圈。
- 定理在一个有向完全图中必存在一条哈密顿通路
- 定理是充分条件



充分条件

▪ **定理** 若 G 中每一对节点的次数之和大于或等于 n , 则 G 中存在一条哈密顿圈。 $d(v_1) + d(v_2) \geq n$

• 由以上定理可知, 存在经过 $v_1 v_2 \dots v_n$ 的汉密尔顿通路

– i) 如果 v_1, v_n 相邻, 则存在 Γ 与 (v_1, v_n) 构成回路。

– ii) 如果 v_1, v_n 不相邻

□ 设 v_1 与 $v_{i1}, v_{i2}, \dots, v_{ik}$ 相邻

假设有 k 个结点

□ 则 v_n 相邻结点数最多为: $n - k - 1$; 不能与 $v_{i1-1}, v_{i2-1}, \dots, v_{ik-1}$ 相邻 (否则有回路), 并且 v_n 不与自己相连

□ 则 $d(v_1) + d(v_l) = k + n - k - 1 = n - 1 < n$, 矛盾



哈密顿圈

- **定理 11.2.6** 在有向完全图中必存在一条哈密顿通路
- 对于完全图 $[V, E] = \text{completeset}(n)$, $\text{Hamiltonpath}(V, E, v_0)$ 可以求哈密顿通路, 进而可以求哈密顿回路。

```
import graph.graph as gg
n = 5
[V,E] = gg.completegraphset(n)
v0 = 0
paths = gg.Hamiltonpath(V,E,v0)
gg.drawgraph(E)
print(paths)
print(len(paths))
```

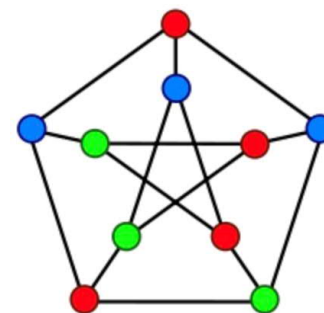
```
{(0, 1, 3, 4, 2), (0, 2, 1, 3, 4), (0, 3, 2, 4, 1),
(0, 4, 2, 3, 1), (0, 2, 4, 1, 3), (0, 3, 1, 4, 2),
(0, 3, 4, 2, 1), (0, 4, 3, 2, 1), (0, 3, 4, 1, 2),
(0, 2, 3, 1, 4), (0, 2, 3, 4, 1), (0, 1, 4, 3, 2),
(0, 2, 4, 3, 1), (0, 3, 1, 2, 4), (0, 1, 2, 4, 3),
(0, 4, 1, 3, 2), (0, 1, 3, 2, 4), (0, 4, 1, 2, 3),
(0, 1, 4, 2, 3), (0, 3, 2, 1, 4), (0, 2, 1, 4, 3),
(0, 4, 2, 1, 3), (0, 1, 2, 3, 4), (0, 4, 3, 1, 2)}
24
```



必要条件：哈密顿图子集

- **定理：** 设 $G = \langle V, E \rangle$ 是哈密顿图, 则对 V 的任意非空真子集 V_1 有

$$W(G - V_1) \leq |V_1|$$



- 本定理是哈密顿图的必要条件，不是充分条件
- 可以验证彼得松图满足定理中的条件，但不是哈密顿图(不是充分)。
- **逆否定理：** 若一个图不满足定理中的条件，它一定不是哈密顿图。

如果存在 V 的一个非空真子集 V_1 有

$$W(G - V_1) > |V_1|$$

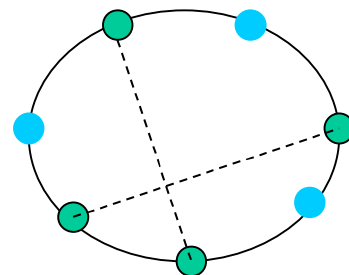
则 G 不是哈密顿图



必要条件：哈密顿图子集

- **定理：** 设 $G = \langle V, E \rangle$ 是哈密顿图, 则对 V 的任意非空真子集 V_1 有

$$W(G - V_1) \leq |V_1|$$



证明： 若连通图 $G = \langle V, E \rangle$ 是哈密顿图

设 C 是 G 的哈密顿回路, 则 $C - V_1$ 是 $G - V_1$ 的生成子图,

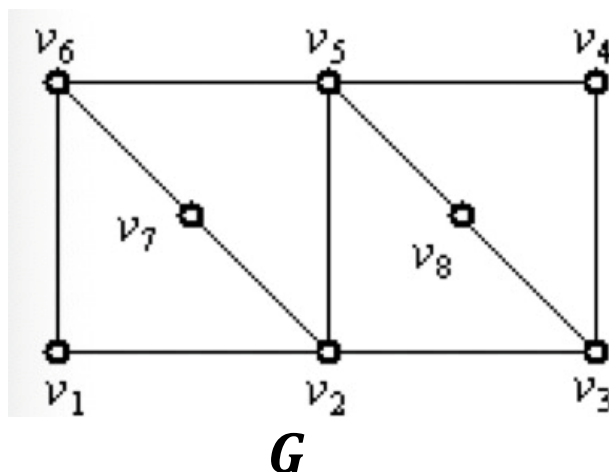
所以 $W(G - V_1) \leq W(C - V_1)$ 。

易知当 V_1 中的结点互不邻接时, $C - V_1$ 的连通分支数达最大值 $|V_1|$

所以有 $W(G - V_1) \leq |V_1|$ 。



例：说明图 G 不是哈密顿图。



解：取 $V_1 = \{v_2, v_6\}$

则 $G - V_1$ 有 3 个连通分图，

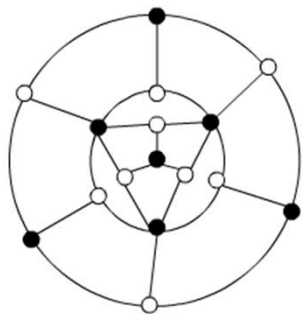
得 $W(G - V_1) > |V_1|$ 。

因此，图 G 不是哈密顿图。



必要条件2：标点法

例：判断下图是否有哈密顿回路和哈密顿路径？



黑色点：7个

白色点：9个

因此，既无哈密顿回路又无哈密顿路径。

用黑白两种颜色给图中的点着色，使相邻点的颜色不同。

(1) 对图中任何回路 C ，任取 C 中一白点 v ，在 C 中必存在 v 到 v 的闭路径 P ，此时除 v 外， C 中其余结点在 P 中恰好出现一次。

P 所经过的结点颜色必定是：白，黑，白，黑，...，白，黑，白。

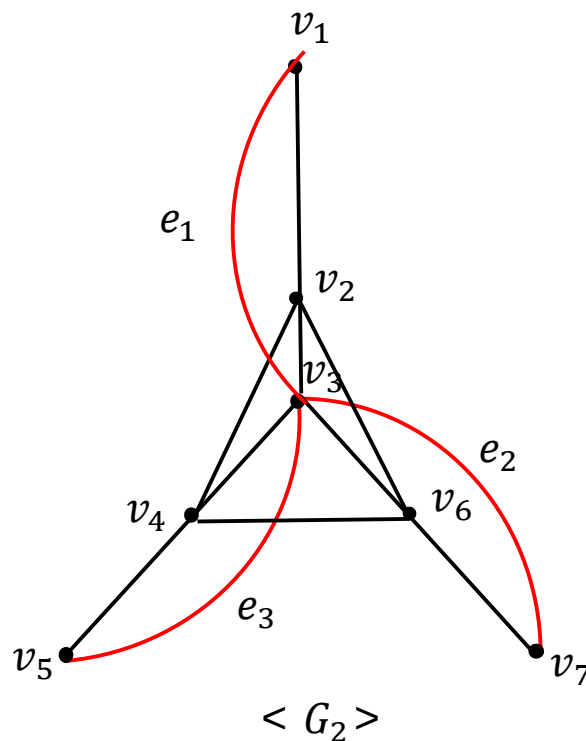
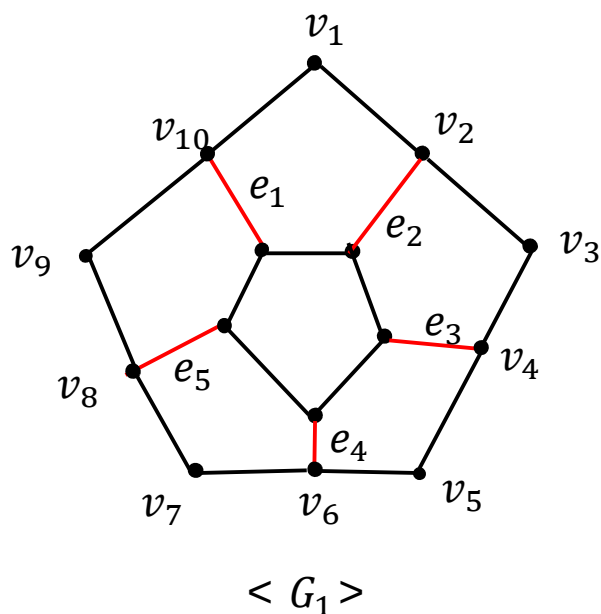
由于起、终点两白点为同一点 v ，故 C 中两色结点个数一定相等。

(2) 对图中任何基本路径 P ，假设 P 从白点出发，则 P 所经过的结点颜色必定是：白，黑，白，黑，...，最后一点可能是白色也可能是黑色，故 P 中两色结点个数相等或差 1



必要条件3：去边法

例：判断下面两图 G_1, G_2 是否哈密顿图



思想：考虑哈密顿回路 C ，图中每个结点都恰有两条和它关联的边在 C 上。因此，可以通过对每个结点去掉“多余的边”得到 C 。



货郎担问题

设有 n 个城市，城市之间均有道路，道路的长度均大于或等于0，可能是 ∞ （对应关联的城市之间无交通线）。一个旅行商从某个城市出发，要经过每个城市一次且仅一次，最后回到出发的城市，问他如何走才能使他走的路线最短？

- 这个问题可化归为如下的图论问题。

设 $G = \langle V, E \rangle$ ，为一个 n 阶完全带权图 K_n ，各边的权非负，且有的边的权可能为 ∞ 。求 G 中一条最短的哈密顿回路，这就是货郎担问题的数学模型。

- 此问题中不同哈密顿回路的含义：

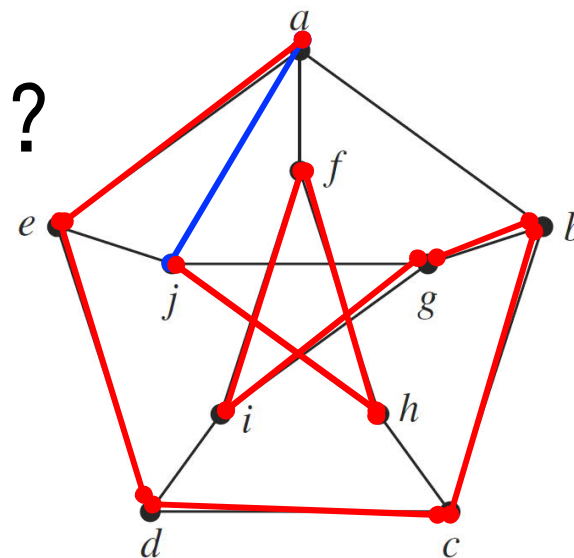
将图中生成圈看成一个哈密顿回路，即不考虑始点(终点)的区别以及顺时针行遍与逆时针行遍的区别。

- NP难



彼得森图

- 1、是否有欧拉回路？
- 2、加几条边有欧拉回路？
- 3、如何证明没有汉密尔顿回路？
- 4、是否有汉密尔顿通路？
- 5、加几条边有汉密尔顿回路？





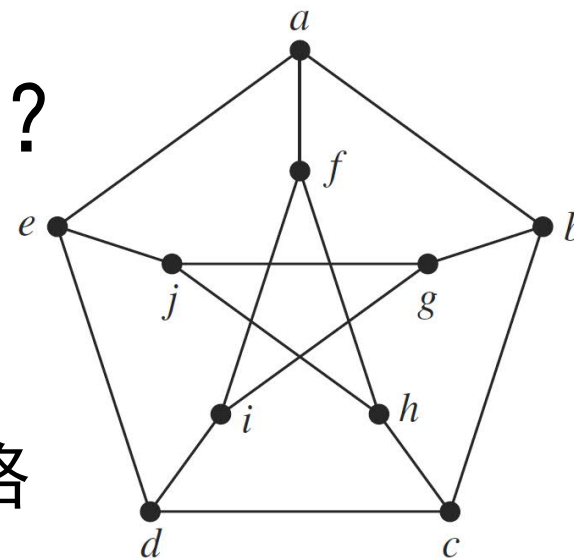
彼得森图

- 1、是否有欧拉回路？
- 2、加几条边有欧拉回路？
- 3、如何证明没有汉密尔顿回路？
- 4、是否有汉密尔顿通路？
- 5、加几条边有汉密尔顿回路？

证明思路：

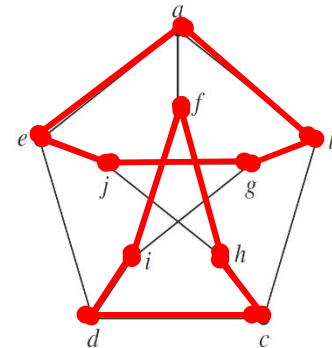
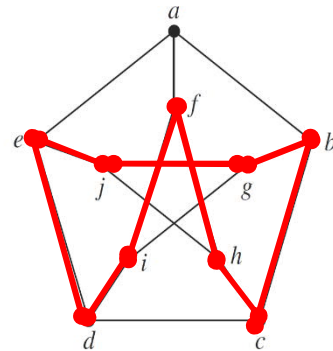
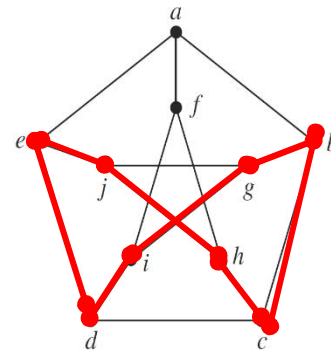
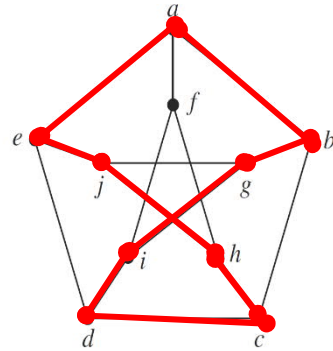
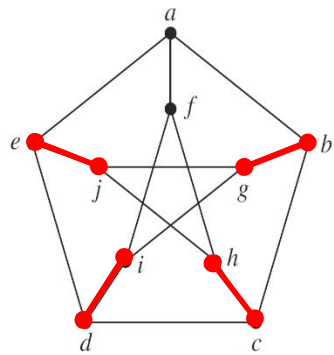
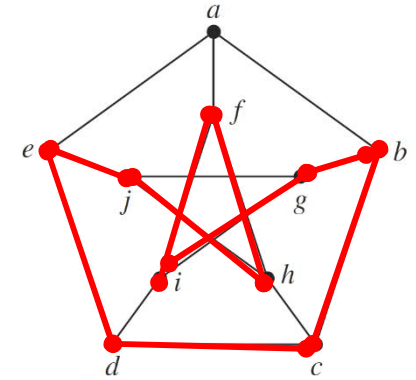
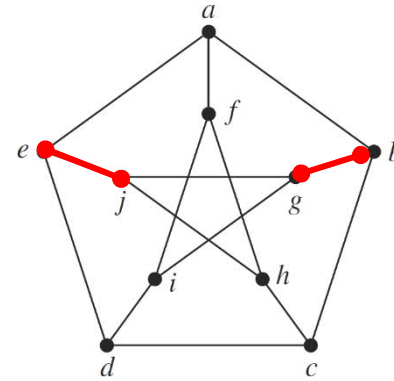
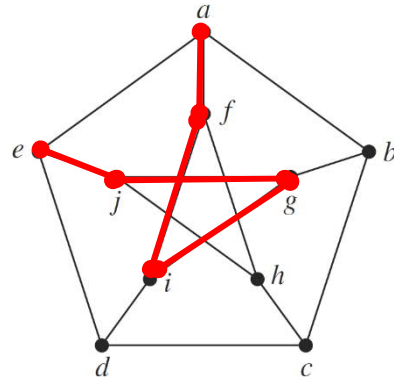
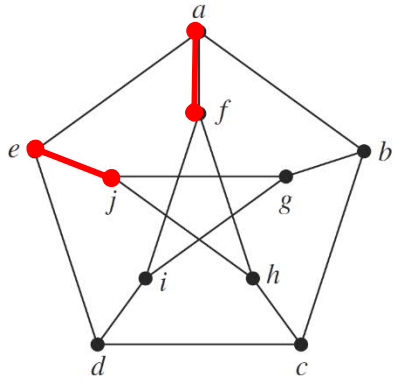
- 1、分为内外两部分
- 2、（反证法）假设存在汉密尔顿回路
考虑回路上跨越内外两部分的边数

- A、两条
- B、四条





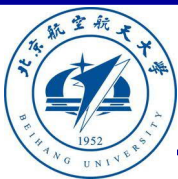
彼得森图





主要内容

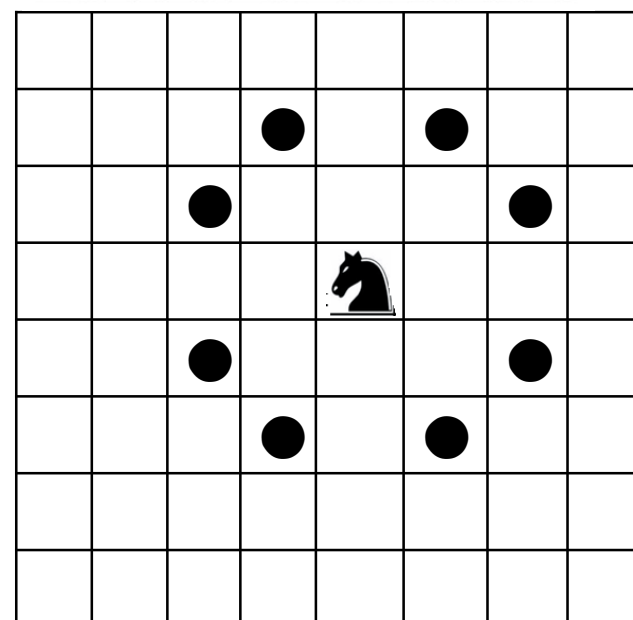
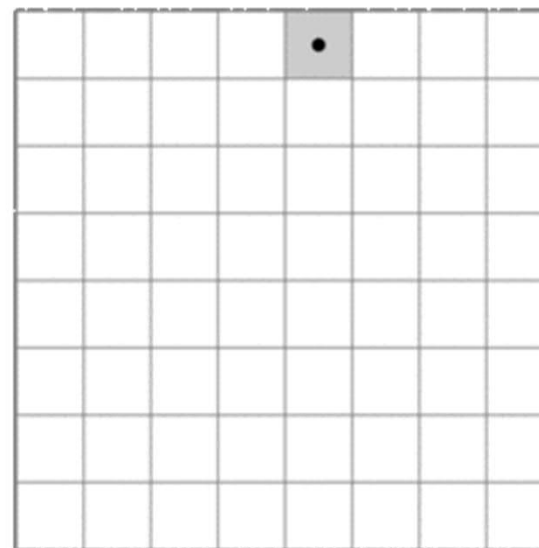
- 穿程问题
 - 欧拉图
 - 哈密顿图
 - 骑士周游
- 通路问题
 - 最短通路
 - 关键通路
- 匹配问题
 - 二分图
 - 二分图匹配



周游问题

- 骑士周游问题
- 旅行商问题
- 素数链问题
- 美国克雷数学研究所
 - 千年数学难题

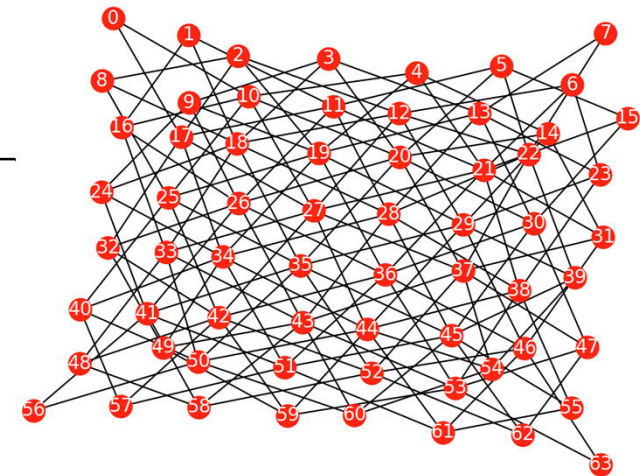
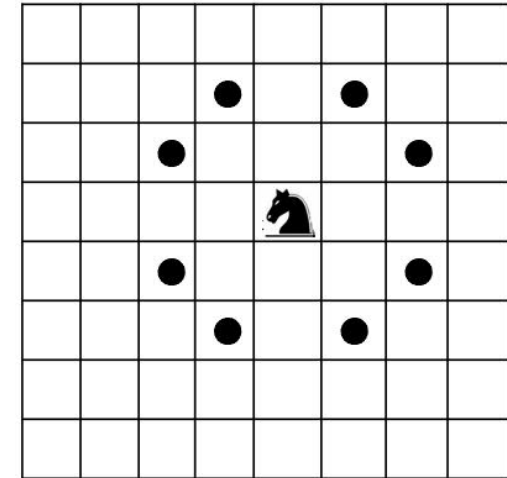
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63



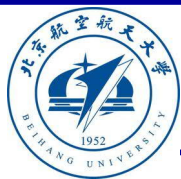


骑士周游图

```
def Knighttouregraph(n):  
    P8=[[-1,-2],[-2,-1],[-2,1],[-1,2],[1,2],[2,1],[2,-1],[1,-2]]  
    N=n*n  
    V=list(range(N))  
    E=set({})  
    for k in range(N):  
        j=k%n  
        i=int(k/n)  
        for [u,v] in P8:  
            if 0<=(i+u)and (i+u)<n and 0<=(j+v) and (j+v)<n:  
                E=E|{(k,(i+u)*n+(j+v))}  
    return [V,E]  
  
import graph.graph as gg  
[V,E]=gg.Knighttouregraph(8)  
gg.drawgraph(E)  
[d,di,do]=gg.degreeset(V,E)  
print(d)
```



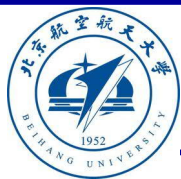
```
[4, 6, 8, 8, 8, 8, 6, 4, 6, 8, 12, 12, 12, 12, 8, 6, 8, 12, 16, 16, 16, 16, 12, 8, 8, 12, 16,  
16, 16, 16, 12, 8, 8, 12, 16, 16, 16, 16, 12, 8, 8, 12, 16, 16, 16, 16, 12, 8, 6, 8, 12,  
12, 12, 12, 8, 6, 4, 6, 8, 8, 8, 8, 6, 4]
```

骑士周游图 (续)

要求:将马随机放在国际象棋的 8×8 棋盘的某个方格中, 马按走棋规则(马走日字)进行移动。要求每个方格只进入一次, 走遍棋盘上全部 64 个方格。

1	12	9	6	3	14	17	20
10	7	2	13	18	21	4	15
31	28	11	8	5	16	19	22
64	25	32	29	36	23	48	45
33	30	27	24	49	46	37	58
26	63	52	35	40	57	44	47
53	34	61	50	55	42	59	38
62	51	54	41	60	39	56	43



骑士周游图 (续)

1	12	9	6	3	14	17	20
10	7	2	13	18	21	4	15
31	28	11	8	5	16	19	22
64	25	32	29	36	23	48	45
33	30	27	24	49	46	37	58
26	63	52	35	40	57	44	47
53	34	61	50	55	42	59	38
62	51	54	41	60	39	56	43



骑士周游图 (续)

```
import graph.graph as gg
n = 5
[V,E]=gg.Knighttouregraph(n)
v0 = 0
Ec = gg.Eulercircuit(E,v0)
paths = gg.Hamiltonpath(V,E,v0)
print("欧拉圈",Ec)
print("哈密顿回路",len(paths))
print(paths)
```

欧拉圈 (0, 7, 4, 7, 10, 21, 10, 7, 18, 11, 2, 11, 8, 11, 18, 21, 12, 1, 10, 17, 10, 1, 8, 17, 14, 7, 14, 17, 20, 17, 6, 15, 22, 13, 2, 5, 16, 7, 16, 13, 24, 17, 24, 13, 4, 13, 16, 5, 12, 19, 12, 3, 6, 17, 8, 1, 12, 9, 18, 9, 2, 13, 22, 19, 8, 19, 22, 15, 6, 13, 6, 3, 14, 23, 16, 23, 14, 3, 12, 5, 2, 9, 12, 23, 12, 15, 12, 21, 18, 7, 0, 11, 20, 11, 22, 11, 0)

哈密顿回路 304

{(0, 11, 20, 17, 24, 13, 16, 5, 2, 9, 12, 23, 14, 3, 6, 15, 22, 19, 8, 1, 10, 21, 18, 7, 4), (0, 7, 4, 13, 24, 17, 14, 23, 16, 5, 2, 9, 18, 21, 10, 1, 8, 19, 12, 3, 6, 15, 22, 11, 20), (0, 11, 20, 17, 24, 13, 4, 7, 10, 21, 18, 9, 2, 5, 16, 23, 14, 3, 6, 15, 22, 19, 12, 1, 8), (0, 11, 20, 17, 14, 23, 12, 3, 6, 15, 22, 19, 8, 1, 10, 21, 18, 9, 2, 5, 16, 7, 4, 13, 24), (0, 7, 16, 23, 14, 3, 6, 15, 12, 5, 2, 9, 18, 21, 10, 1, 8, 19, 22, 11, 20, 17, 24, 13, 4), (0, 7, 4, 13, 24, 17, 10, 21, 18, 9, 12, 1, 8, 19, 22, 15, 6, 3, 14, 23, 16, 5, 2, 11, 20),}

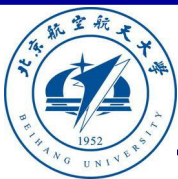


骑士周游图 (续)

```
import graph.graph as gg
n=5
[V,E]=gg.MNgraph(n,n)
gg.drawgraph(E)
paths=gg.Hamiltonpath(V,E,0)
print(len(paths))
```

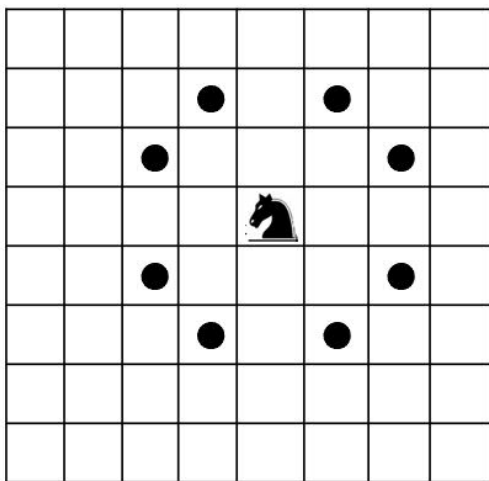
n=5
824

n=6
22144

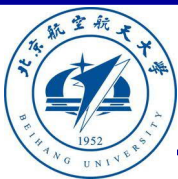


骑士周游图 – 欧拉图

- 骑士周游图是欧拉图吗？
- 骑士周游图是汉密尔顿图吗？

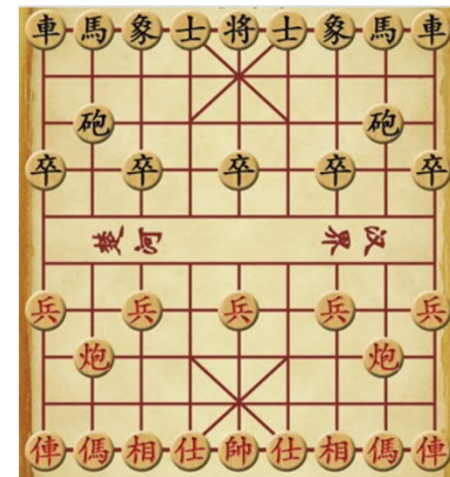
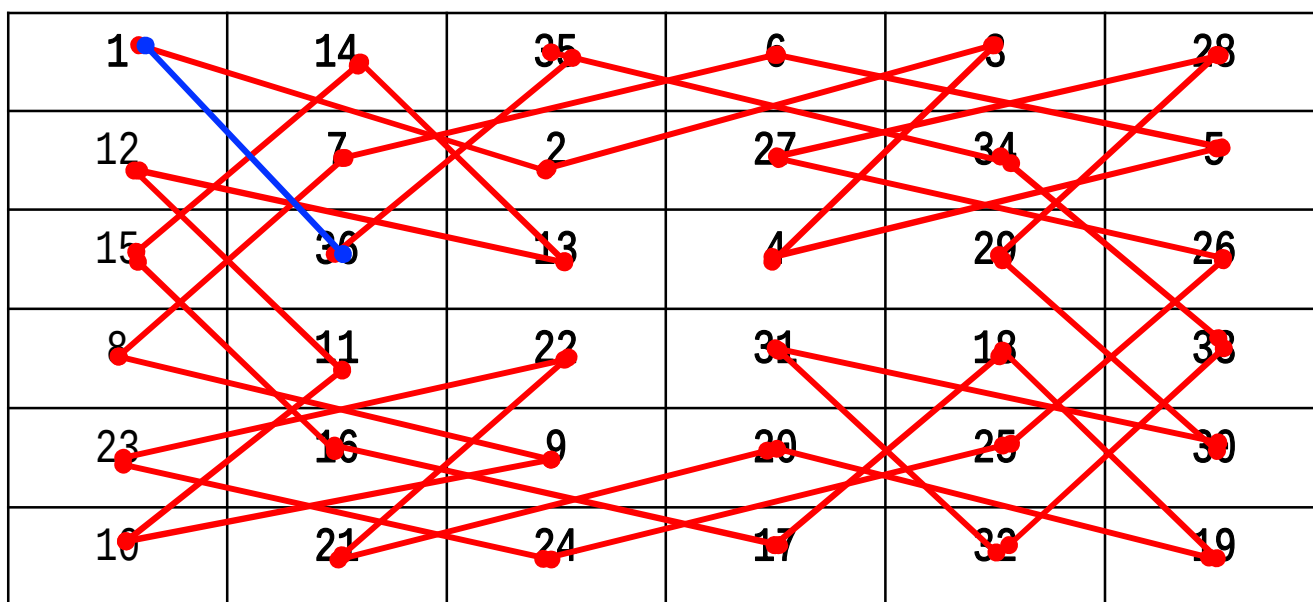


1	12	9	6	3	14	17	20
10	7	2	13	18	21	4	15
31	28	11	8	5	16	19	22
64	25	32	29	36	23	48	45
33	30	27	24	49	46	37	58
26	63	52	35	40	57	44	47
53	34	61	50	55	42	59	38
62	51	54	41	60	39	56	43



骑士周游图 – 欧拉图

- 在 $n \times n$ 的棋盘上, 当 $n \geq 5$ 且为偶数时, 以任意点作为起点, 都有解
- 骑士周游图的判定问题存在线性算法



THEOREM. An $m \times n$ chessboard with $m \leq n$ has a knight's tour unless one or more of these three conditions holds:

- (a) m and n are both odd;
- (b) $m = 1, 2$, or 4 ; or
- (c) $m = 3$ and $n = 4, 6$, or 8 .



主要内容

■ 穿程问题

- 欧拉图
- 哈密顿图
- 骑士周游

■ 通路问题

- 最短通路
- 关键通路

■ 匹配问题

- 二分图
- 二分图匹配



通路问题

- 某市启动村村通工程后，已修建了很多路，但尚未完成整个工程。不过路多了也烦恼，每次要从一个村镇到另一个村镇时，都有许多种道路方案可以选择，但**走最短距离总是每个人追求的目标**。
- 现在，已知一个村庄和另一个作为终点的村庄，请你计算出**最短需要行走**多少路程。



通路问题

- 通路问题有**最短路径问题**和**关键路径问题**。
- **最短路径问题**是指一个加权有向图，从开始节点出发到终止节点结束，求**权值最小通路**的问题。1959年迪杰斯特拉（E. Dijkstra）给出最短路径算法。
- **关键路径问题**是指一个带权有向图，从**开始节点**出发到**终止节点**结束，求节点的**最早完成时间等于最晚完成时间**所有节点构成通路的问题。



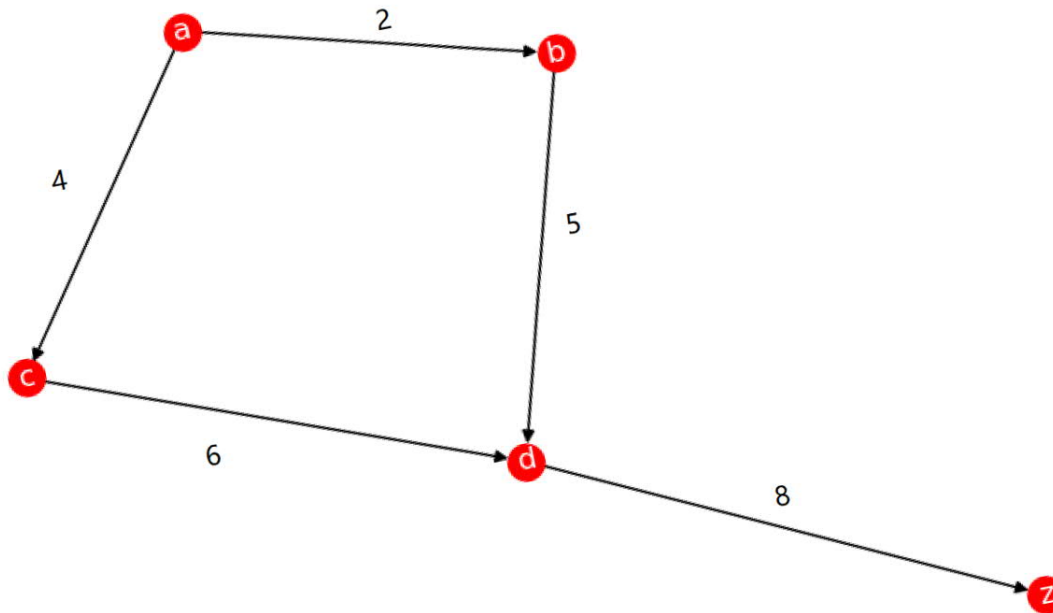
最短路径

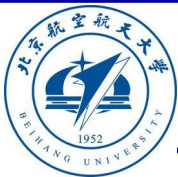
- **定义11.3.1（弧的长度）** 设 $G = \langle V, E \rangle$ 是加权有向图，对 G 的每一条弧 $e = \langle u, v \rangle$ ，都指定一个非负实数的权 $l(e) = l(u, v)$ ，称 $l(e)$ 为弧 e 的长度。
- **定义11.3.2（通路的长度）** 在加权有向图 G 中，一条通路的长度是这条通路所经过的各条弧的长度之和。即如果 p 是 G 中的一条通路，则 p 的长度为 $l(p) = \sum_{e \in p} l(e)$ 。



最短路径

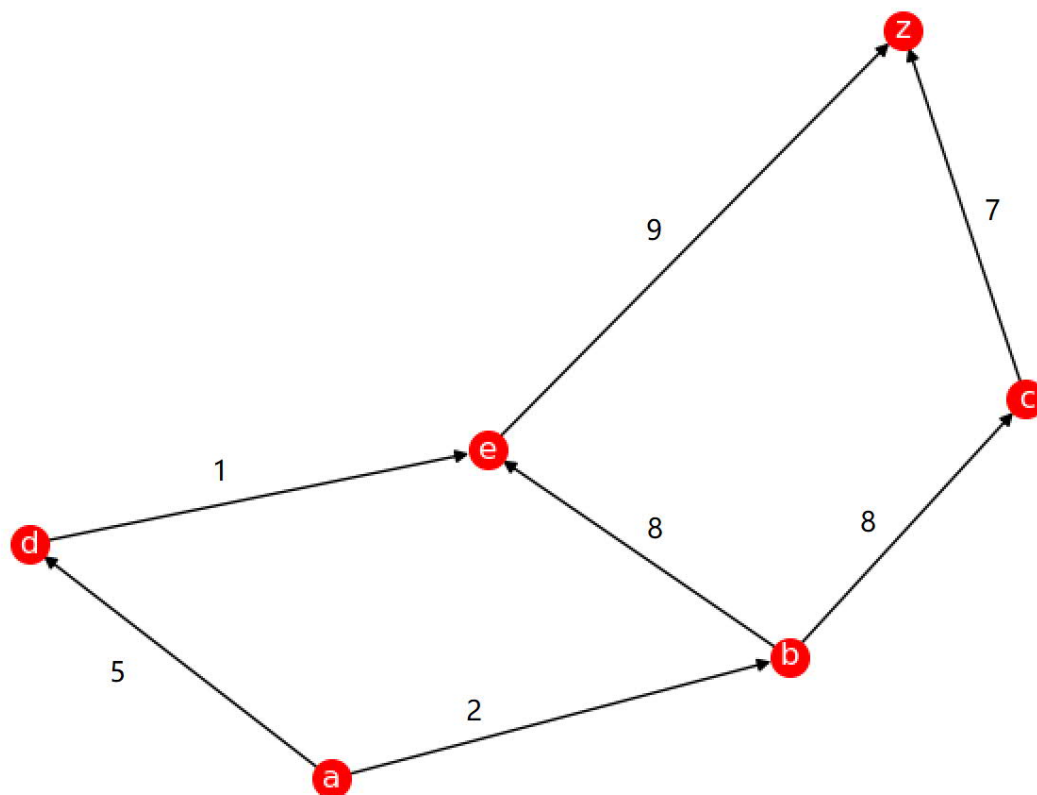
- **定义11.3.3（最短通路）**：在带权有向图 G 中，给定一个称为始点的节点 u 和一个称为终点的节点 v ，如果 P 是从 u 到 v 的通路中权值最小的通路，则称 P 为从 u 到 v 的最短通路。
- $[(2, 'a', 'b'), (4, 'a', 'c'), (5, 'b', 'd'), (6, 'c', 'd'), (8, 'd', 'z')]$





示例

- [(2,'a', 'b'), (5,'a', 'd'), (8,'b', 'c'), (8,'b', 'e'), (7,'c', 'z'), (1,'d', 'e'), (9,'e', 'z')]





Dijkstra 最短通路算法

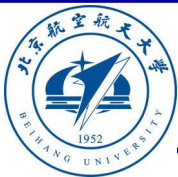
- Dijkstra 算法思路如下：
 - 1) 初始化 $H_x[0, n-1]$, $k = 0$
 - 2) 如果 $\langle w, u_k, v_m \rangle \in E_w$, 且 $H_x[k] + w < H_x[m]$, 则 $H_x[m] = H_x[k] + w$, 并且, 若 $p[-1] = u_k$, 则 $p = p + [v_m]$
 - 3) $k = k + 1$
 - 4) 如果 $k < n$, 则继续步骤2), 否则, 结束。



Dijkstra 最短通路算法

DIJKSTRA(G, w, s)

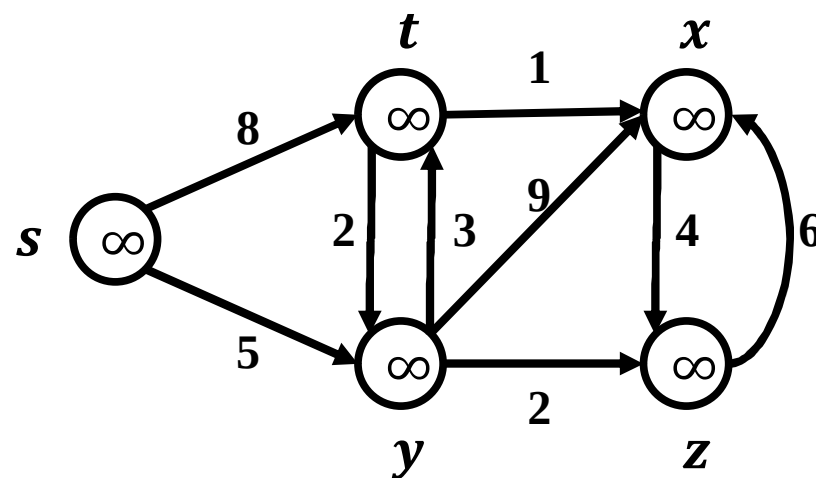
```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```

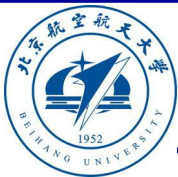


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	W	W	W	W	W
$pred$	N	N	N	N	N
$dist$	∞	∞	∞	∞	∞

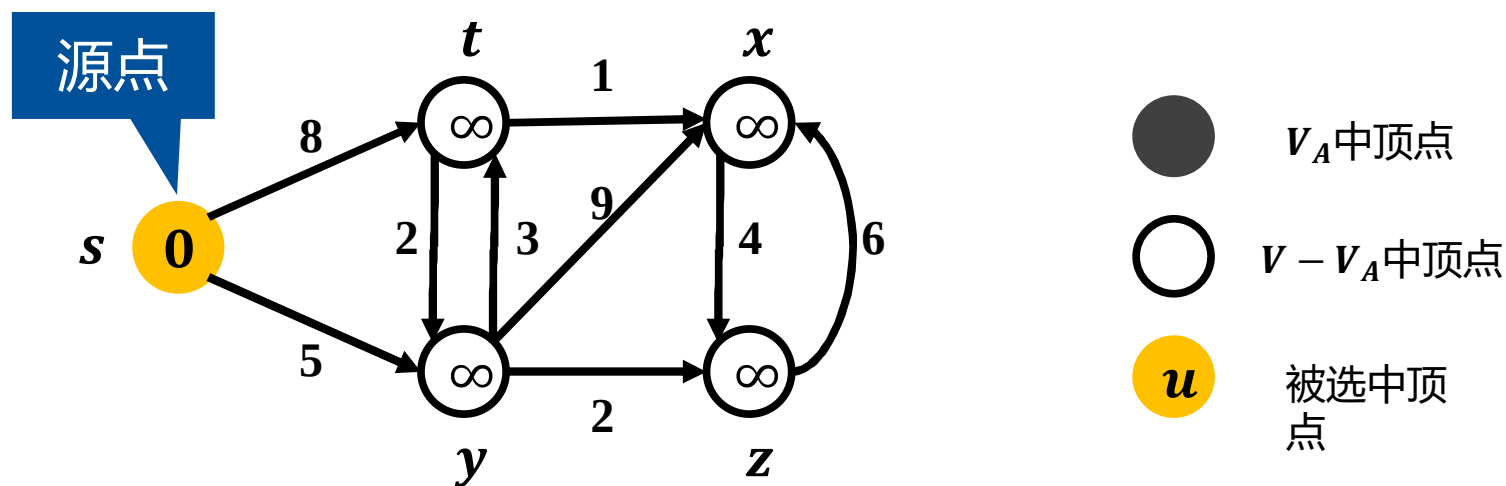


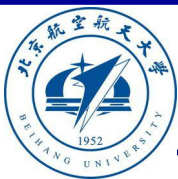


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	W	W	W	W	W
$pred$	N	N	N	N	N
$dist$	0	∞	∞	∞	∞

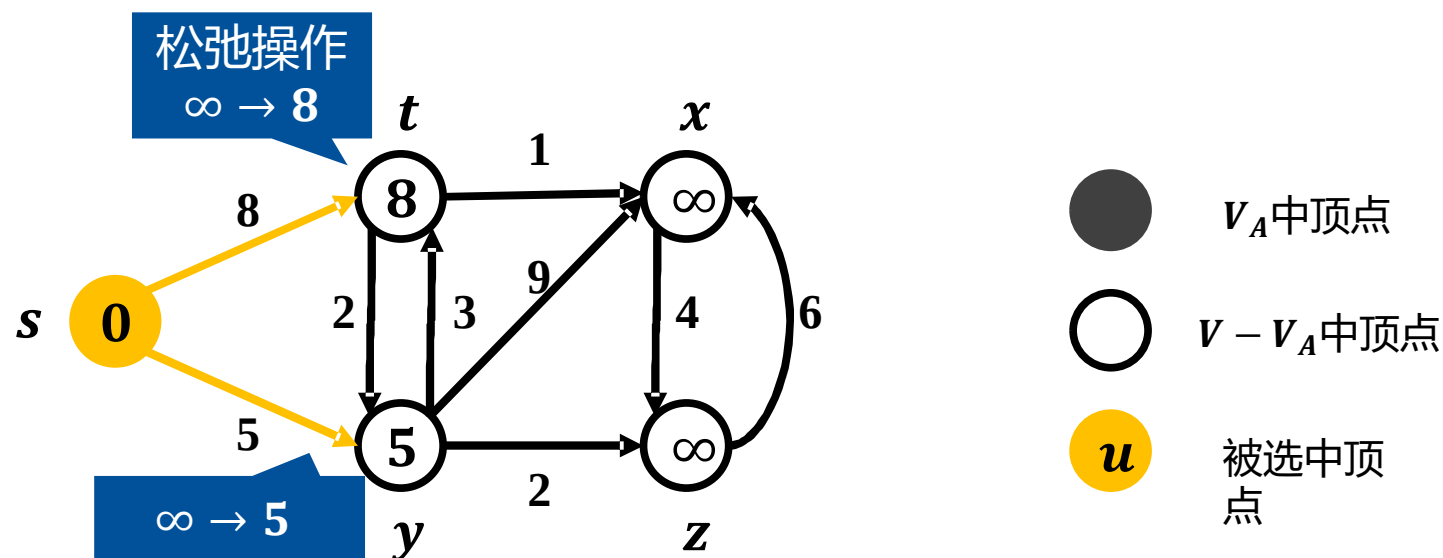


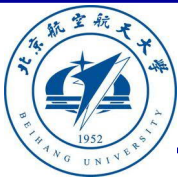


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	W	W	W	W	W
$pred$	N	s	N	s	N
$dist$	0	8	∞	5	∞

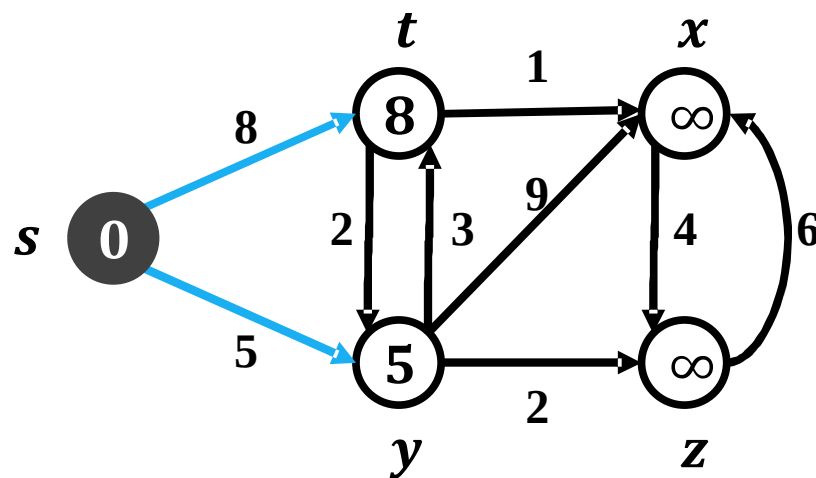




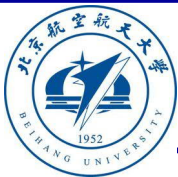
算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	W	W
$pred$	N	s	N	s	N
$dist$	0	8	∞	5	∞



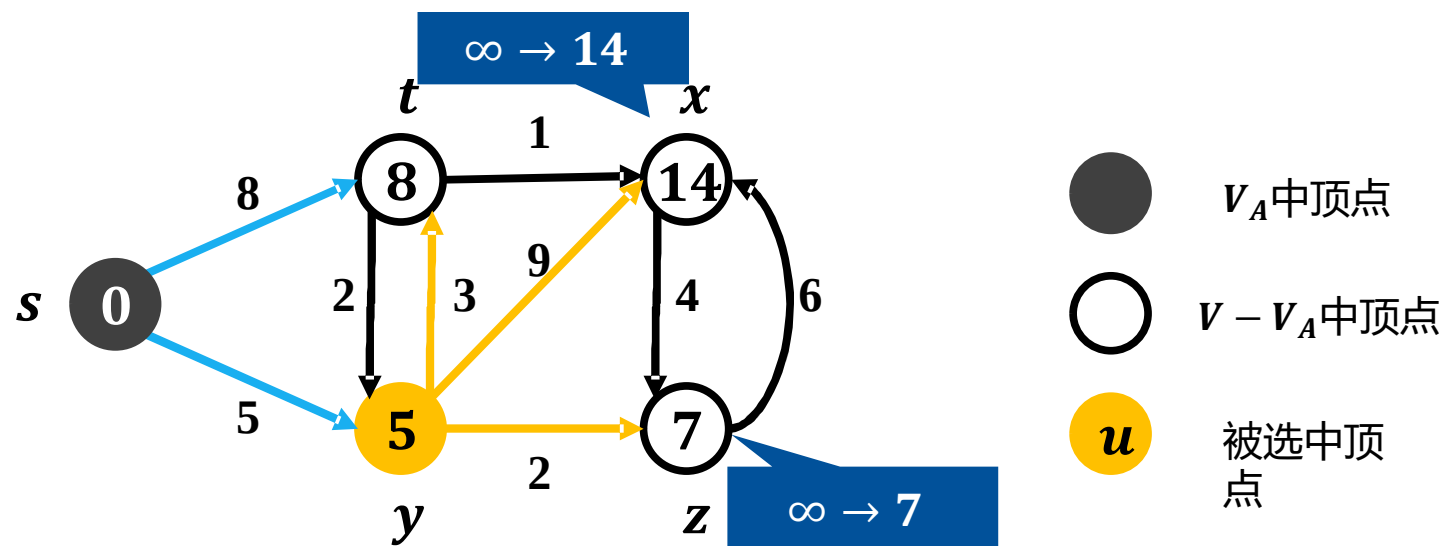
- V_A 中顶点
- $V - V_A$ 中顶点
- 被选中顶点 u

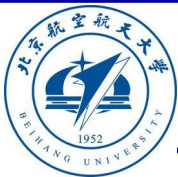


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	W	W
$pred$	N	s	y	s	y
$dist$	0	8	14	5	7

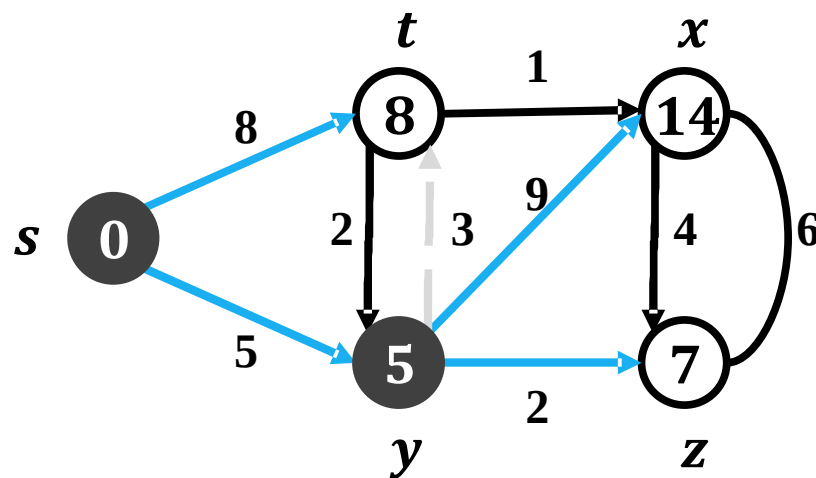




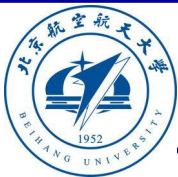
算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	B	W
$pred$	N	s	y	s	y
$dist$	0	8	14	5	7



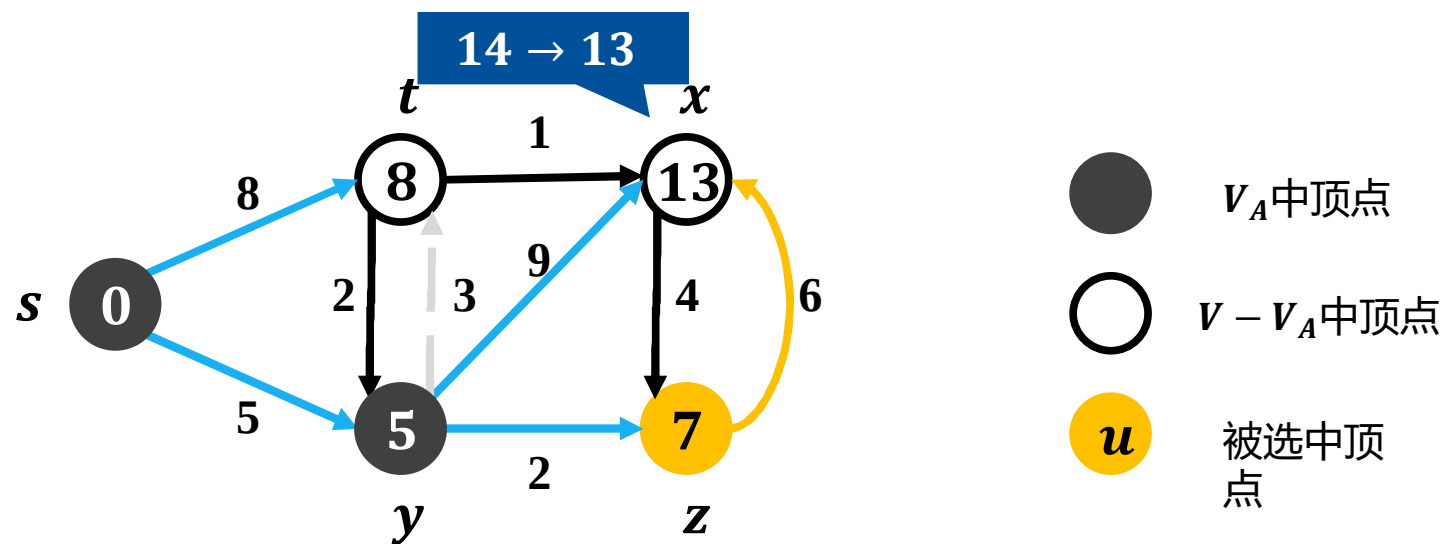
- V_A 中顶点
- $V - V_A$ 中顶点
- u 被选中顶点

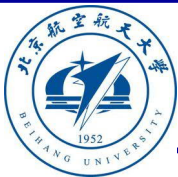


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	B	W
$pred$	N	s	z	s	y
$dist$	0	8	13	5	7

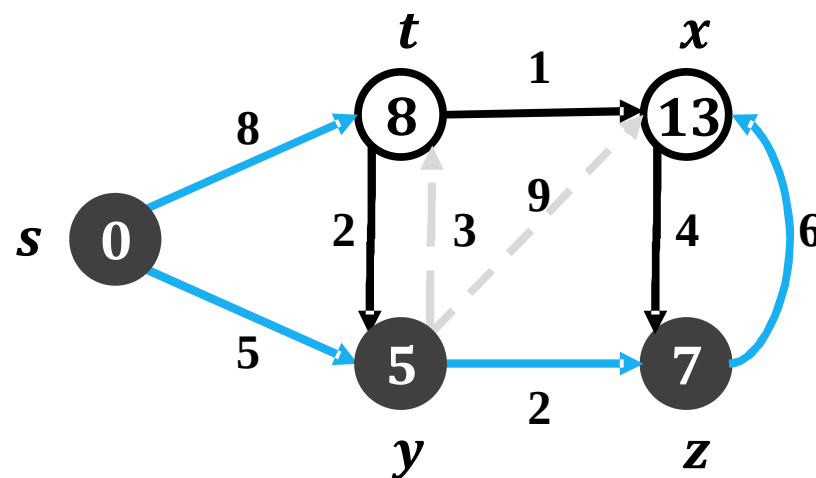


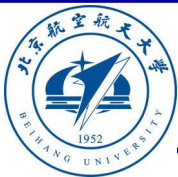


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	B	B
$pred$	N	s	z	s	y
$dist$	0	8	13	5	7

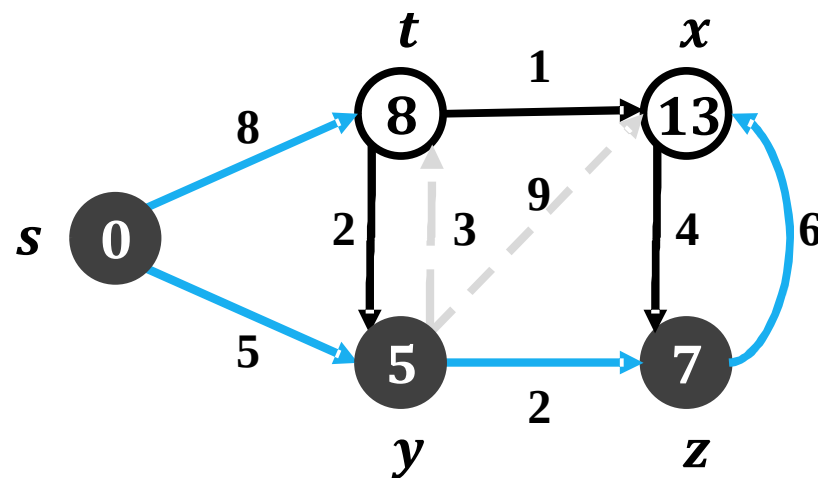


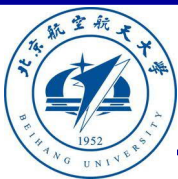


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	B	B
$pred$	N	s	z	s	y
$dist$	0	8	13	5	7

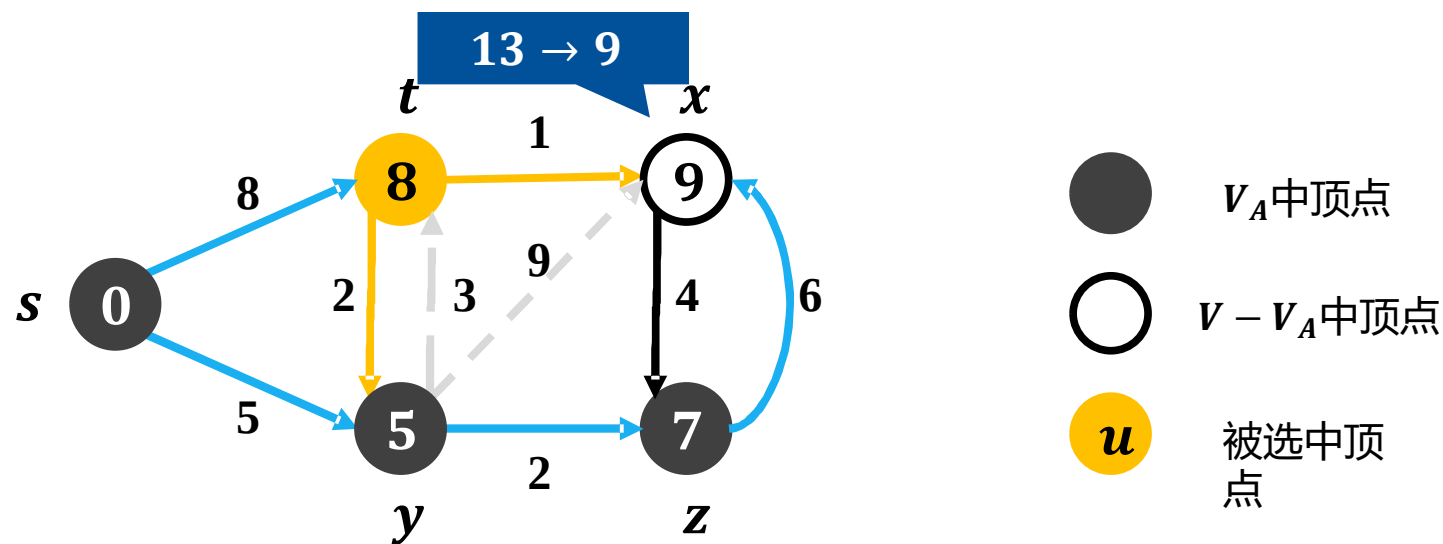


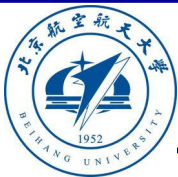


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	W	W	B	B
$pred$	N	s	t	s	y
$dist$	0	8	9	5	7

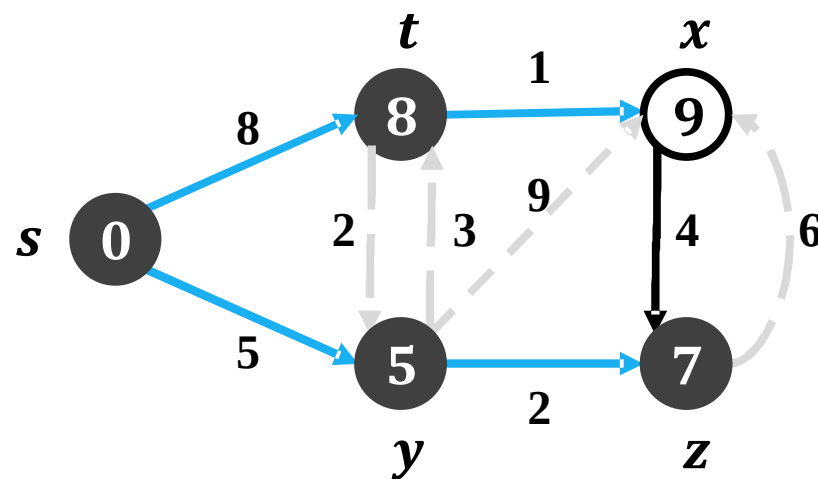




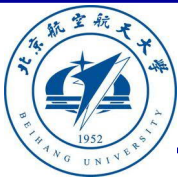
算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	B	W	B	B
$pred$	N	s	t	s	y
$dist$	0	8	9	5	7



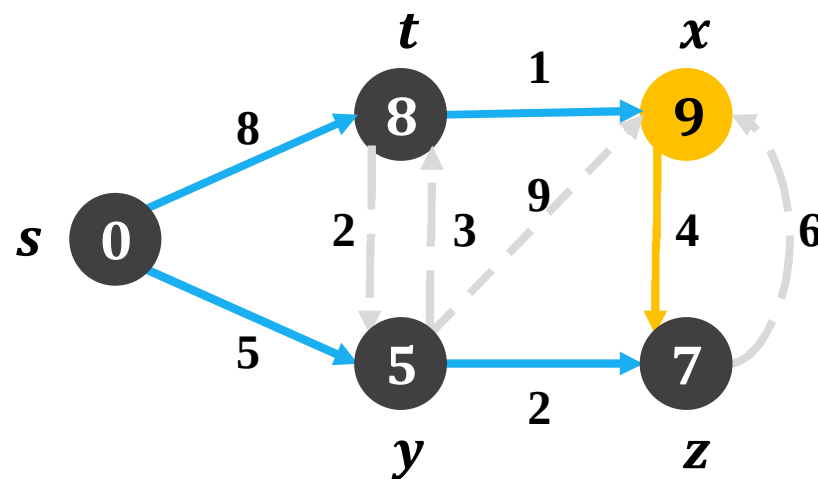
- V_A 中顶点
- $V - V_A$ 中顶点
- 被选中顶点 u



算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

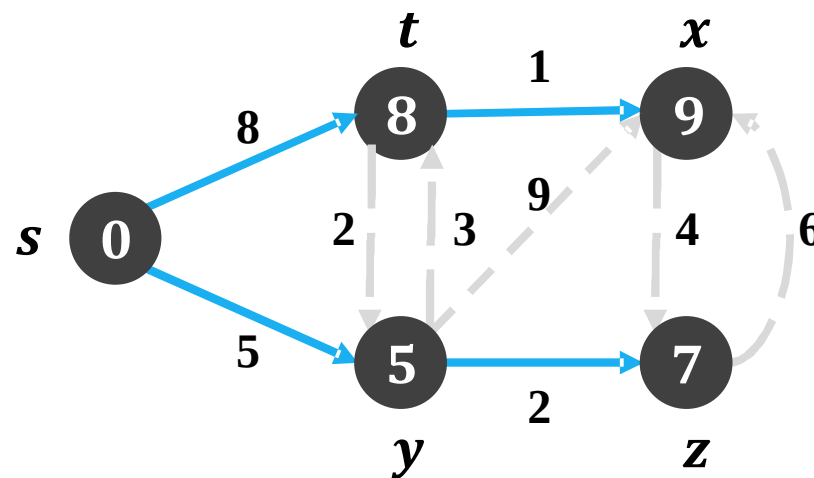
V	s	t	x	y	z
$color$	B	B	W	B	B
$pred$	N	s	t	s	y
$dist$	0	8	9	5	7



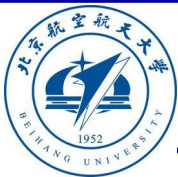


- 辅助数组
 - *dist*表示距离上界（估计距离）
 - *color*表示顶点状态;
黑色：已被确定; 白色：尚未确定
 - *pred*表示前驱顶点
 - (*pred*[*u*], *u*)为最短路径上的边

V	s	t	x	y	z
$color$	B	B	B	B	B
$pred$	N	s	t	s	y
$dist$	0	8	9	5	7



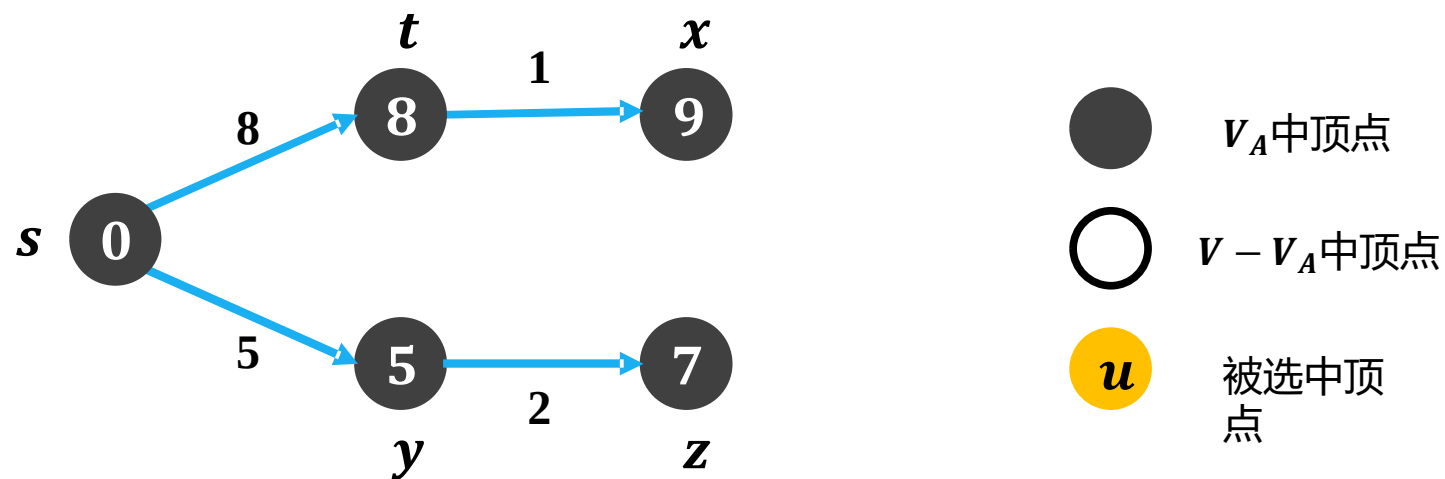
- V_A 中顶点
- $V - V_A$ 中顶点
- u 被选中顶点

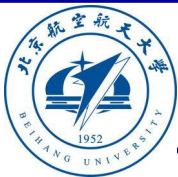


算法实例

- 辅助数组
 - $dist$ 表示距离上界 (估计距离)
 - $color$ 表示顶点状态;
 - 黑色: 已被确定; 白色: 尚未确定
 - $pred$ 表示前驱顶点
 - $(pred[u], u)$ 为最短路径上的边

V	s	t	x	y	z
$color$	B	B	B	B	B
$pred$	N	s	t	s	y
$dist$	0	8	9	5	7





Dijkstra 最短通路算法

获取权重最小的点

1、直接搜索

$$O(|E| + |V|^2)$$

2、heap

$$O((|E| + |V|) \log |V|)$$

3、Fibonacci heap

$$O(|E| + |V| \log |V|)$$

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```

Operation	find-max	delete-max	increase-key	insert	meld	make-heap ^[b]
Binary ^[8]	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
Fibonacci ^{[8][20]}	$\Theta(1)$	$O(\log n)$ am.	$\Theta(1)$ am.	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$



Dijkstra 最短通路算法

获取权重最小的点

1、直接搜索

$$O(|E| + |V|^2)$$

2、heap

$$O((|E| + |V|) \log |V|)$$

3、Fibonacci heap

$$O(|E| + |V| \log |V|)$$

DIJKSTRA(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $S = \emptyset$

3 $Q = G.V$

4 **while** $Q \neq \emptyset$

5 $u = \text{EXTRACT-MIN}(Q)$

6 $S = S \cup \{u\}$

7 **for each** vertex $v \in G.Adj[u]$

8 RELAX(u, v, w)

当权重都是很小的整数时，最优可达

$$O(|E| \log \log |V|)$$

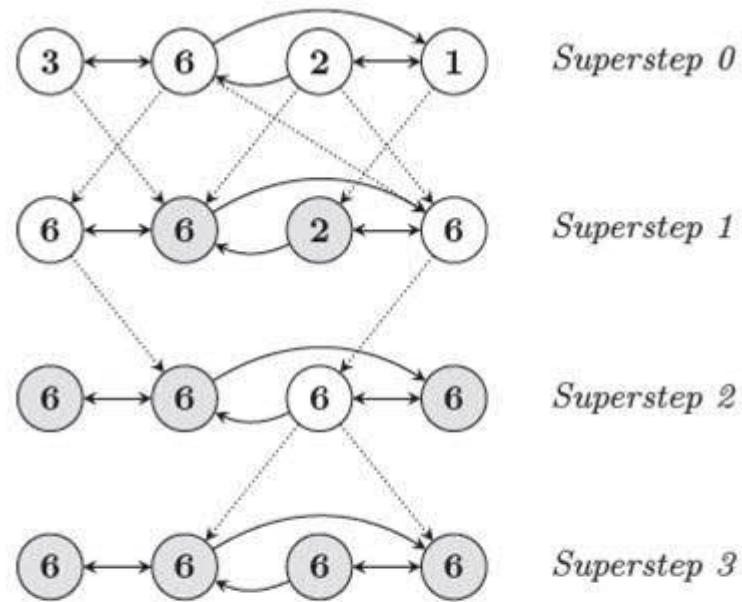
Universally Optimal Dijkstra's Algorithm via Heaps with the Working Set Property

"Universally optimal"指在所有情况下均表现最优的算法或设计



图计算

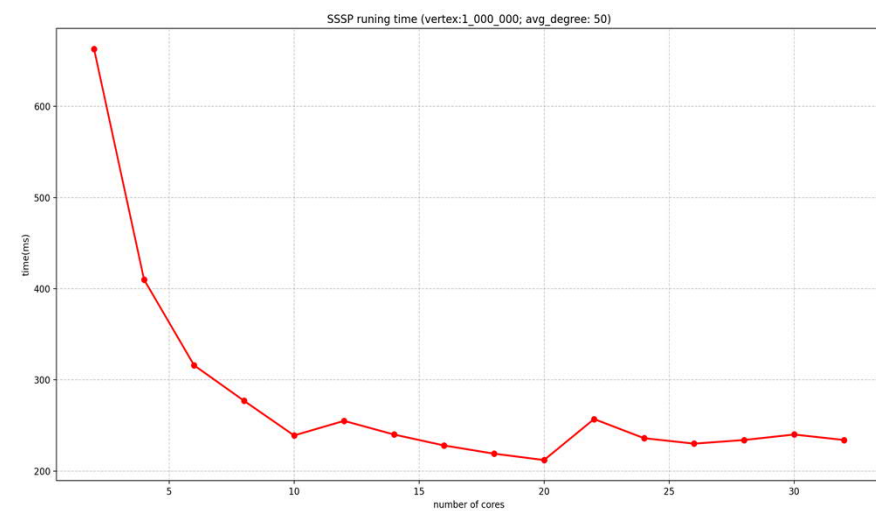
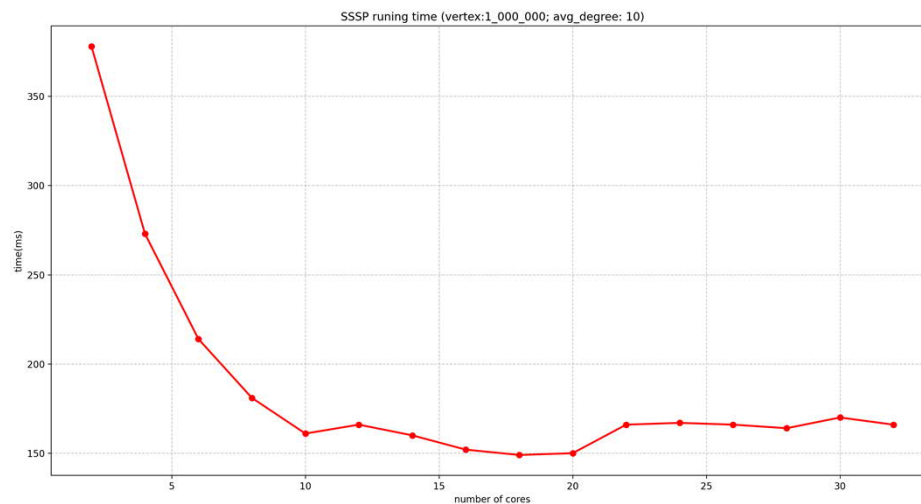
Google新 “三驾马车” —Caffeine、Pregel、Dremel





Pregel上的实现

```
class ShortestPathVertex
: public Vertex<int, int, int> {
void Compute(MessageIterator* msgs) {
    int mindist = IsSource(vertex_id()) ? 0 : INF;
    for (; !msgs->Done(); msgs->Next())
        mindist = min(mindist, msgs->Value());
    if (mindist < GetValue()) {
        *MutableValue() = mindist;
        OutEdgeIterator iter = GetOutEdgeIterator();
        for (; !iter.Done(); iter.Next())
            SendMessageTo(iter.Target(),
                           mindist + iter.GetValue());
    }
    VoteToHalt();
}
};
```





主要内容

■ 穿程问题

- 欧拉图
- 哈密顿图
- 骑士周游

■ 通路问题

- 最短通路
- 关键通路

■ 匹配问题

- 二分图
- 二分图匹配



工序流程图

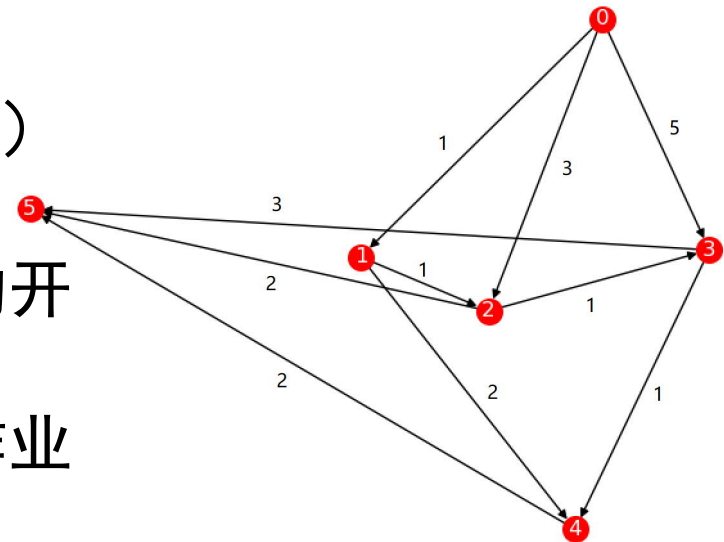
例题：某项目的任务是对A公司的办公室重新进行装修如果10月1日前完成装修工程，项目最迟应该何时开始?(需要完成的活
动、每个活动所需时间、及先期需完成工作如下图所示)

序号	项目	时间(天)	先期完成
1	清空办公室	3	无
2	拆除非承重墙	2	1
3	装修天花板	4	1
4	安装办公家具	5	2
5	重新布线	8	2
6	装修墙壁	3	3
7	装修地板	5	6
8	安装智能系统	10	5
9	清扫办公室	2	4,7,8
合计		42	



工序流程图

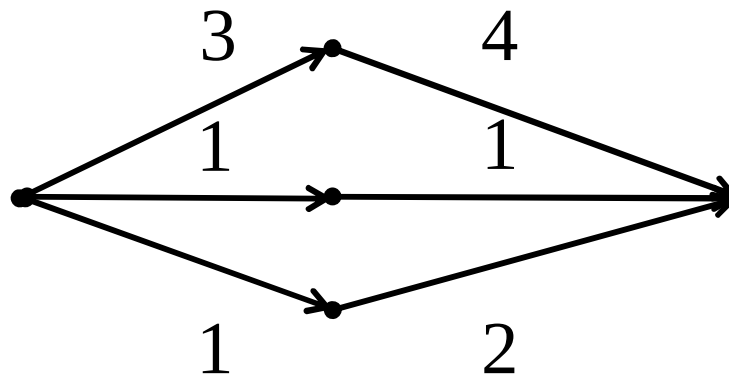
- 在一个表示工序的带权有向图中，用节点表示事件，用有向边表示活动，边上的权值表示活动的持续时间，构成带权有向图称为工序流程图。
- 工序流程图是没有平行边和回路的有限有向图。
- 引入次数为0的节点称为始点，引出次数为0的节点称为终点。仅有一个始点和终点。
- 图中的每条边都有指定的权（时间）
 - 边的意义是表示作业
 - 由边所连接的节点分别表示作业的开始时刻和结束时刻
 - 每一条边上的权值表示完成这一作业所需要的时间。





最早完成时间

- **定义11.3.4:** 在一给定的工序流线图中, 设从开始点 u_1 到终点 u_i 有 m 条通路 $p_1, p_2, \dots, p_m (m \geq 1)$, $t(p_i)$ 表示通路 p_i 上各个作业所用的时间之和, 即构成通路 p_i 的各条边上的权值之和, 则称 $t(p_1), t(p_2), \dots, t(p_m)$ 中的最大值为事件 u_i 的最早完成时间, 表示为 $TE(u_i)$, 即 u_1 到 u_j 的时间最长通路, $TE(u_j) = \max\{t(p_i)\}$





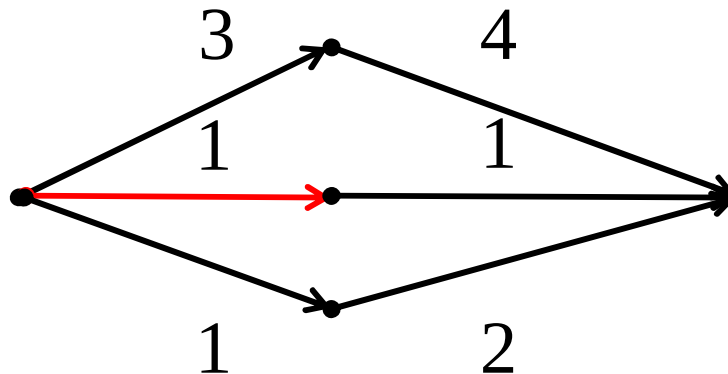
最早完成时间

- **定义11.3.4:** 在一给定的工序流线图中, 设从开始点 u_1 到终点 u_i 有 m 条通路 $p_1, p_2, \dots, p_m (m \geq 1)$, $t(p_i)$ 表示通路 p_i 上各个作业所用的时间之和, 即构成通路 p_i 的各条边上的权值之和, 则称 $t(p_1), t(p_2), \dots, t(p_m)$ 中的最大值为事件 u_i 的**最早完成时间**, 表示为 $TE(u_i)$, 即 u_1 到 u_j 的时间最长通路, $TE(u_j) = \max\{t(p_i)\}$
- 求事件 u_j 的最早完成时间的算法思路如下:
 - $TE(u_1) = 0$
 - $TE(u_j) = \max\{TE(u_i) + w_{ij}\}$, 其中 u_i 为邻接 u_j 的节点, w_{ij} 为其权值



最迟完成时间

- **定义11.3.5:** 在一给定的工序流线图中, 事件 u_j 的最迟完成时间, 是完成该事件所需时间的最大值, 使得用这个最大完成时间完成该事件后, 不会增加整个计划的最早完成时间。事件 u_j 的**最迟完成时间**表示为 $TL(u_j)$, 即 $TL(u_j) = TE(u_n) - \max\{t(p'_i)\}$, 其中 $t(p'_i)$ 为 u_j 到 u_n 的通路 p'_i 上的各个作业所用的时间之和。





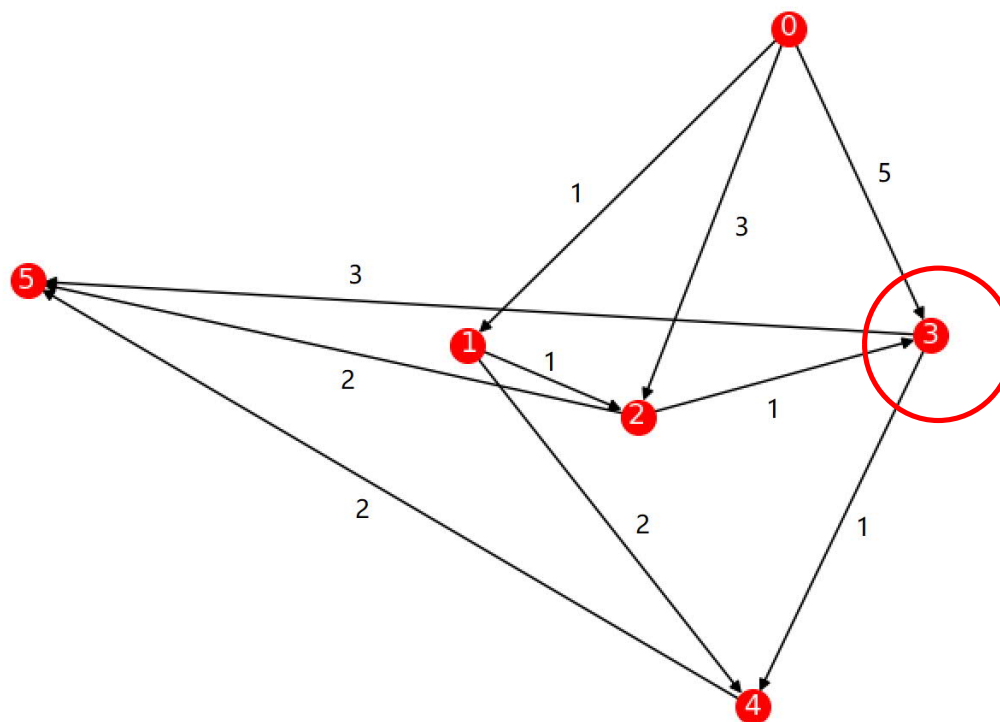
最迟完成时间

- **定义11.3.5:** 在一给定的工序流程图中, 事件 u_j 的最迟完成时间, 是完成该事件所需时间的最大值, 使得用这个最大完成时间完成该事件后, 不会增加整个计划的最早完成时间。事件 u_j 的**最迟完成时间**表示为 $TL(u_j)$, 即 $TL(u_j) = TE(u_n) - \max\{t(p'_i)\}$, 其中 $t(p'_i)$ 为 u_j 到 u_n 的通路 p'_i 上的各个作业所用的时间之和。
- 求事件 u_j 的最迟完成时间的算法思路如下:
 - $TL(u_n) = TE(u_n)$
 - $TL(u_j) = \min\{TL(u_k) - w_{jk}\}$, 其中 u_k 为 u_j 的邻居节点, w_{jk} 为其权值



性质

- 1) 只有在某节点所代表的事件发生后，从该节点出发的各活动才能开始；
- 2) 只有在进入某节点的各活动都结束，该节点所代表的事件才能发生。





关键通路

- **定义11.3.6:** 在一个给定的工序流程图中, 从始点到终点的这样一条通路称为**关键通路**, 在这条通路的各个节点上, 其 TE 值与 TL 值相等, 并且各边的权值之和为最大值。
- 关键通路可能**不止一条**
- 关键通路上**任何事件耽误时间 t** , 则整个计划就耽误时间 t 。
- $TE(u_k) == TL(u_k)$

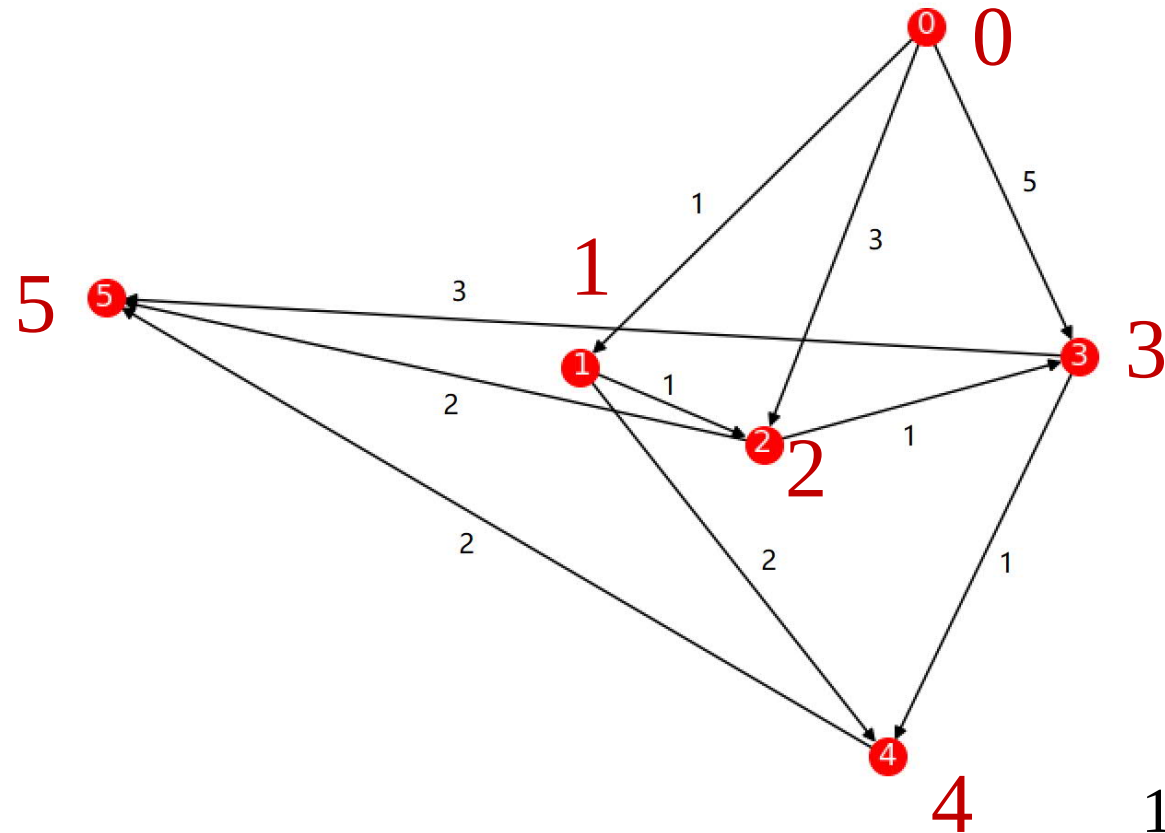


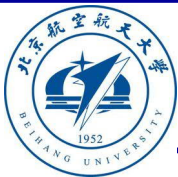
示例

$V=[0,1,2,3,4,5]$

$E=\{(0,1),(0,2),(0,3),(1,2),(1,4),(2,3),(2,5),(3,4),(3,5),(4,5)\}$

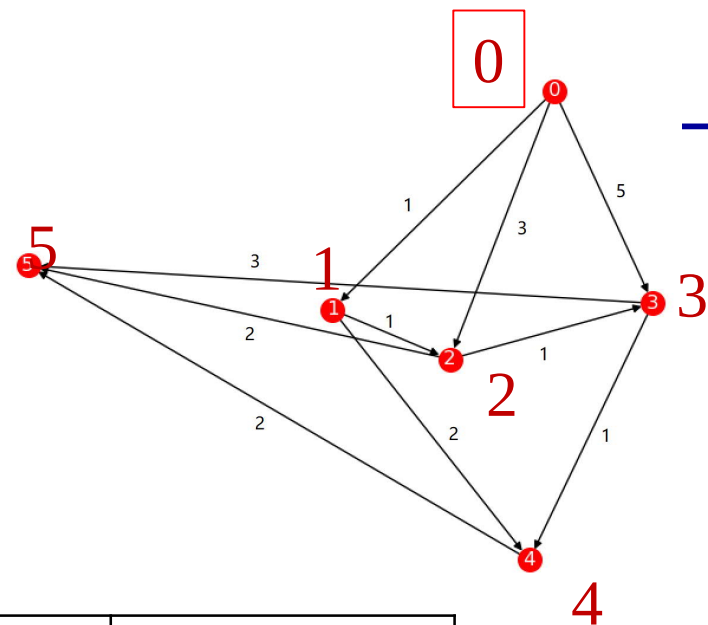
$E_w=\{(1,0,1),(3,0,2),(5,0,3),(1,1,2),(2,1,4),(1,2,3),(2,2,5),(1,3,4),(3,3,5),(2,4,5)\}$





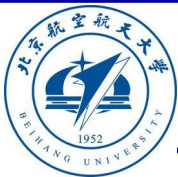
关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



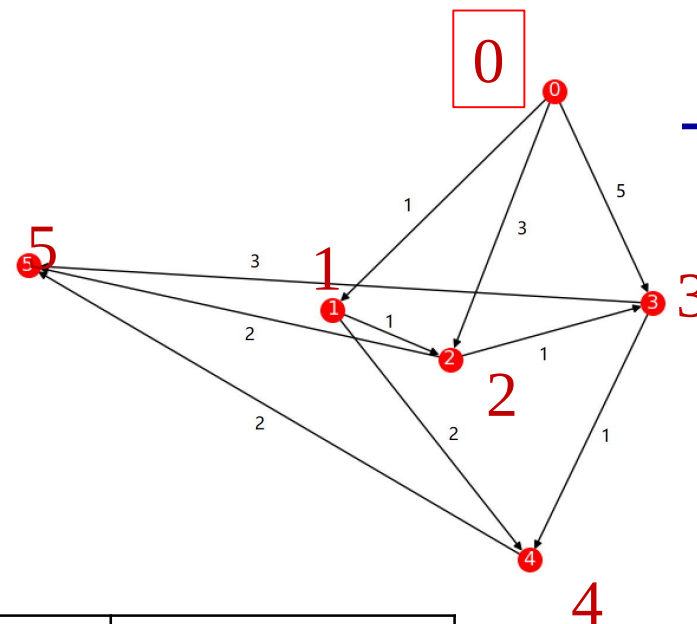
0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=∞		TL(1)=∞	
2	TE(2)=∞		TL(2)=∞	
3	TE(3)=∞		TL(3)=∞	
4	TE(4)=∞		TL(4)=∞	
5	TE(5)=∞		TL(5)=∞	

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



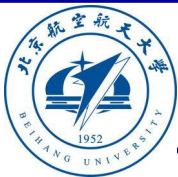
关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



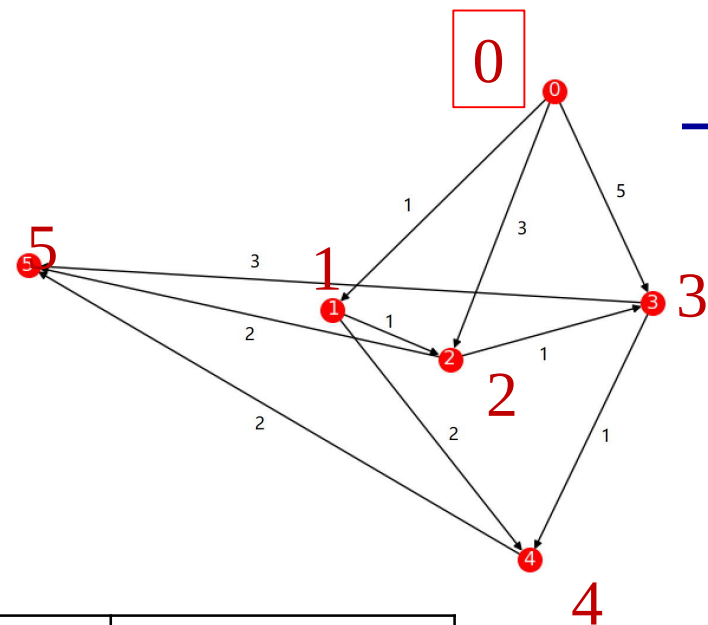
0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=1	[0, 1]	TL(1)=∞	
2	TE(2)=3	[0, 2]	TL(2)=∞	
3	TE(3)=5	[0, 3]	TL(3)=∞	
4	TE(4)=∞		TL(4)=∞	
5	TE(5)=∞		TL(5)=∞	

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



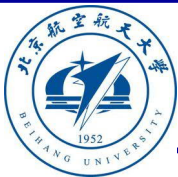
关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



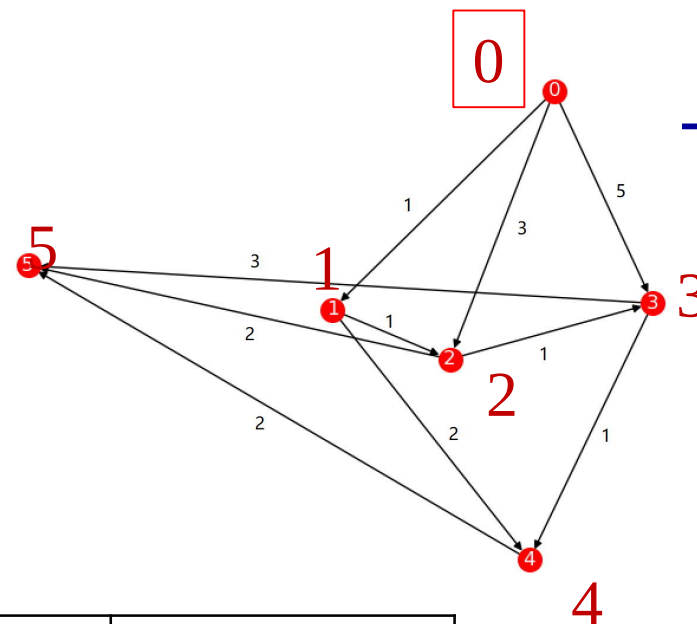
0	$TE(0)=0$	[0]	$TL(0)=0$	[0]
1	$TE(1)=1$	[0, 1]	$TL(1)=\infty$	
2	$TE(2)=3$	[0, 2]	$TL(2)=\infty$	
3	$TE(3)=5$	[0, 3]	$TL(3)=\infty$	
4	$TE(4)=6$	[0, 3, 4]	$TL(4)=\infty$	
5	$TE(5)=8$	[0, 3, 5]	$TL(5)=\infty$	

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



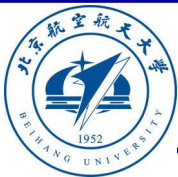
关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



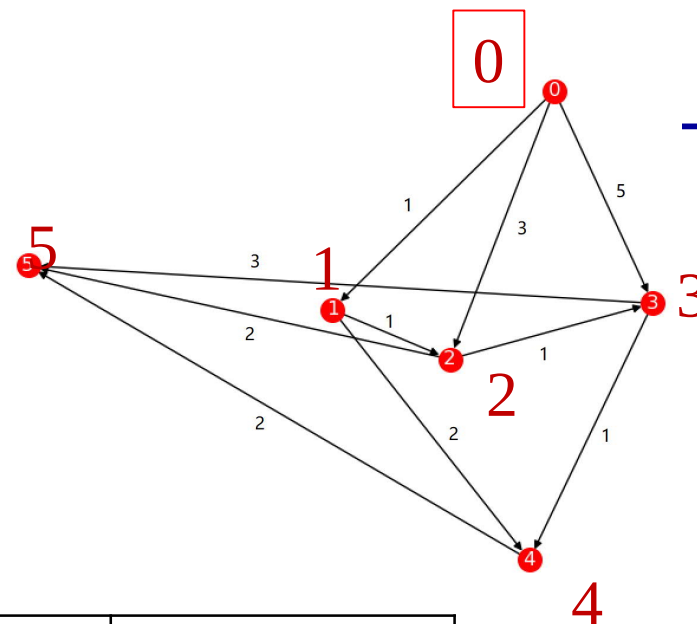
0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=1	[0, 1]	TL(1)=∞	
2	TE(2)=3	[0, 2]	TL(2)=∞	
3	TE(3)=5	[0, 3]	TL(3)=∞	
4	TE(4)=6	[0, 3, 4]	TL(4)=∞	
5	TE(5)=8	[0, 3, 5]	TL(5)=8	[0, 3, 5]

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



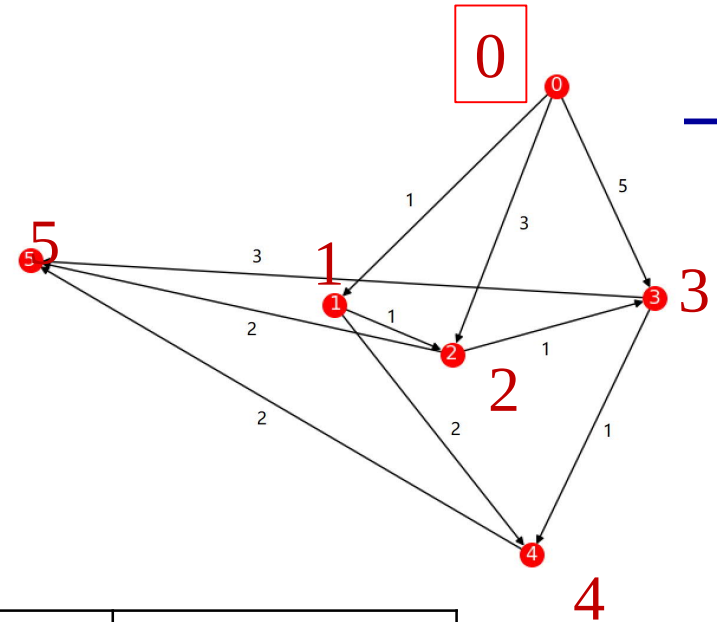
0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=1	[0, 1]	TL(1)=∞	
2	TE(2)=3	[0, 2]	TL(2)=6	[2,5]
3	TE(3)=5	[0, 3]	TL(3)=5	[3,5]
4	TE(4)=6	[0, 3, 4]	TL(4)=6	[4,5]
5	TE(5)=8	[0, 3, 5]	TL(5)=8	[0, 3, 5]

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



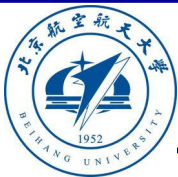
关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



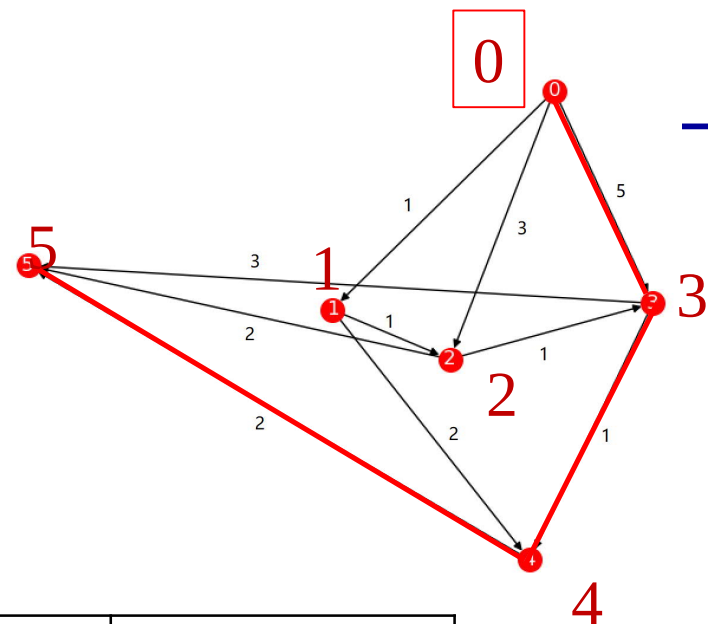
0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=1	[0, 1]	TL(1)=4	[1,4,5]
2	TE(2)=3	[0, 2]	TL(2)=6	[2,5]
3	TE(3)=5	[0, 3]	TL(3)=5	[3,5], [3,4,5]
4	TE(4)=6	[0, 3, 4]	TL(4)=6	[4,5]
5	TE(5)=8	[0, 3, 5]	TL(5)=8	[0, 3, 5]

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



关键路径

- $TE(u_1) = 0$
- $TE(u_k) = \max\{TE(u_i) + w(u_i, u_k)\}$
- $TL(u_n) = TE(u_n)$
- $TL(u_k) = \min\{w(u_k, u_j) + TL(u_j)\}$



0	TE(0)=0	[0]	TL(0)=0	[0]
1	TE(1)=1	[0, 1]	TL(1)=4	[1,4,5]
2	TE(2)=3	[0, 2]	TL(2)=6	[2,5]
3	TE(3)=5	[0, 3]	TL(3)=5	[3,5], [3,4,5]
4	TE(4)=6	[0, 3, 4]	TL(4)=6	[4,5]
5	TE(5)=8	[0, 3, 5]	TL(5)=8	[0, 3, 5]

$E_w = \{(1,0,1), (3,0,2), (5,0,3), (1,1,2), (2,1,4), (1,2,3), (2,2,5), (1,3,4), (3,3,5), (2,4,5)\}$



主要内容

■ 穿程问题

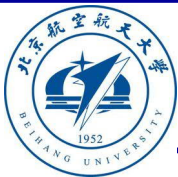
- 欧拉图
- 哈密顿图
- 骑士周游

■ 通路问题

- 最短通路
- 关键通路

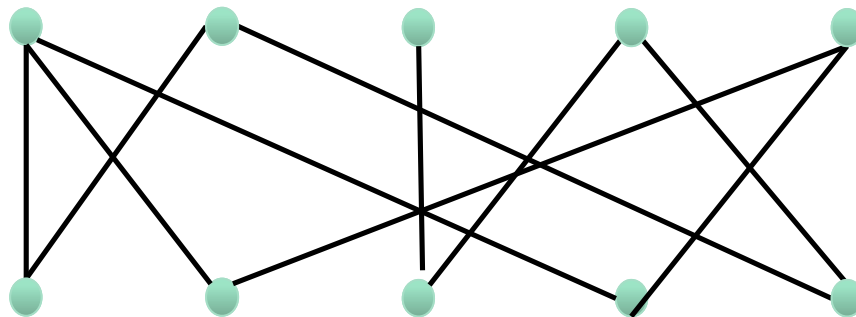
■ 匹配问题

- 二分图
- 二分图匹配



飞行员配对方案问题

- 第二次世界大战时期，英国皇家空军从沦陷国征募了大量外籍飞行员。由皇家空军派出的每一架飞机都需要配备在航行技能和语言上能互相配合的2名飞行员，其中**1名是英国飞行员，另1名是外籍飞行员**。在众多的飞行员中，每一名外籍飞行员都可以与其他若干名英国飞行员很好地配合。如何选择配对飞行的飞行员才能使一次派出最多的飞机。





二分图

- **定义11.4.1:** 设 $G = \langle V, E \rangle$ 是一个无向图, 如果将 V 划分为两个子集 V_1 和 V_2 , 使得同一子集中的任何两个节点都不邻接, 则称图 G 为一个**二分图**, 子集 V_1 和 V_2 称为 G 的互补子集。
 - $V = V_1 \cup V_2$
 - $V_1 \cap V_2 = \emptyset$
 - $\forall u \forall v ((u, v) \in E \rightarrow u \in V_1 \wedge v \in V_2)$

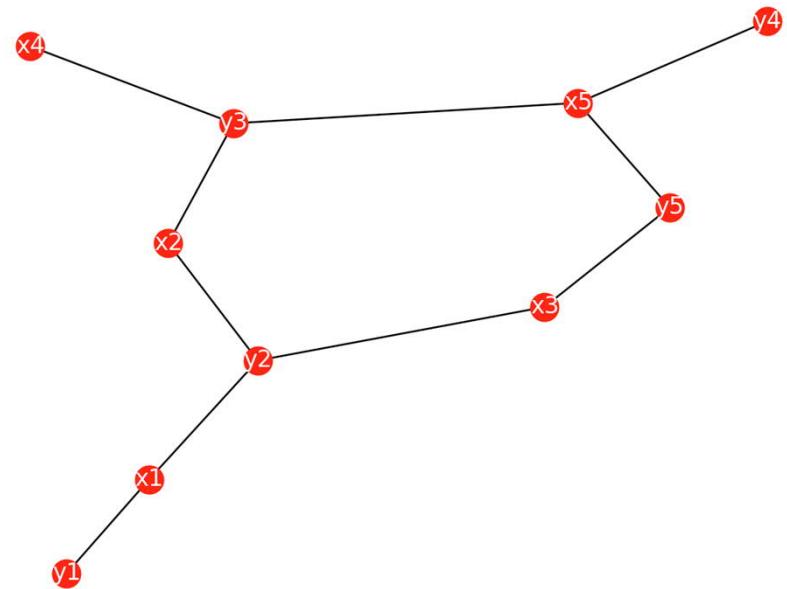


二分图判断算法

```
def isbigraph(V,E,V1,V2):  
    tv=(V==V1|V2)  
    tv=tv&(V1&V2==set({}))  
    for (u,v) in E:  
        tv=tv&(u in V1)&(v in V2)  
    return tv
```

```
import graph.graph as gg  
V1={'x1','x2','x3','x4','x5'}  
V2={'y1','y2','y3','y4','y5'}  
V={'x1','x2','x3','x4','x5','y1','y2','y3','y4','y5'}  
E={('x1','y1'),('x1','y2'),('x2','y2'),('x2','y3'),('x3','y2'),('x3','y5'),('x4','y3'),('x5','y3'),('x5','y4'),('x5','y5')}
```

True





完全二分图

- **定义11.4.2（完全二分图）**：设 $G = \langle V, E \rangle$ 是二分图， $V = V_1 \cup V_2$ ， $V_1 \cap V_2 = \emptyset$ ，并且 $|V_1| = m$ ， $|V_2| = n$ ，对于任意节点 $u \in V_1, v \in V_2$ ，有 $(u, v) \in E$ ，则称 $G = \langle V, E \rangle$ 是完全二分图，或者 (m, n) 完全二分图。
- **定理11.4.1**：设 $G = \langle V, E \rangle$ 是无向图， G 是二分图当且仅当 G 的所有圈的长度都是偶数。



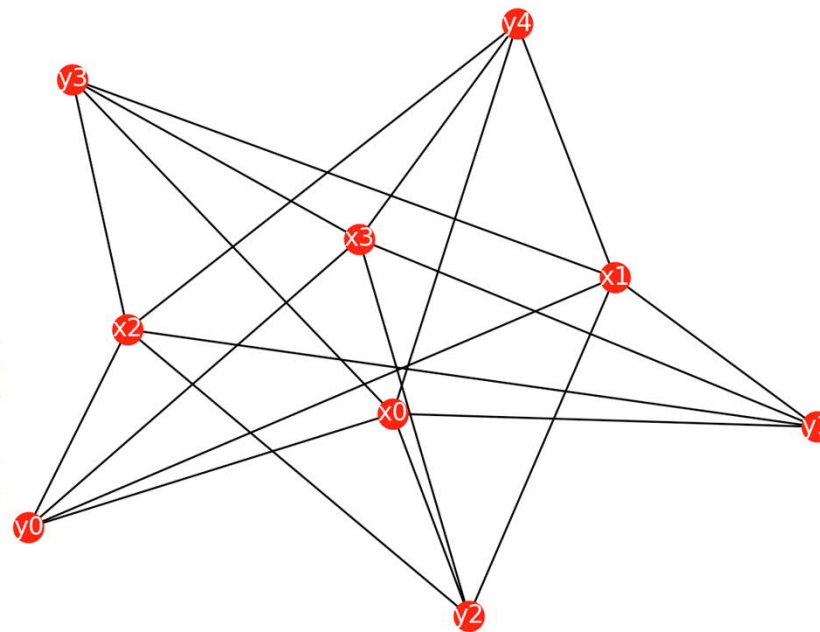
完全二分图

- completebigraph(m,n)生成 $m \times n$ 的完全二分图

```
def completebigraph(m,n):  
    V0=set({})  
    V1=set({})  
    for k in range(m):  
        V0=V0 | {'x'+str(k)}  
    for k in range(n):  
        V1=V1 | {'y'+str(k)}  
    E=set({})  
    for u in V0:  
        for v in V1:  
            E=E | {(u,v)}  
    return [V0,V1,E]
```

```
{'x3', 'x2', 'y0', 'y4', 'x1', 'y1', 'y'  
{('x2', 'y2'), ('x3', 'y2'), ('x2', 'y1'  
{('x1', 'y0'), ('x3', 'y0'), ('x0', 'y0'  
{('y4'), ('x2', 'y4'), ('x3', 'y4'), ('x3'  
{('x0', 'y2')}
```

```
import dmath.graph as gt  
[V1,V2,E]=gt.completebigraph(4,5)  
V=V1|V2  
print(V)  
print(E)  
gt.drawgraph(E)
```





匹配

- **定义11.4.3:** 设 $G = \langle V, E \rangle$ 是一个二分图, 如果 E 的子集 E' 中的任何两个边都不相邻, 则称 E' 为二分图 G 的**匹配**。
 - $E' \subseteq E$
 - $\forall e_1 \forall e_2 (e_1 \in E' \wedge e_2 \in E' \rightarrow \neg(e_1 | e_2))$



匹配

```
import graph.graph as gg
V1={'x1','x2','x3','x4','x5'}
V2={'y1','y2','y3','y4','y5'}
V={'x1','x2','x3','x4','x5','y1','y2','y3','y4','y5'}
E={('x1','y1'),('x1','y2'),('x2','y2'),('x2','y3'),('x3','y2'),('x3','y5'),('x4','y3'),('x5','y3'),('x5','y4'),('x5','y5')}
gg.drawgraph(E)
tv=gg.isbigraph(V,E,V1,V2)
E1=E-({'x1','y2'),('x5','y3'),('x5','y5'),('x2','y3'),('x3','y2'})
gg.drawgraph(E1)
print(tv)
```

True



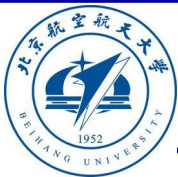
极（最）大匹配，完全匹配和完美匹配

- **定义11.4.4:** 设 $G = \langle V, E \rangle$ 是二分图， M 是匹配，若再加任意边，那么 M 不是匹配，则 M 是**极大匹配**。
- **定义11.4.5:** 设 $G = \langle V, E \rangle$ 是二分图，并且 M 是最大的极大匹配，则称 M 是**最大匹配**。
- **定义11.4.6:** 设 $G = \langle V, E \rangle$ 是二分图， $|V_1| \leq |V_2|$ ， M 为 G 中一个最大匹配，并且 $|M| = \min\{|V_1|, |V_2|\}$ ，则称 M 为从 V_1 到 V_2 的**完备匹配**。



匹配问题

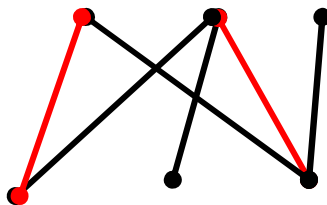
- **定义11.4.7:** 设 $G = \langle V, E \rangle$ 是二分图, M 为 G 中一个完备匹配, 并且 $|V_1| = |V_2|$, 则称 M 为**完美匹配**
- 匹配问题
 - 匹配 M 有最多匹配边是最大匹配问题。
 - 匹配 M 并且 $|M| = |V_1|$ 是完备匹配问题。
 - 匹配 M 并且 $|M| = |V_1| = |V_2|$ 是完美匹配问题。



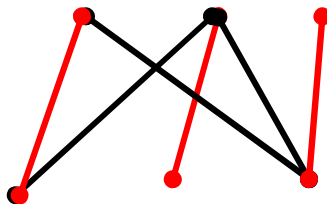
匹配问题

- 极大匹配与最大匹配示例：

- 极大匹配示例：



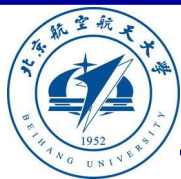
- 最大匹配显然都是极大匹配，但极大匹配不一定是最大匹配。下图是最大匹配的示例：





完备匹配定理

- **定理：** 设 $G = \langle V, E \rangle$ 是二分图，若 V_1 中每个节点的度数至少为 t ，并且 V_2 中每个节点的度数至多为 t ，则图 G 存在完备匹配。
- **定理11.4.2（Hall定理）：** 设 $G = \langle V, E \rangle$ 是二分图， $V = V_1 \cup V_2$ ， $V_1 \cap V_2 = \emptyset$ 。 G 存在完备匹配，当且仅当，对于任意 $S \subseteq V_1$ ， $N(S)$ 是 S 的相邻节点集合，有 $|S| \leq |N(S)|$ 。



完备匹配定理

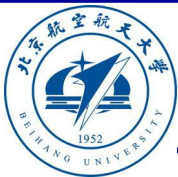
G 存在完备匹配，当且仅当，对于任意 $S \subseteq V_1$ ， $N(S)$ 是 S 的相邻节点集合，有 $|S| \leq |N(S)|$ 。

- **定理：** 设 $G = \langle V, E \rangle$ 是二分图，若 V_1 中每个节点的度数至少为 t ，并且 V_2 中每个节点的度数至多为 t ，则图 G 存在完备匹配。
- 对于任意 $S \subseteq V_1$ ， $N(S)$ 是 S 的相邻节点集合
- 令 $n = |S|$ 且 $m = |N(S)|$
- 对任意 S 中的任意一个节点 v ，都存在 V_2 中节点 v_2 与之相连
$$\sum_{v \in S} d(v) \leq \sum_{v \in N(S)} d(v)$$
- $t \times n \leq \sum_{v \in S} d(v) \leq \sum_{v \in N(S)} d(v) \leq t \times m$
- $n \leq m$
- 由Hall定理可以该定理成立



交替路径和增广路径

- **定义11.4.9** : 设 $v \in V$, 若有边 $e \in E'$, 使得 v 与 e 关联, 则称 v 是 E' 的**饱和节点**, 否则, 称 v 是 E' 的**非饱和节点**。
- **定义11.4.9**: 从一个未匹配点出发, 依次经过非匹配边、匹配边、非匹配边...形成的路径叫**交替路径**。
- **定义11.4.10**: 从一个未匹配点出发, 走交替路径, 如果途径另一个未匹配点 (出发的点不算), 则这条交替路径称为**增广路径**。



匈牙利算法

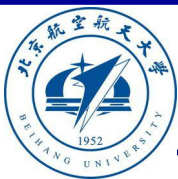
- 匈牙利数学家埃德蒙兹（Edmonds）于1965年提出匈牙利算法。匈牙利算法是基于Hall定理中充分性证明的思想，它是二分图匹配最常见的算法，该算法的核心就是寻找增广路径，它是一种用增广路径求二分图最大匹配的算法。

增广路径

- (1) P 的路径长度必定为奇数，第一条边和最后一条边都不属于 M 。
- (2) P 经过取反操作可以得到一个更大的匹配 M' 。
- (3) M 为 G 的最大匹配当且仅当不存在相对于 M 的增广路径。

算法

- (1) 置 M 为空。
- (2) 找出一条增广路径 P ，通过取反操作获得更大的匹配 M' 代替 M 。
- (3) 重复(2)操作直到找不出增广路径为止。



匈牙利算法

$G = \langle V_1, V_2, E \rangle$	$V_1 = \{x_1, x_2, x_3, x_4, x_5, x_6\}$ $V_2 = \{y_1, y_2, y_3, y_4, y_5, y_6, y_7\}$ $E = \{(x_1, y_1), (x_1, y_2), (x_1, y_4), (x_2, y_2), (x_2, y_5), (x_3, y_1), (x_3, y_4), (x_3, y_7), (x_4, y_3), (x_4, y_4), (x_4, y_6), (x_5, y_4), (x_6, y_4), (x_6, y_7)\}$
(x_1, y_1)	$M = \{(x_1, y_1)\}$
(x_2, y_2)	$M = \{(x_1, y_1), (x_2, y_2)\}$
(x_3, y_1, x_1, y_4)	$M = \{(x_1, y_4), (x_2, y_2), (x_3, y_1)\}$
(x_4, y_3)	$M = \{(x_1, y_4), (x_2, y_2), (x_3, y_1), (x_4, y_3)\}$
$(x_5, y_4, x_1, y_1, x_3, y_7)$	$M = \{(x_1, y_1), (x_2, y_2), (x_3, y_7), (x_4, y_3), (x_5, y_4)\}$

- 初始匹配集match。
- 从 V_1 依序检查非饱和节点 x ，若 $(x, y) \in E$ ，并且 y 是非饱和节点，则 (x, y) 加入match。
- 交替路径检查，存在增广路径，则生成新的match。



匈牙利算法

```
def saturatedV(path):
```

```
    Vs=set(path)
```

```
    return Vs
```

```
def unsaturatedV(V,Vs):
```

```
    Vu=V-Vs
```

```
    return Vu
```

```
def path2match(V1,V2,path):
```

```
    match=set({})
```

```
    n=len(path)
```

```
    for k in range(0,n,2):
```

```
        u=path[k]
```

```
        v=path[k+1]
```

```
        if u in V1:
```

```
            match=match|{(u,v)}
```

```
        else:
```

```
            match = match | {(v, u)}
```

```
    return match
```

```
def augmentpath(Vu,Eu,path):
```

```
    v0=path[0]
```

```
    un=path[-1]
```

```
    u0=-1
```

```
    vn=-1
```

```
    for (u,v) in Eu:
```

```
        if (v==v0) and (u in Vu):
```

```
            u0=u
```

```
        if (u == v0) and (v in Vu):
```

```
            u0=v
```

```
    for (u,v) in Eu:
```

```
        if (v == un) and (u in Vu):
```

```
            vn=u
```

```
        if (u == un) and (v in Vu):
```

```
            vn=v
```

```
    if (u0!=-1) and (vn!=-1):
```

```
        path=[u0]+path+[vn]
```

```
    return path
```



匈牙利算法

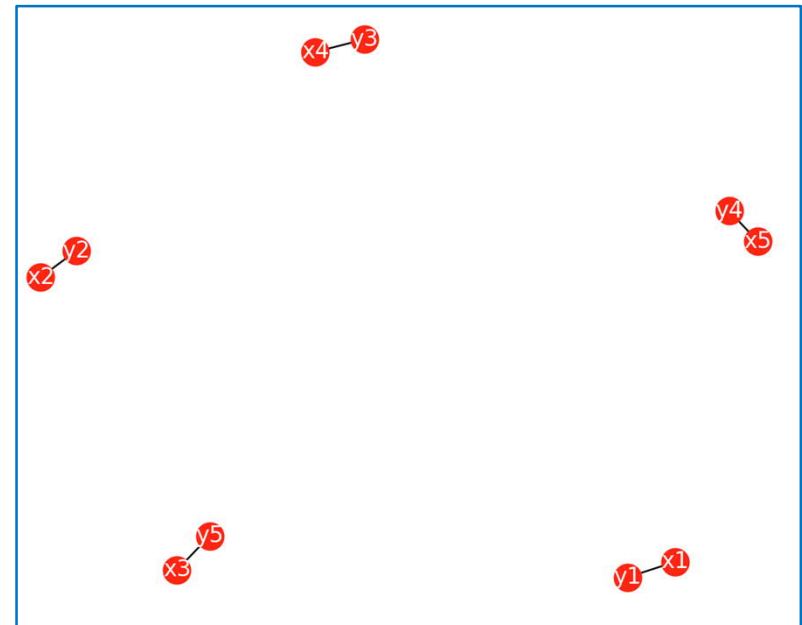
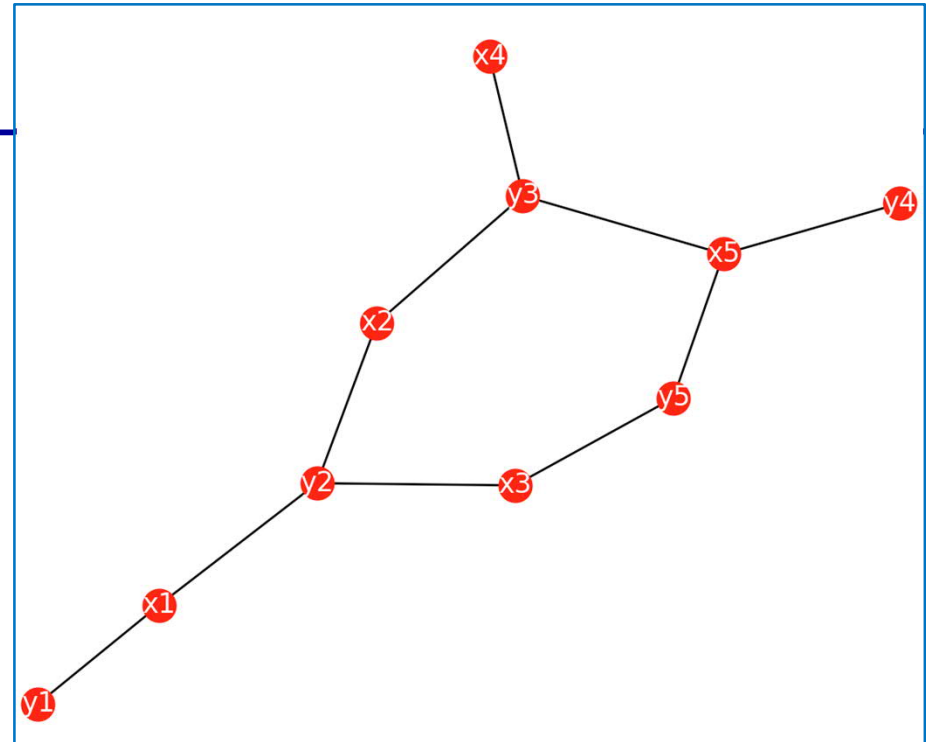
```
def augmentpaths(V,E,path):  
    Vs=saturatedV(path)  
    Vu=unsaturatedV(V,Vs)  
    Eu=E  
    m=0  
    while len(path)!=m:  
        m=len(path)  
        path=augmentpath(Vu,Eu,path)  
        Vs=saturatedV(path)  
        Vu = unsaturatedV(V, Vs)  
    return path
```

```
def matchmaxset(V1,V2,E,path):  
    Vu=V1|V2  
    Eu=E  
    match=set({})  
    matchset=set({})  
    Vs=set({})  
    m=0  
    while len(path) != m:  
        m=len(path)  
        path=augmentpaths(Vu,Eu,path)  
        match = path2match(V1, V2, path)  
        matchset=matchset|match  
        Vs=Vs|saturatedV(path)  
        Eu=E  
        for (u,v) in E:  
            if (u in Vs) or (v in Vs):  
                Eu=Eu-{(u,v)}  
        if Eu==set({}):  
            break  
        for (u,v) in Eu:  
            break  
        match=set({})  
        path=[u,v]  
        m=0  
    return matchset
```



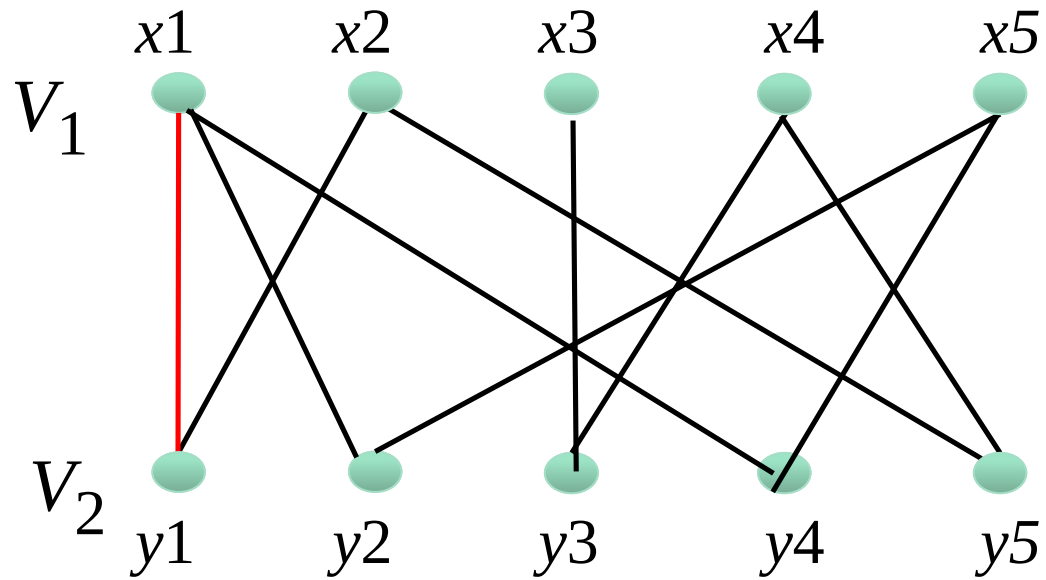
匈牙利算法

```
import graph.graph as gg
V1={'x1','x2','x3','x4','x5'}
V2={'y1','y2','y3','y4','y5'}
V=V1|V2
E={('x1','y1'),('x1','y2'),('x2','y2'),('x2','y3'),
   ('x3','y2'),('x3','y5'),('x4','y3'),('x5','y3'),('x5','y4'),
   ('x5','y5')}
#[V1,V2,E]=gg.completebigraph(10,10)
gg.drawgraph(E)
path=['x1','y1']
matchset=gg.matchmaxset(V1,V2,E,path)
gg.drawgraph(matchset)
print(matchset)
{('x2', 'y2'), ('x5', 'y4'), ('x3', 'y5'), ('x4', 'y3'), ('x1', 'y1')}
```



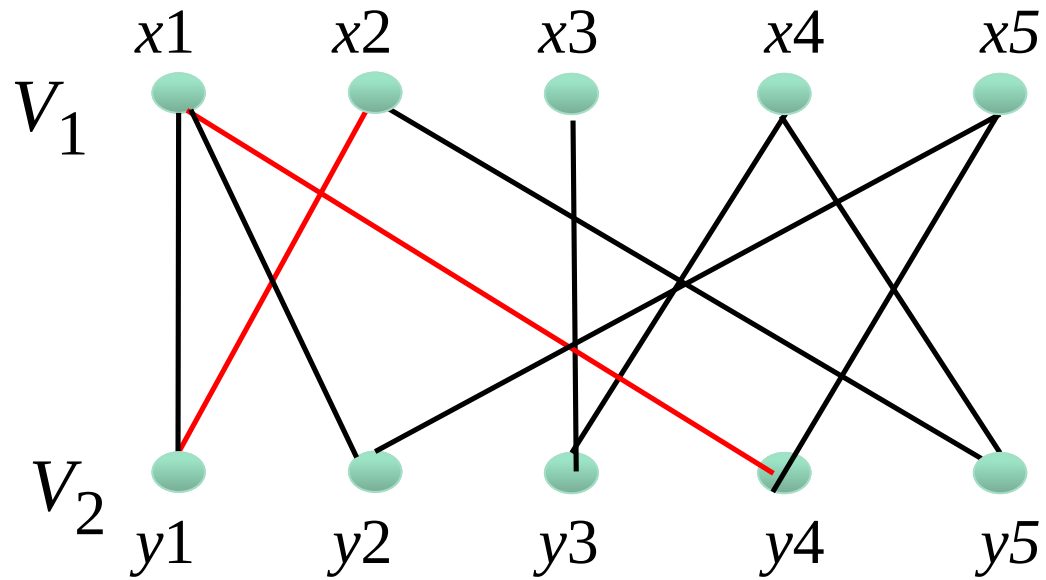


最大匹配



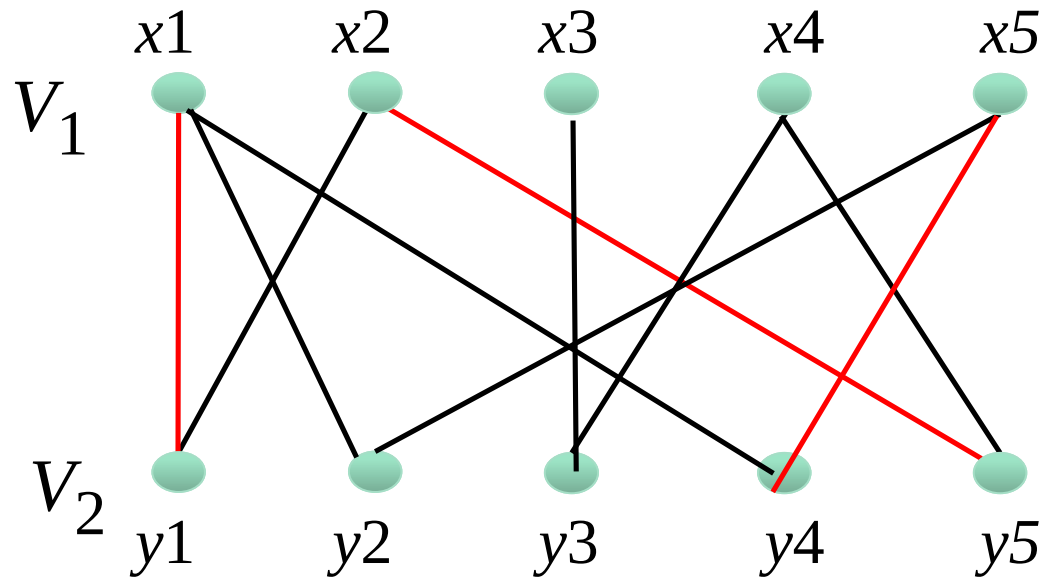


最大匹配



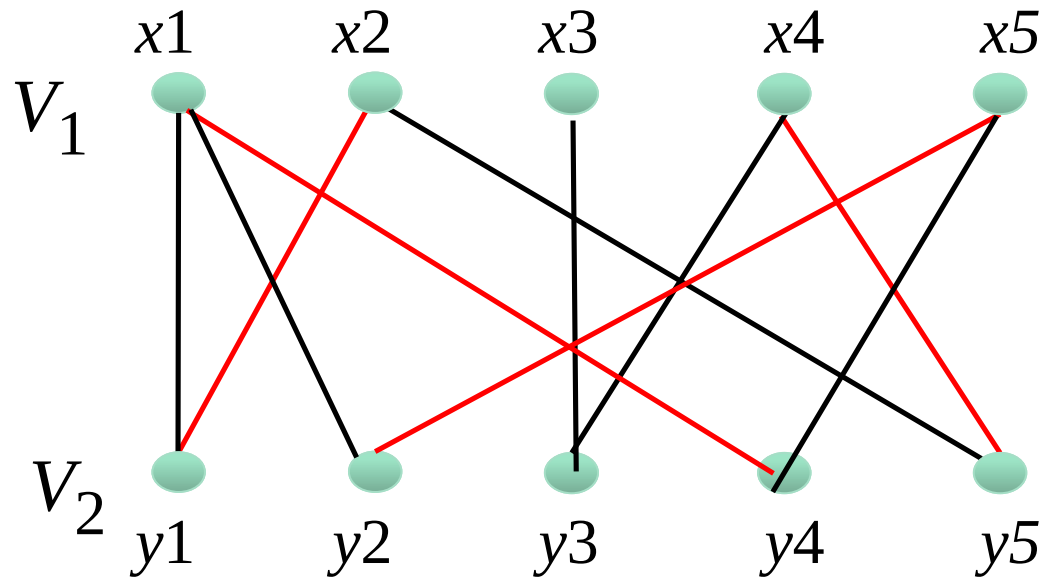


最大匹配



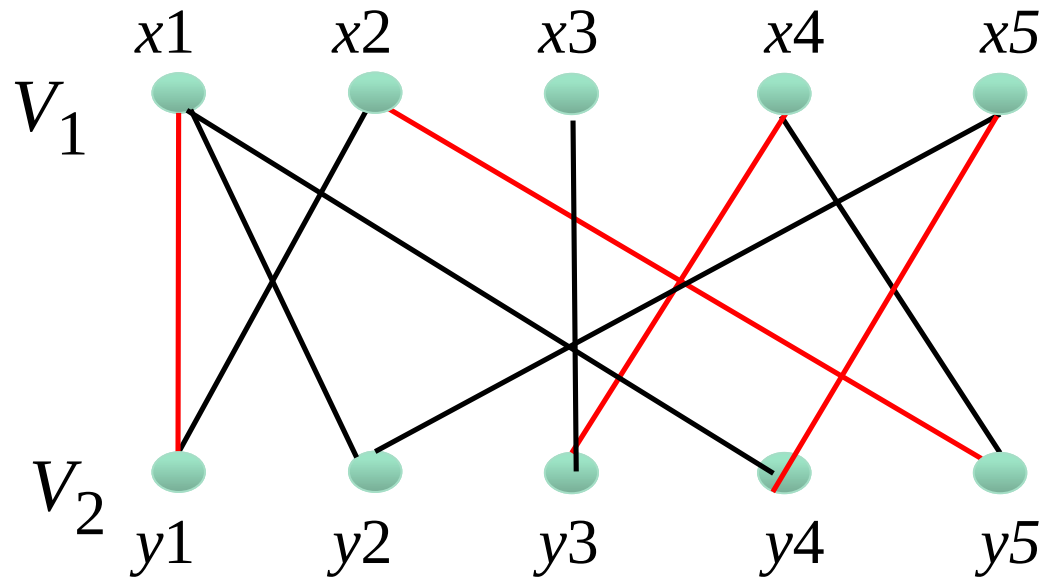


最大匹配



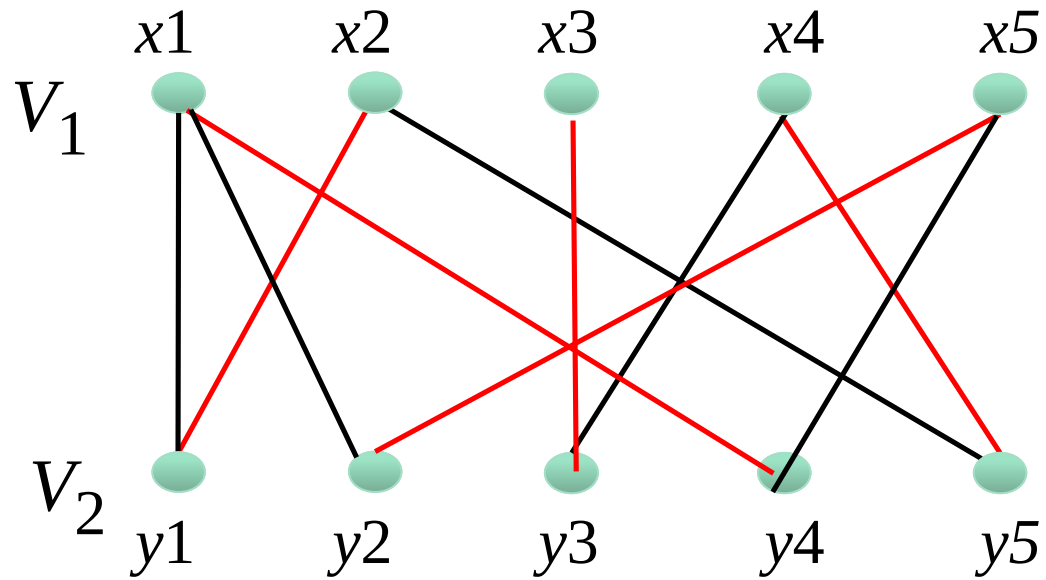


最大匹配





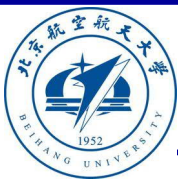
最大匹配



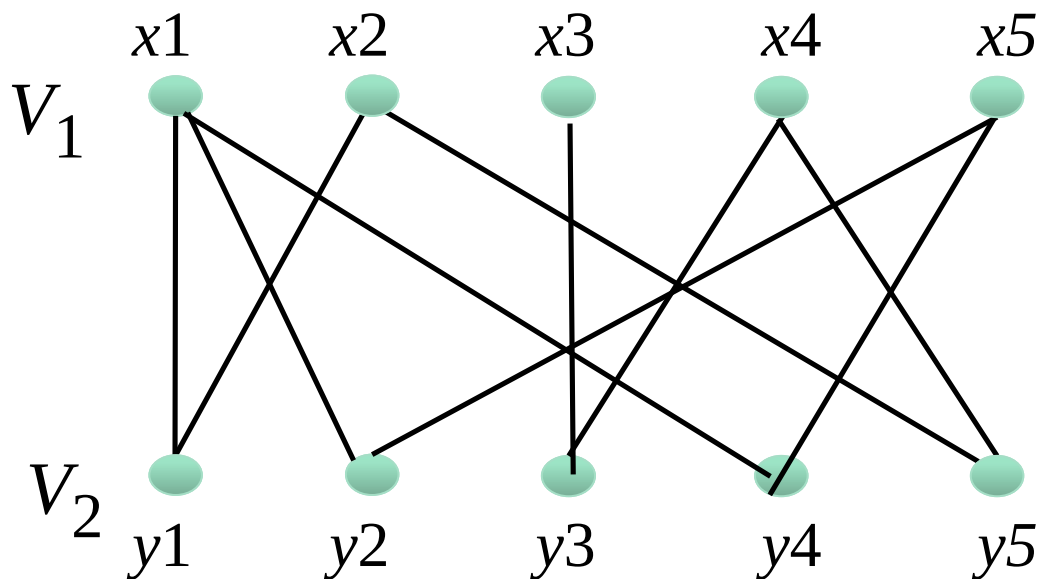


最大匹配

- 匈牙利算法的时间复杂度为 $O(|V||E|)$
- John Hopcroft和Richard Karp于1973年时间复杂度为 $O(|E|\sqrt{|V|})$ 的Hopcroft-Karp算法
- 基本思路：BFS地寻找增广路径

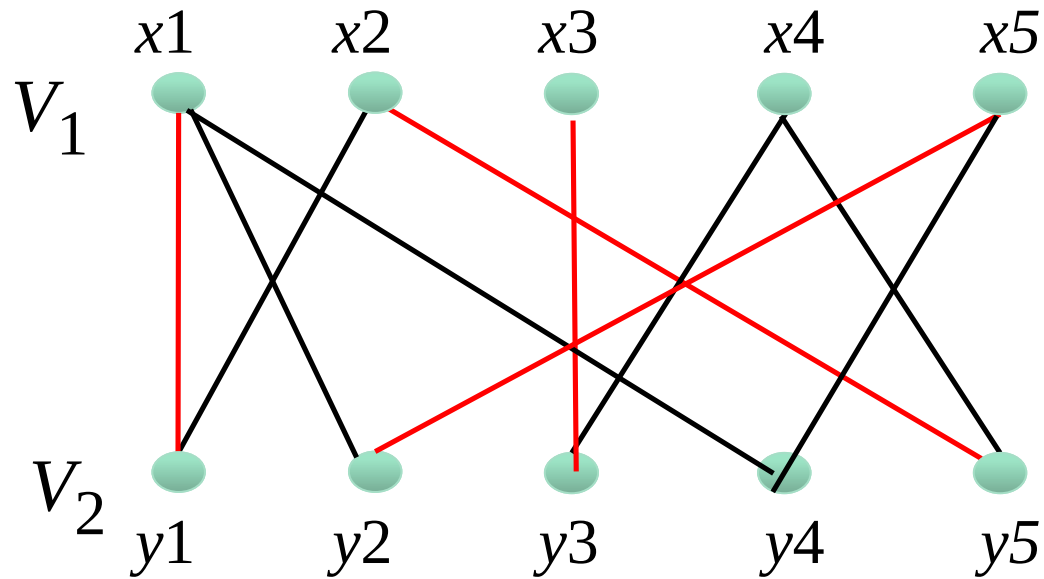


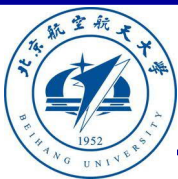
最大匹配(霍普克洛夫特-卡普算法)



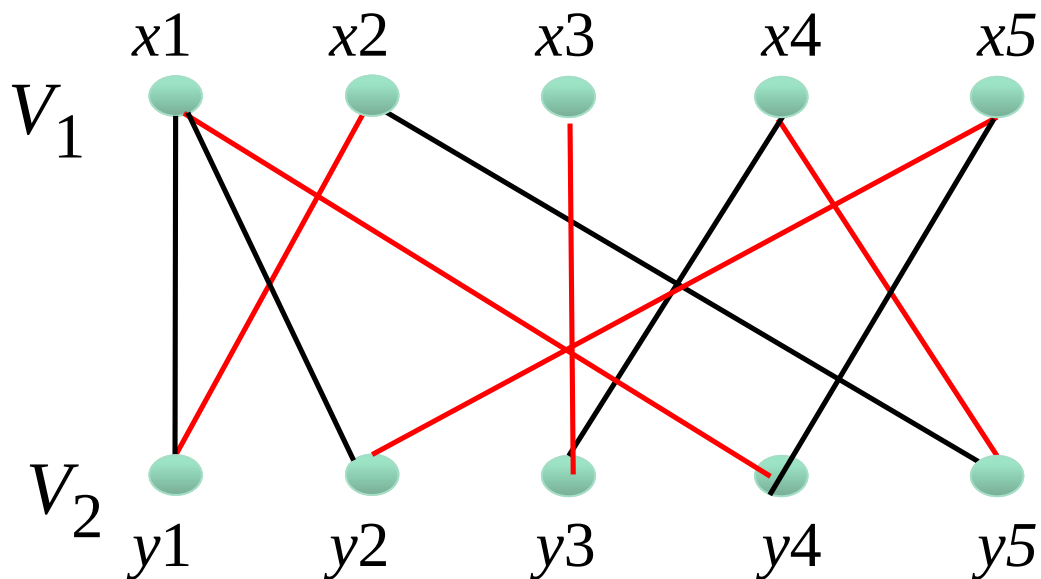


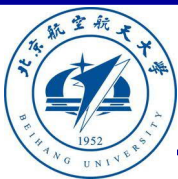
最大匹配(霍普克洛夫特-卡普算法)





最大匹配(霍普克洛夫特-卡普算法)





商品推荐

	1	2	3	4
A		4.5	2.0	0.32
B	4.0		3.5	
C		5.0		2.0
D		3.5	4.0	1.0

A	2.21	3.80	2.03	1.52
B	3.93	2.37	3.42	0.57
C	3.19	4.83	2.90	1.88
D	4.3	3.40	3.79	1.01

	1	2	3	4
A		4.5	2.0	
B	4.0		3.5	
C		5.0		2.0
D		3.5	4.0	1.0

$$\approx \begin{array}{|c|c|} \hline \text{用户矩阵Q} \\ \hline \begin{array}{|c|c|} \hline A \begin{array}{c} \text{User A} \end{array} \begin{array}{|c|c|} \hline 1.40 & -1.18 \\ \hline \end{array} \\ \hline B \begin{array}{c} \text{User B} \end{array} \begin{array}{|c|c|} \hline 1.62 & 0.51 \\ \hline \end{array} \\ \hline C \begin{array}{c} \text{User C} \end{array} \begin{array}{|c|c|} \hline 1.88 & -1.31 \\ \hline \end{array} \\ \hline D \begin{array}{c} \text{User D} \end{array} \begin{array}{|c|c|} \hline 1.93 & 0.07 \\ \hline \end{array} \\ \hline \end{array} \times \begin{array}{|c|c|c|c|} \hline \begin{array}{|c|c|c|c|} \hline 1 & 2 & 3 & 4 \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|} \hline \begin{array}{c} \text{Book 1} \end{array} \begin{array}{c} \text{Book 2} \end{array} \begin{array}{c} \text{Book 3} \end{array} \begin{array}{c} \text{Book 4} \end{array} \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|} \hline 2.19 & 1.80 & 1.93 & 0.54 \\ \hline 0.73 & -1.08 & 0.56 & -0.64 \\ \hline \end{array} \\ \hline \text{物品矩阵P} \\ \hline \end{array}$$



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- 当 $n = 3$ 时， G 至少还有3条；显然 G 有哈密顿回路



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条边，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- 存在一个度数至少为 $n - 2$ 的点 v
- 否则的话，每个点的度数不超过 $n - 3$
- 那么边数最多只有 $\frac{n(n-3)}{2} < \frac{(n-1)(n-2)}{2} + 2$
- 如果 v 的度数是 $n - 2$ ，那么把 v 从图中去掉，剩余子图含有 $n - 1$ 个点以及至少 $\frac{(n-2)(n-3)}{2} + 2$ 条边



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条边，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- 由归纳假设，子图中有哈密顿回路： $v_1 v_2 \dots v_{n-1} v_1$ 。
- 从这 $n - 1$ 个点里任取两个与 v 相连的，比如 v_i 与 v_j ，那么原图的哈密顿回路就是 $v_1 v_2 \dots v_i v v_j \dots v_{n-1} v_1$



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条边，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- 如果 v 的度数是 $n - 1$ ，那么把 v 从图中去掉，剩余子图有至少 $\frac{(n-2)(n-3)}{2} + 1$ 条边
- 在子图里任意加一条边，使得边数达到 $\frac{(n-2)(n-3)}{2} + 2$
- 用归纳假设，得到一个哈密顿回路



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- 如果哈密顿回路中不包含新加 $v_i v_j$ 的边，用以上的方法构造新哈密顿回路
- 如果包含，那么把这条 $v_i v_j$ 边替换成两条 $v_i v v_j$ （ v 的度数是 $n - 1$ ）



练习题

- 假设图 G 是具有 $n \geq 3$ 个节点的无向简单图。若 G 至少还有 $\frac{(n-1)(n-2)}{2} + 2$ 条边，则 G 中存在一条哈密顿回路。当 $n \geq 3$ ，证明存在 G 有 $\frac{(n-1)(n-2)}{2} + 1$ 条边，但 G 没有哈密顿回路。
- $n = 3$ ， G 有 $\frac{(n-1)(n-2)}{2} + 1 = 2$ 条边，显然没有哈密顿回路

谢谢

